
Aplicación para la navegación en colecciones de
contenidos multimedia
Application for Browsing Multimedia Content
Collections



Trabajo de Fin de Grado
Curso 2025–2026

Autores

Juan Andrés Hibjan Cardona
Leonardo Prado de Souza

Director

José Luis Sierra Rodríguez

Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid

Aplicación para la navegación en
colecciones de contenidos multimedia
Application for Browsing Multimedia
Content Collections

Trabajo de Fin de Grado en Ingeniería Informática

Autor

Juan Andrés Hibjan Cardona
Leonardo Prado de Souza

Director

José Luis Sierra Rodríguez

Convocatoria: *Junio 2026*

Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid

10 de Junio de 2026

Dedicatoria

...

Agradecimientos

...

Resumen

Aplicación para la navegación en colecciones de contenidos multimedia

El acúmulo de contenidos en las plataformas digitales contemporáneas ha sobrepasado la capacidad de los mecanismos clásicos de búsqueda y filtrado para acompañar al usuario en su exploración. La búsqueda directa exige conocer de antemano lo que se busca, el filtrado por categorías opera sobre taxonomías rígidas, y los sistemas de recomendación restan agencia al usuario al ocultar los criterios sobre los que operan. Este Trabajo de Fin de Grado propone un motor de exploración de colecciones digitales que combina tres mecanismos complementarios: filtrado facetado por metadatos, navegación transversal entre colecciones relacionadas mediante referencias semánticas y operaciones de conjuntos sobre los resultados parciales.

El sistema se articula como una arquitectura cliente-servidor. El núcleo de navegación se ha desarrollado en Java sobre una base de datos relacional, exponiendo sus capacidades a través de una *API RESTful*. La interfaz web, desarrollada en JavaScript, da al usuario acceso interactivo a los filtros, a los enlaces entre colecciones y al conjunto unión, que conserva los resultados guardados durante la sesión. La validación funcional se ha realizado sobre volcados de dos catálogos públicos de naturaleza distinta, TMDb y DBLP, y la usabilidad de la interfaz se ha evaluado mediante un protocolo formal de pruebas con usuarios.

Palabras clave

Navegación facetada, exploración de colecciones, hipermedia, filtrado relacional, operaciones de conjuntos, motor de navegación, ingeniería del software, evaluación de usabilidad, TMDb, DBLP.

Abstract

Application for Browsing Multimedia Content Collections

An abstract in English, half a page long, including the title in English. Below, a list with no more than 10 keywords.

Keywords

10 keywords max., separated by commas.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	2
1.3.1. Fase 1: Investigación y Prototipo en Java (Septiembre 2025 - Enero 2026)	2
1.3.2. Fase 2: Desarrollo Web e Integración (Enero 2026 - Marzo 2026)	3
1.3.3. Cronograma de Ejecución y Diagrama Gantt	4
2. Estado de la Cuestión	5
2.1. Marco teórico	5
2.1.1. Navegación basada en etiquetas de grandes colecciones de objetos	6
2.1.2. Modelos de hipertexto y navegación estructurada	7
2.2. Trabajos relacionados	10
2.2.1. Clavy	10
2.2.2. Knowledge Navigator y la exploración asistida por modelos del lenguaje	13
3. Arquitectura y Metodología	15
3.1. Motor de navegación	15
3.1.1. Del prototipo al modelo formal	16
3.1.2. Modelo formal de navegación	17
3.2. Metodología de Ingeniería del Software	21
3.3. Arquitectura del Sistema	23
3.4. Persistencia y Acceso a Datos	26
4. Desarrollo e Implementación	29
4.1. Modelo de datos	29
4.1.1. Modelo Entidad-Relación	30
4.1.2. Modelo relacional	30
4.2. Representación del contexto de exploración	32

4.2.1.	La clase State : filtros y contexto acumulado	33
4.2.2.	StateManager : gestión del ciclo de vida del estado	34
4.2.3.	API REST: endpoints expuestos al cliente	35
4.3.	Filtrado clásico y relacional	36
4.3.1.	Construcción dinámica de queries SQL	37
4.3.2.	Filtros de metadatos	37
4.3.3.	Filtros de referencia	38
4.3.4.	Facetas: cómputo dinámico de valores disponibles	39
4.4.	Navegación transversal	40
4.4.1.	El mecanismo de link y goback	40
4.4.2.	Subconsultas anidadas: traducción del historial de <i>links</i> a SQL	42
4.5.	Operaciones de conjuntos	44
4.5.1.	Construcción del unionSet	44
4.5.2.	Resolución de la unión en base de datos	46
4.6.	Desarrollo del frontend	46
4.7.	Ejemplo ilustrativo (recorrido end-to-end)	48
4.7.1.	Mini-dataset reproducible	49
4.7.2.	Vista esquemática del dataset	51
4.7.3.	Estado inicial	52
4.7.4.	Flujo 1: acotación dentro de una colección	53
4.7.5.	Flujo 2: navegación entre colecciones	57
4.7.6.	Flujo 3: unión de contextos independientes	61
4.8.	Obstáculos encontrados y soluciones implementadas	64
5.	Pruebas y Validación	69
5.1.	Validación funcional sobre TMDb y DBLP	69
5.2.	Evaluación de usabilidad con usuarios	70
5.2.1.	Objetivos del estudio	70
5.2.2.	Metodología	70
5.2.3.	Perfil de los participantes	71
5.3.	Resultados cuantitativos	71
5.3.1.	Tasa de éxito y completitud	71
5.3.2.	Tiempo por tarea	72
5.3.3.	Facilidad percibida (SEQ)	73
5.3.4.	Satisfacción global (SUS)	73
5.4.	Hallazgos cualitativos y problemas detectados	74
5.5.	Iteraciones de la interfaz derivadas de la evaluación	76
5.6.	Casos de uso validados	76
5.7.	Limitaciones del estudio	77
6.	Conclusiones y Trabajo Futuro	79
6.1.	Conclusiones	79
6.2.	Limitaciones	80
6.3.	Trabajo futuro	80

6.3.1.	Asistencia mediante modelos del lenguaje	80
6.3.2.	Distribución del motor sobre múltiples nodos	81
6.3.3.	Capa de usuario, persistencia e historial compartido	81
6.3.4.	Extensión a nuevos dominios y datasets	81
6.3.5.	Mejoras incrementales en la experiencia de usuario	82
Introduction		83
Conclusions and Future Work		85
Contribuciones Personales		87
Bibliografía		89
A. Guion de Evaluación de Usabilidad		91
A.1.	Información general	91
A.2.	Información para el facilitador	91
A.2.1.	Objetivos	91
A.2.2.	Perfil del participante	91
A.2.3.	Material necesario	92
A.2.4.	Checklist previo a la sesión	92
A.2.5.	Protocolo general	92
A.3.	Introducción al participante (texto literal)	92
A.4.	Consentimiento informado	93
A.5.	Cuestionario previo (5 minutos)	93
A.6.	Instrucciones sobre el método (2 minutos)	94
A.7.	Tareas	94
A.7.1.	Tarea 1 – Empezar a navegar (límite: 2 min)	94
A.7.2.	Tarea 2 – Filtrar por metadatos (límite: 3 min)	94
A.7.3.	Tarea 3 – Combinar varios filtros (límite: 3 min)	94
A.7.4.	Tarea 4 – Filtros de exclusión (límite: 3 min)	95
A.7.5.	Tarea 5 – Filtros de referencia (límite: 3 min)	95
A.7.6.	Tarea 6 – Quitar filtros (límite: 2 min)	95
A.7.7.	Tarea 7 – Seguir un <i>link</i> a otra colección (límite: 2 min)	95
A.7.8.	Tarea 8 – Ver el historial de navegación (límite: 2 min)	96
A.7.9.	Tarea 9 – Volver atrás un paso (límite: 1 min)	96
A.7.10.	Tarea 10 – Ver el detalle de una entidad (límite: 2 min)	96
A.7.11.	Tarea 11 – Guardar en <i>Union</i> y cambiar de colección (límite: 4 min)	96
A.7.12.	Tarea 12 – Reordenar paneles (límite: 2 min)	97
A.7.13.	Tarea 13 – Cambiar de colección (sin <i>Union</i>) (límite: 2 min) .	97
A.7.14.	Tarea 14 – Salir del modo navegación (límite: 1 min)	97
A.8.	Cuestionario posterior (SUS, 5 minutos)	97
A.9.	Preguntas abiertas de cierre (10 minutos)	98

A.10.Cierre y agradecimiento	98
A.11.Plantilla de observación del facilitador	99
A.11.1.Datos de la sesión	99
A.11.2.Registro por tarea	99
A.11.3.Incidencias destacables y citas literales	99
A.11.4.Conclusiones del facilitador	99
A.12.Notas metodológicas	99
A.12.1.Número de participantes	99
A.12.2.Perfil equilibrado recomendado	99
A.12.3.Análisis posterior	100
A.12.4.Criterios de éxito por tarea	100
A.12.5.Variables a controlar entre sesiones	100
B. Integración e ingesta de fuentes de datos	101
B.1. Pipeline general	101
B.2. TMDb	102
B.3. DBLP	104
B.4. Ingesta en PostgreSQL	104
B.5. Ejecución completa desde cero	105
C. Despliegue del sistema	107
C.1. Modo de desarrollo	107
C.2. Instancia compartida con túnel Cloudflare	108
C.3. Base de datos e ingesta inicial	109

Índice de figuras

1.1. Diagrama Gantt TFG	4
2.1. Capas del modelo Dexter	9
2.2. Autómata de navegación de Clavy	11
2.3. Flujo de Knowledge Navigator	13
3.1. Captura del backlog del proyecto en GitHub Projects.	22
3.2. Arquitectura general del sistema en capas.	23
4.1. Diagrama Entidad-Relación del modelo de datos persistente.	30
4.2. Modelo relacional implementado en PostgreSQL.	31
4.3. Diagrama UML del modelo de exploración	33
4.4. Ejemplo de objeto State	34
4.5. Diagrama de secuencia de link	41
4.6. Diagrama de secuencia de goback	42
4.7. Diagrama de secuencia de union	45
4.8. Fichas por entidad del mini-dataset	52
4.9. Estado inicial del frontend	53
4.10. Flujo 1, Paso 1.1	54
4.11. Flujo 1, Paso 1.2	54
4.12. Flujo 1, Paso 1.3	55
4.13. Estado final del Flujo 1	55
4.14. Flujo 2, Paso 2.1	57
4.15. Flujo 2, Paso 2.2	58
4.16. Flujo 2, Paso 2.3	58
4.17. Estado final del Flujo 2	59
4.18. Flujo 3, Paso 3.1	61
4.19. Flujo 3, Paso 3.2	62
4.20. Flujo 3, Paso 3.3	62
4.21. Flujo 3, Paso 3.4	63
4.22. Estado final del Flujo 3	63
4.23. Flujo de una petición HTTP	67

5.1. Distribución de la completitud por tarea	72
5.2. Facilidad percibida (SEQ) por tarea	73
5.3. Puntuación SUS por participante	74
C.1. Topología de la instancia compartida con túnel Cloudflare.	109

Índice de tablas

2.1. Correspondencia HAM	8
2.2. Comparativa Clavy	12
4.1. Endpoints REST expuestos por el backend.	36
4.2. Tabla datasets del mini-dataset.	49
4.3. Tabla collections del mini-dataset	49
4.4. Tabla entities — <i>Movies</i>	49
4.5. Tabla entities — <i>People</i>	50
4.6. Tabla entities — <i>Studios</i>	50
4.7. Tabla metadata del mini-dataset	50
4.8. Tabla reference del mini-dataset	51
5.1. Perfil demográfico de los participantes	71
5.2. Tiempos y éxitos por tarea	72
5.3. Catálogo de incidencias detectadas	74
5.4. Resumen de casos de uso validados	77

Índice de códigos

4.1. Filtros de metadatos	38
4.2. Filtros de referencia	39
4.3. Cómputo dinámico de facetas	39
4.4. Operación <code>link</code>	40
4.5. Operación <code>goback</code>	42
4.6. Subconsultas anidadas	43
4.7. Query SQL de la navegación transversal	44
4.8. Operación <code>union</code>	45
4.9. Query de unión	46
4.10. SQL del Flujo 1	56
4.11. SQL del Flujo 2	60
4.12. SQL del Flujo 3	64
4.13. Configuración del filtro por popularidad	65
4.14. Función <code>bucket_range</code>	65
4.15. Filtro <code>CORSFilter</code>	66
B.1. Esquema del JSONL intermedio	102
B.2. Ejecución del pipeline completo	105

Introducción

“Sólo hay una victoria decisiva: la última”

— Carl Von Clausewitz

El presente Trabajo de Fin de Grado aborda el diseño y la implementación de un motor de exploración de colecciones digitales que combina filtrado facetado por metadatos, navegación transversal entre colecciones relacionadas y operaciones de conjuntos sobre los resultados. La propuesta nace en respuesta a un problema concreto del consumo de información digital contemporáneo, que motivamos a continuación, y persigue devolver al usuario una herramienta que combine la precisión de la búsqueda directa, la flexibilidad del filtrado por etiquetas y la riqueza expresiva de la navegación hipermedia.

Este capítulo introductorio se organiza en tres bloques: la motivación que originó el trabajo, los objetivos perseguidos y el plan de ejecución del proyecto.

1.1. Motivación

En los últimos tiempos hemos pasado por un gran cambio en el paradigma de consumo digital. El acúmulo cuantitativo de contenidos digitales ha sobrepasado su umbral cualitativo y hoy supone una manera distinta de relacionarse con ello. La abundancia de información genera una "sobrecarga de información" (o *information overload*), en la cual todo y cualquier usuario necesita de una cantidad cada vez mayor de tiempo para navegar. Eso se hace especialmente evidente en las plataformas digitales, en particular las de streaming multimedia (como Netflix, Disney+ y HBO Max), en las que difícilmente encontramos qué consumir sin invertir un tiempo sustancial en ello.

Ese cambio de paradigma, y el subsecuente problema, sigue siendo abordado con métodos tradicionales como: la búsqueda directa (por palabras clave), el filtrado clásico por género o medio y los sistemas de recomendación basados en algoritmos ocultos al consumidor. Estos abordajes son limitados y sirven poco al nuevo contexto de consumo digital. La búsqueda directa es determinista y no permite ninguna flexibilidad, especialmente al suponer que el usuario sabe exactamente lo que está buscando de antemano. El filtrado tradicional suele ser incompleto e inconsistente, usando taxonomías ocultas e inescrutables. Los algoritmos de recomendación suelen

servir más como un propagador de la burbuja de filtros (o *filter bubble*), al depender directamente del historial de consumo, o de lo que la plataforma desee promover, quitando la agencia del usuario en última instancia.

Este problema, entretanto, no es exclusivo de experiencia de usuario, sino también supone un problema tecnológico relacionado con la forma en la que se diseñan y utilizan las colecciones de información. Los modelos relacionales de navegación y los sistemas de navegación sobre colecciones (*collection navigation systems*), traen un enfoque distinto en el que los datos no se representan como elementos aislados, sino como entidades conectadas mediante relaciones semánticas. Ese abordaje permite recoger colecciones de manera interactiva, con mecanismos de navegación más expresivos en los que el usuario puede explorar progresivamente una colección compleja combinando filtros, relaciones y transiciones entre entidades.

Existe, por lo tanto, una necesidad declarada de que los sistemas de navegación acompañen ese cambio de paradigma. Eso solo se puede cumplir permitiendo a los usuarios herramientas de exploración interactiva más ricas y flexibles. Dichas herramientas deben combinar la precisión establecida por la búsqueda directa, pero con una mayor expresividad que el filtrado clásico sin quitar la agencia del usuario.

1.2. Objetivos

Descripción de los objetivos del trabajo.

1.3. Plan de trabajo

Hemos optado por una metodología ágil, basada en su mayor parte en el marco de trabajo **Scrum**, adaptada a un proyecto en pareja. La adopción de ese método viene dada por la naturaleza exploratoria e innovadora del proyecto, ya que los requisitos y el motor de navegación necesitaban un refinamiento y optimización iterativos.

El desarrollo fue organizado mediante *sprints* (ciclos de trabajo entre 2 y 3 semanas), definiendo un backlog inicial (*Product Backlog*) con los requisitos fundamentales del sistema. Además, se realizarían reuniones periódicas de seguimiento (*Sprint Reviews*), que permiten evaluar cada incremento del software, adaptar la planificación de acuerdo con los problemas y obstáculos encontrados, en conjunto con la tutoría del profesor.

Con respecto a la macroplanificación, el proyecto se estructuró en dos grandes fases secuenciales de desarrollo:

1.3.1. Fase 1: Investigación y Prototipo en Java (Septiembre 2025 - Enero 2026)

Con el objetivo de validar completa y rigurosamente la viabilidad del motor de navegación y los algoritmos involucrados, se optó por desarrollar un prototipo en Java. Deliberadamente abandonando la implementación inicial de una interfaz gráfica, hemos centralizado los esfuerzos en la eficacia, eficiencia y robustez técnica

de los componentes del sistema. Este enfoque permitió analizar en profundidad el comportamiento del algoritmo planteado y la correcta integración con la base de datos TMDB.

- **Estudio precedente y proceso de diseño arquitectónico:** Es ese momento hemos definido el estado de la cuestión basándonos en artículos de investigación previos, seleccionamos los datasets pertinentes y de dominio público y diseñamos el modelo básico de datos relacional.
- **Desarrollo del motor de navegación (núcleo backend):** Siguiendo con la implementación de la lógica en Java, desarrollamos una máquina de estados (`State`, `StateManager`) con capacidad para: gestionar el historial, los filtros de metadatos, los filtros relacionales y los saltos transversales `Link`.
- **Integración de TMDB:**

Para empezar la validación del motor de navegación, hemos optado por integrar la base de datos pública TMDB (*The Movie Database*), que permite obtener información sobre películas, series y personas. Además de consumir la API, se diseñó un proceso de transformación y almacenamiento de los datos en nuestro modelo relacional, permitiendo adaptar la estructura de TMDB a las necesidades de navegación del motor.

1.3.2. Fase 2: Desarrollo Web e Integración (Enero 2026 - Marzo 2026)

Con el núcleo del modelo de navegación funcional, hemos trasladado el esfuerzo en exponer sus funcionalidades a través de una arquitectura cliente-servidor, además de añadir una interfaz de usuario.

- **Exposición de API RESTful:** Hemos desarrollado una capa de controladores usando Servlets de Java (*API RESTful*) para exponer, con seguridad y de manera estructurada, las capacidades del motor (`FacetsServlet`, `NavigationServlet`, `UnionServlet`, etc.) que gestiona el contexto mediante sesiones de usuario.
- **Desarrollo del Frontend:**

Trás definir la API, se desarrolló una interfaz web para la interacción con el motor de navegación. Entre sus principales componentes están: el área de resultados, sistema de filtrado dinámico de metadatos, controles de navegación de estados, conjunto de controles de filtros y exploración de relaciones. Todo desde el paradigma de API RESTful con actualización dinámica.
- **Pruebas de Integración y Refinamiento:** Unimos todos los componentes, hemos hecho pruebas de carga con los *datasets* completos y evaluado la experiencia de usuario (*UX*).

1.3.3. Cronograma de Ejecución y Diagrama Gantt

- **Septiembre 2025:** Definición del proyecto, revisión de la literatura (estado del arte) y selección de datasets.
- **Octubre 2025:** Diseño del modelo de datos y desarrollo inicial del motor de navegación (Fase 1).
- **Noviembre 2025:** Implementación completa de la máquina de estados e historial de navegación.
- **Diciembre 2025:** Prototipado de la integración del dataset TMDb y cierre de la Fase 1.
- **Enero 2026:** Diseño e implementación de la API RESTful (controladores y enrutamiento) (Fase 2).
- **Febrero 2026:** Desarrollo del cliente web (Frontend) e integración con el backend.
- **Marzo 2026:** Pruebas de integración, resolución de bugs (obstáculos técnicos) y validación de casos de uso.
- **Abril 2026:** Redacción intensiva de la memoria del Trabajo de Fin de Grado.
- **Mayo 2026:** Revisiones finales de la memoria, preparación de la presentación, ensayo y defensa ante el tribunal universitario.

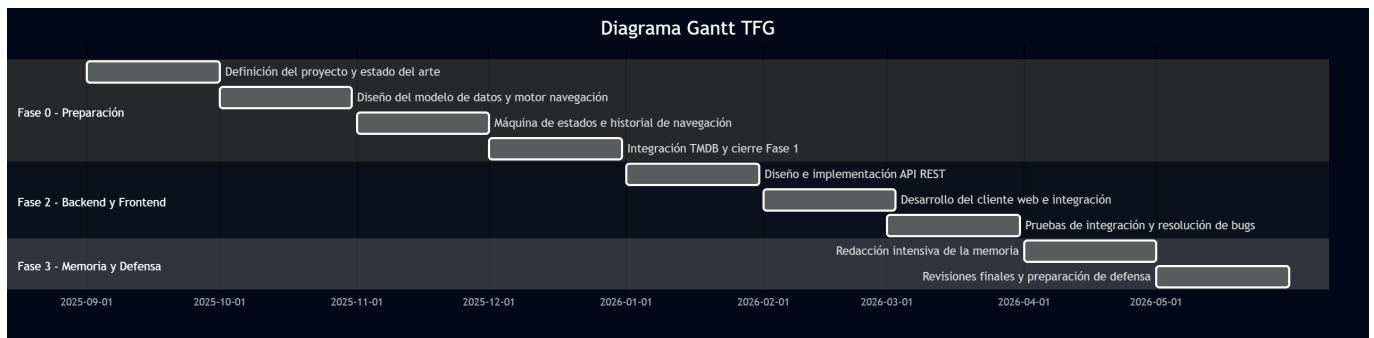


Figura 1.1: Diagrama Gantt TFG

Capítulo 2

Estado de la Cuestión

“En primer lugar, nada nace de la nada; pues, si así fuera, cualquier cosa nacería de cualquier otra, sin necesidad de semilla alguna.”

— Epicuro

El motor de exploración descrito en este trabajo se apoya en una tradición de décadas de investigación, las cuales indagan en cómo permitir que un usuario explore colecciones digitales de tamaño considerable sin conocer de antemano lo que busca, y se inscribe igualmente en la línea de los modelos de hipermedia que articularon, mucho antes de la web tal y como la conocemos, las primitivas básicas para representar información interconectada. Este capítulo recorre ese marco con dos objetivos complementarios. En primer lugar, identificar qué decisiones técnicas del proyecto son herederas directas de aportaciones previas, y cuáles otras se separan deliberadamente de ellas. En segundo lugar, delimitar el espacio dejado por los trabajos relacionados para justificar la existencia de una nueva propuesta. La sección 2.1 desarrolla el marco teórico, distinguiendo la navegación basada en etiquetas de los modelos de hipermedia más estructurados. La sección 2.2 revisa los proyectos con los que el sistema está en diálogo más estrecho, en particular la plataforma Clavy y propuestas recientes basadas en grandes modelos del lenguaje.

2.1. Marco teórico

La pregunta de fondo que recorre este capítulo es cómo organizar una colección de objetos heterogénea de manera que un usuario pueda acercarse a ella de forma exploratoria, sin formular una consulta exacta de antemano y sin depender exclusivamente de un sistema de recomendación opaco. El recorrido histórico de esa pregunta se ha articulado, a grandes rasgos, en torno a dos paradigmas. El primero, de inspiración bibliotecaria, organiza los objetos a través de descripciones controladas (etiquetas, atributos, metadatos) y permite recorrerlos refinando progresivamente esas descripciones. El segundo, nacido en el ámbito de la documentación interconectada, modela la colección como un grafo de fragmentos enlazados y cuyo mecanismo primario de exploración se centra en la transición entre nodos. Nuestro motor toma elementos de

ambos. Aplica filtrado por metadatos en el sentido del primer paradigma, y permite saltar entre colecciones siguiendo relaciones tipadas, en línea con el segundo.

2.1.1. Navegación basada en etiquetas de grandes colecciones de objetos

La navegación basada en etiquetas, o *tag-based browsing*, parte de un principio sencillo. Cada objeto de la colección se describe mediante un conjunto de pares atributo-valor, también llamados etiquetas, y la interacción del usuario consiste en seleccionar una o varias de esas etiquetas para restringir progresivamente el conjunto de objetos visibles. La selección de una etiqueta no solo filtra los resultados, sino que reduce también el espacio de etiquetas disponibles, presentando al usuario únicamente aquellas que pueden seguir refinando el conjunto activo. Este patrón tiene su origen formal en el trabajo seminal de Godin y Saunders (1986), que modela el espacio de navegación como un retículo de conceptos, donde cada estado de exploración corresponde a un par formado por un conjunto de objetos y un conjunto de etiquetas que los caracteriza, y las transiciones entre estados se obtienen añadiendo o eliminando etiquetas. Esta intuición resurge con fuerza en los *semantic file systems* descritos por Gifford et al. (1991), donde el sistema de ficheros abandona la jerarquía rígida de directorios y la sustituye por una organización plana de etiquetas que se combinan a demanda para generar vistas consultables como si fueran carpetas virtuales.

Sobre estas dos referencias se construye buena parte de la literatura posterior. La idea común es que el usuario opera sobre un *estado de navegación* caracterizado por el conjunto de etiquetas activas en ese momento, al que se asocian dos derivados: el conjunto de objetos filtrados por dichas etiquetas, y el conjunto de etiquetas seleccionables que pueden añadirse para reducir aún más esos objetos sin vaciarlos. Cada acción del usuario, ya sea añadir o eliminar una etiqueta, transforma el estado y obliga a recalcular ambos conjuntos derivados. La formalización de este modelo y su materialización algorítmica han sido el objeto de un cuerpo notable de trabajo, en el que se sitúa de forma directa la línea de investigación que dio lugar a la plataforma Clavy (Gayoso-Cabada et al., 2016a), predecesora más inmediata de nuestra propuesta. Los autores formalizan la exploración como un autómata de navegación cuyos estados son conjuntos de objetos y cuyas transiciones están etiquetadas con pares atributo-valor.

Este modelo tiene una propiedad relevante para nuestro trabajo, y que conviene subrayar antes de seguir. La caracterización del filtrado como una conjunción acumulativa de condiciones se traduce de forma natural a una consulta sobre una base de datos relacional o sobre un índice invertido, dado que cada etiqueta seleccionada corresponde a una condición **AND** adicional. Nuestra implementación hereda exactamente esa estructura. Los filtros de metadatos descritos en el capítulo de desarrollo se construyen como cláusulas **EXISTS** encadenadas, y el conjunto de etiquetas seleccionables, que en nuestro sistema denominamos *facetas*, se computa de manera dinámica agregando los valores presentes en los objetos visibles. La diferencia, sin embargo, es que la literatura clásica de navegación por etiquetas se ha centrado mayoritariamente en colecciones planas, en las que todos los objetos son del mismo

tipo y comparten un único espacio de etiquetas. En escenarios de mayor riqueza semántica, donde conviven entidades de naturaleza distinta (películas y personas, libros y autores, productos y fabricantes), el modelo plano resulta limitado. Allí donde la literatura propone una única colección descrita por etiquetas heterogéneas, nuestro motor distingue colecciones múltiples descritas cada una por sus propios atributos y conectadas entre sí por relaciones explícitas, lo que abre la puerta al segundo paradigma del que hablábamos al inicio de la sección.

2.1.2. Modelos de hipermedia y navegación estructurada

La tradición de la navegación por etiquetas describe el qué (qué condiciones se cumplen, qué objetos las satisfacen) pero deja en segundo plano el cómo (cómo el usuario llega de un objeto al siguiente, qué relaciones explícitas existen entre ellos). Este vacío lo cubre la tradición de la hipermedia, una línea de investigación que ha producido una colección de modelos de referencia cuyo objetivo común era proporcionar un vocabulario formal para describir documentos interconectados. Gruzman y Senichkin (2000) ofrecen una revisión panorámica de esta familia de modelos que conviene resumir aquí, porque varias de sus categorías son directamente trasladables a las decisiones de diseño de nuestro sistema.

El primero y más antiguo de estos modelos es el HAM (*Hypertext Abstract Machine*) (Campbell y Goodman, 1988), concebido como una máquina servidor de propósito general para almacenar sistemas de hipertexto. Su modelo de memoria distingue cinco objetos básicos: grafos, contextos, nodos, enlaces entre nodos y atributos semánticos. La correspondencia entre estas cinco abstracciones y los componentes de nuestro sistema es notablemente directa, lo que justifica con detalle la afirmación de que el HAM puede leerse como un antepasado conceptual del diseño aquí presentado. El *grafo* del HAM, entendido como la red global de información que da soporte al hiperdocumento, encuentra su equivalente en la combinación de las tablas **collections** y **reference** de nuestra base de datos, que en conjunto definen el universo de las entidades disponibles y de los vínculos tipados que las conectan. Los *nodos*, unidades indivisibles de contenido en el HAM, se corresponden con las filas de la tabla **entities**, cada una de las cuales representa una película, una serie, una persona o cualquier otra unidad atómica del dominio. Los *enlaces* entre nodos son la traducción literal de las filas de la tabla **reference**, que registran asociaciones bidireccionales entre entidades acompañadas de una *razón* (*Director*, *Actor*, etc.) que tipifica la naturaleza del vínculo. Los *atributos semánticos* del HAM, expresados en aquel modelo como pares clave-valor adheridos a nodos y enlaces, tienen un reflejo casi idéntico en nuestra tabla **metadata**, que asocia a cada entidad un conjunto de pares atributo-valor sobre los que opera el filtrado clásico. La Tabla 2.1 sintetiza esta correspondencia.

El paralelismo se completa, y resulta especialmente revelador, en el caso del *contexto*. El contexto del HAM se concibe como un fragmento de datos del grafo que recoge una vista parcial pertinente para una sesión de usuario, separada del grafo persistente y susceptible de modificarse sin alterar la información subyacente. Esa es exactamente la función que cumple la clase **State** de nuestro sistema. **State** encapsula los filtros activos, el historial de transiciones y la colección sobre la que

Objeto HAM	En nuestro sistema	Nota
Grafo	tablas <code>collections</code> y <code>reference</code>	Universo global de entidades y vínculos tipados
Nodo	filas de <code>entities</code>	Unidad atómica del dominio
Enlace	filas de <code>reference</code>	Asociaciones bidireccionales con razón
Atributo semántico	filas de <code>metadata</code>	Pares atributo-valor para filtrado
Contexto	clase <code>State</code>	Vista por sesión, separada del grafo persistente

Tabla 2.1: Correspondencia entre los cinco objetos básicos del modelo HAM (Campbell y Goodman, 1988) y los componentes equivalentes de nuestro sistema.

se está operando en un momento dado, viviendo en memoria dentro de la sesión HTTP y articulando una vista del grafo persistente que es específica de cada usuario. La distinción entre datos del dominio, persistentes y compartidos, y contexto de exploración, momentáneo y por sesión, una decisión que se detalla en el capítulo de arquitectura, encuentra aquí su raíz conceptual.

El segundo modelo de referencia, conceptualmente posterior y más elaborado, es el modelo Dexter (*Dexter Hypertext Reference Model*) (Halasz y Schwartz, 1994). Dexter organiza el sistema de hipermedia en tres capas con interfaces bien definidas. Una capa de almacenamiento, donde residen los componentes persistentes y sus enlaces. Una capa de ejecución, que coordina las sesiones de usuario y la instanciación de los componentes activos. Y una capa intracomponente, que describe la estructura interna de cada componente y se conecta con el resto a través de un mecanismo explícito de *anclajes* (*anchors*) (Figura 2.1). Esta triple separación encuentra paralelismos en la arquitectura de nuestro sistema, donde la capa de persistencia (PostgreSQL y la capa DAO), la capa de ejecución (`StateManager` y los *servlets* que orquestan la sesión) y la capa de presentación (*frontend*) cumplen funciones análogas. Más relevante aún es la noción de *enlace* en Dexter, donde un enlace no es una propiedad implícita de un nodo sino un componente de primera clase, dotado de identidad propia, posiblemente multidireccional y susceptible de ser él mismo referenciado. Nuestro objeto `Link`, que representa una transición entre colecciones e incluye la dirección, la razón de la relación y una copia del estado en el momento de realizarla, se inscribe en esta misma tradición. La transición no es un efecto secundario opaco de hacer clic en un elemento, sino un objeto explícito y persistente, cuya cadena conforma el historial de exploración y se integra literalmente en la consulta SQL que se construye en cada petición.

Una tercera línea, mucho más reciente y específicamente vinculada al desarrollo web moderno, es la recuperación de la idea de *Hypermedia As The Engine Of Application State* (HATEOAS), formulada originalmente por Fielding como uno de los principios constitutivos del estilo arquitectónico REST. Vepsäläinen (2024) señala que HATEOAS, pese a haber sido el motor conceptual detrás de REST, es probablemente la parte menos implementada del modelo, y que los sistemas web mayoritarios (basados en arquitecturas de página única que mantienen el estado en

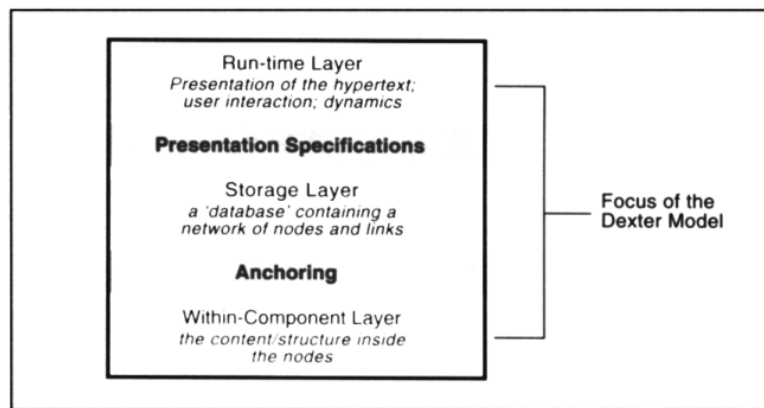


Figura 2.1: Las tres capas del modelo Dexter, con las interfaces *Presentation Specifications* y *Anchoring* entre ellas. Figura tomada de Halasz y Schwartz (1994) (Halasz y Schwartz, 1994).

el cliente) han desplazado el espíritu original. La idea fundamental de HATEOAS es que la representación que el servidor envía al cliente no contiene únicamente datos, sino también las acciones que el usuario puede ejecutar a continuación, codificadas como controles hipermedia. El cliente, por tanto, no necesita conocer de antemano la estructura completa de la API, sino que descubre el espacio de acciones disponibles a partir de la respuesta a su petición actual. Vepsäläinen (2024) destaca también la noción de *transclusión*, esto es, la sustitución parcial de fragmentos de una página por contenido nuevo recibido del servidor, como una de las primitivas características de los *frameworks* hipermedia recientes (htmx, Datastar, Hotwire), frente a la actualización completa o al renderizado en cliente.

Examinado bajo esta lente, nuestro sistema satisface el principio en sentido estricto. El *endpoint* `GET /api/facets` devuelve, junto a los objetos visibles, el conjunto completo de acciones legales en ese estado: valores de metadatos seleccionables, entidades disponibles como filtros de referencia y relaciones tipadas que habilitan saltos a otras colecciones. El cliente no codifica internamente el modelo de datos ni el espacio de transiciones, sino que presenta lo recibido y traduce las elecciones del usuario en peticiones a los *endpoints* que el servidor indica, mientras el contexto de exploración acumulado, motor real del estado de la aplicación, reside íntegramente en el `StateManager`. La transclusión, por su parte, se materializa mediante actualizaciones dinámicas que reemplazan únicamente las regiones afectadas tras cada acción.

Estos dos paradigmas, el de la navegación por etiquetas y el de los modelos hipermedia, se han mantenido tradicionalmente en compartimentos separados de la literatura. Los sistemas que filtran colecciones mediante etiquetas raramente describen sus enlaces entre objetos como elementos de primera clase, y los sistemas hipermedia tienden a primar la navegación entre nodos sobre el filtrado dentro de un nodo. La síntesis de ambos paradigmas, que es donde se sitúa nuestra contribución, no está extensamente cubierta en la literatura. Los trabajos que sí han abordado esa intersección, y de los que nuestro proyecto es heredero, en mayor o menor medida, son los que se revisan en la siguiente sección.

2.2. Trabajos relacionados

Los modelos discutidos en la sección anterior forman el sustrato conceptual sobre el que se asienta nuestro motor, pero no son trabajos directamente comparables. En esta sección se revisan dos proyectos cuya distancia con nuestra propuesta es lo suficientemente corta como para que la comparación sea inevitable. El primero, Clavy, es la línea de investigación que más profundamente ha explorado la combinación de filtrado por etiquetas y reconfiguración dinámica de esquemas en colecciones digitales, y constituye nuestro referente inmediato. El segundo, Knowledge Navigator, representa una corriente reciente que ataca el mismo problema (la exploración de colecciones extensas) desde una perspectiva radicalmente distinta, basada en grandes modelos del lenguaje. Una revisión de estos dos polos basta para situar el espacio que nuestra propuesta pretende ocupar.

2.2.1. Clavy

Clavy es una plataforma experimental para la gestión de repositorios de objetos de aprendizaje con estructuras dinámicamente reconfigurables, desarrollada a lo largo de la última década en la Universidad Complutense de Madrid (Gayoso-Cabada et al., 2016a). Su rasgo distintivo, frente a las propuestas estándar de repositorios de objetos de aprendizaje basadas en esquemas fijos como LOM (IEEE Learning Technology Standards Committee, 2002) o IMS CP (IMS Global Learning Consortium, 2004), es permitir que cada repositorio defina su propio esquema de metadatos, que ese esquema se construya de forma inductiva a medida que se cataloga la colección, y que sea reconfigurable en cualquier momento por los usuarios. Esta filosofía bottom-up acerca a Clavy a las folksonomías y, más en general, a los sistemas en los que la organización de la información emerge del uso colaborativo y no se impone de antemano (Gayoso-Cabada et al., 2016b).

El núcleo conceptual de Clavy, el que más interesa a nuestra discusión, es su modelo de navegación. Internamente, Clavy abstrae cada objeto del repositorio como un conjunto de pares elemento-valor que funcionan como etiquetas. Sobre esa abstracción, la plataforma implementa un modelo de *tag-based browsing* en el sentido descrito en la sección 2.1.1: el usuario añade y elimina etiquetas, el sistema mantiene un estado caracterizado por las etiquetas activas, y por cada acción se actualizan tanto el conjunto de objetos filtrados como el conjunto de etiquetas que siguen siendo seleccionables. La originalidad de Clavy reside en que la jerarquía de elementos del esquema, presentada al usuario como una organización facetada del espacio de navegación, puede modificarse en cualquier momento sin necesidad de reorganizar la colección (Gayoso-Cabada et al., 2017). Para hacer compatibles ambos requisitos, los autores demostraron que el comportamiento del sistema podía caracterizarse como un autómata de navegación finito independiente de la jerarquía (Figura 2.2). Dado que ese autómata puede, en el peor caso, crecer exponencialmente con respecto al tamaño de la colección, la línea posterior de la plataforma se ha dedicado a desarrollar estrategias de indexación y de *caching* que aproximan el comportamiento deseado sin materializar el autómata por completo. Este conjunto de técnicas incluye una estrategia basada en la identificación de estados equivalentes

(estados que filtran exactamente el mismo conjunto de objetos, aunque procedan de combinaciones distintas de etiquetas), que permite reducir sustancialmente el coste de las operaciones de navegación (Gayoso-Cabada et al., 2019).

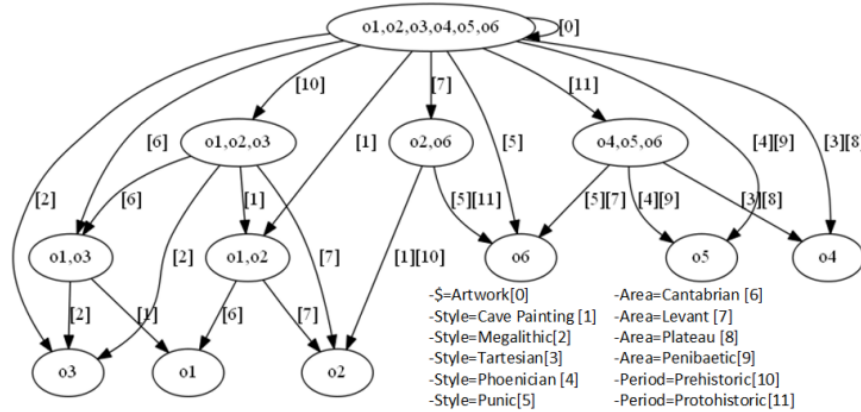


Figura 2.2: Autómata de navegación de Clavy: los estados son conjuntos de objetos y las transiciones están etiquetadas con pares atributo-valor. Figura tomada de Gayoso et al. (2016) (Gayoso-Cabada et al., 2016a).

En paralelo a la consolidación del modelo de navegación, Clavy ha evolucionado para acomodar necesidades expresivas más sofisticadas. La incorporación de elementos multivaluados, esto es, de elementos del esquema que pueden aparecer múltiples veces en la descripción del mismo objeto, fue un paso importante en escenarios reales como las bibliotecas digitales humanísticas, donde un objeto típicamente representa una obra con varios contribuidores, cada uno con sus propios atributos (Gayoso-Cabada et al., 2023). La caracterización de estos esquemas como un tipo restringido de gramática EBNF abre además la posibilidad de extender el modelo a estructuras alternativas y recursivas, acercándolo al espíritu de los lenguajes generalizados de marcado (Coombs et al., 1987). La aplicabilidad práctica de la plataforma se ha demostrado en repositorios reales de humanidades, lingüística y, recientemente, medicina, donde Clavy ha servido como soporte para generar paquetes de contenido educativo estandarizado a partir de colecciones médicas existentes (Buendía et al., 2019).

La proximidad de Clavy con nuestro proyecto es evidente, y esa proximidad nos obliga a hacer explícitas las diferencias. Compartimos la idea de un modelo interno basado en pares atributo-valor con independencia del dominio, la de una navegación guiada por la selección incremental de filtros, la del cómputo dinámico de las facetas a partir del estado actual y la de un motor que opera sobre un modelo abstracto desacoplado de la fuente de datos concreta. Las diferencias, sin embargo, marcan el espacio que nuestra propuesta ocupa. Clavy se centra en colecciones homogéneas, donde todos los objetos pertenecen al mismo dominio y comparten un único esquema, con una organización plana que permite reconfigurar la jerarquía de presentación sin afectar a la estructura subyacente. Nuestro motor, en cambio, asume desde el principio una colección estratificada en varias subcolecciones de naturaleza distinta (por ejemplo, películas, series, personas, productoras, cadenas), cada una con su propio espacio de atributos, y vinculadas entre sí por relaciones explícitamente tipadas. Esa estructura introduce dos primitivas que no tienen análogo

directo en Clavy. Por un lado, los filtros de referencia, que permiten restringir los objetos visibles de una colección por su vinculación con entidades concretas de otra. Por otro lado, la navegación transversal, que permite trasladarse de una colección a otra siguiendo una relación y arrastrando el contexto de filtrado acumulado, con un historial de transiciones que se materializa en subconsultas SQL anidadas. A estas dos primitivas se añade la operación de unión, que permite al usuario construir varios contextos de exploración de manera independiente y combinarlos a posteriori para obtener el conjunto de entidades que satisfacen al menos uno de ellos. Esta operación cubre casos de uso, como la búsqueda de personas que han participado como directoras en películas de un género o como actrices en series de otro, que no pueden expresarse con la semántica puramente conjuntiva del filtrado clásico, en la que todas las condiciones acumuladas deben cumplirse simultáneamente.

Igualmente importante es la diferencia de énfasis. Clavy ha priorizado la reconfigurabilidad dinámica del esquema y la indexación eficiente, problemas cruciales cuando la colección y su organización evolucionan en paralelo, y por ello ha invertido un esfuerzo considerable en el desarrollo de estructuras de índice (autómatas de navegación, dendrogramas, cachés selectivas) que aproximen el comportamiento ideal del autómata de navegación sin pagar su coste exponencial en el peor caso. Nuestro proyecto adopta una posición opuesta. Asume un esquema relativamente estable y delega en PostgreSQL toda la responsabilidad de la indexación, evitando así reimplementar técnicas de recuperación de información que la base de datos ya resuelve de forma madura. El espacio que se libera en el lado del motor se invierte en lo que constituye el aporte diferencial de la propuesta, esto es, la construcción incremental de consultas SQL que reflejen un contexto de navegación enriquecido con filtros relacionales, transiciones tipadas entre colecciones y combinaciones de conjuntos sobre contextos independientes. Es en ese desplazamiento del foco, desde el problema de la indexación hacia el de la traducción dinámica de un estado de exploración complejo a una consulta ejecutable, donde nuestra propuesta complementa, sin solapar, la línea de trabajo de Clavy (Tabla 2.2).

Eje	Clavy	Nuestro motor
Estructura de la colección	Homogénea, esquema único reconfigurable	Estratificada en subcolecciones tipadas con relaciones explícitas
Primitivas de navegación	Filtrado por etiquetas	Filtrado por etiquetas y referencias, navegación transversal, y operación de unión
Esquema	<i>Bottom-up</i> , reconfigurable en caliente	Estable, definido a priori
Estrategia de indexación	Autómatas, dendrogramas, cachés selectivas	Delegada en PostgreSQL; foco en traducción a SQL

Tabla 2.2: Síntesis de las diferencias entre la plataforma Clavy y nuestro motor de exploración a lo largo de cuatro ejes clave.

2.2.2. Knowledge Navigator y la exploración asistida por modelos del lenguaje

En el extremo opuesto del espectro metodológico se sitúan las propuestas recientes que abordan el mismo problema de fondo, la exploración de colecciones extensas, mediante técnicas basadas en grandes modelos del lenguaje. Knowledge Navigator, descrito por Katz et al. (2024), es representativo de esta corriente. Dada una consulta amplia sobre un dominio científico, el sistema recupera centenares de documentos relevantes, los embebe en un espacio vectorial, los agrupa mediante clustering probabilístico (modelos de mezcla gaussiana sobre embeddings reducidos con UMAP), y delega en un modelo de lenguaje las tareas de etiquetar cada *cluster* con un nombre y una descripción, filtrar aquellos que resultan poco relevantes para la consulta original y organizar los resultantes en una jerarquía temática de dos niveles que el usuario puede recorrer (Figura 2.3).

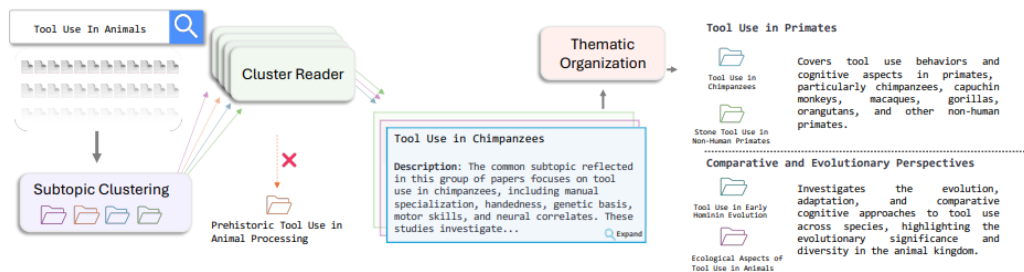


Figura 2.3: Flujo de Knowledge Navigator: recuperación, *embedding* y *clustering*, lectura por el modelo del lenguaje y organización temática final. Figura tomada de Katz et al. (2024) (Katz et al., 2024).

La filiación de Knowledge Navigator con la tradición clásica de *cluster-based browsing* es declarada por sus propios autores, que sitúan su propuesta como una recuperación moderna del paradigma Scatter/Gather de los años noventa. Lo que el modelo del lenguaje aporta a esa tradición es, fundamentalmente, una mejora en la calidad de las representaciones de los *clusters*, durante mucho tiempo el principal obstáculo para la adopción práctica del paradigma. Los nombres y descripciones generados por el modelo, frente a las listas de palabras clave extraídas automáticamente que caracterizaban a los sistemas anteriores, resultan suficientemente legibles como para servir de mapa al usuario. Las evaluaciones reportadas en el artículo, tanto automáticas como con expertos humanos, sugieren que el sistema es capaz de cubrir más del setenta por ciento de los subtemas que un revisor humano habría identificado en una revisión sistemática de literatura, y de generar además subtemas adicionales no presentes en la revisión.

La distancia con nuestra propuesta es, en este caso, mucho más amplia que en el caso de Clavy, y por razones casi opuestas. Knowledge Navigator opera sobre colecciones de texto plano (artículos científicos), sin un modelo relacional explícito, y construye la organización de la colección a partir de propiedades estadísticas del lenguaje. La organización resultante es probabilística, no necesariamente reproducible entre ejecuciones, y depende del sesgo del modelo subyacente. Nuestro motor,

en cambio, opera sobre una colección con un modelo de datos explícito (entidades, atributos, relaciones tipadas) y construye la organización de la exploración mediante operaciones deterministas y trazables sobre ese modelo. La elección entre ambos enfoques no es de mayor o menor sofisticación, sino de adecuación al problema: para una colección no estructurada, donde no existe un modelo relacional latente que un sistema determinista pueda explotar, la aproximación basada en modelos del lenguaje es probablemente la única viable. Para colecciones donde sí existe ese modelo, como las que motivan nuestro trabajo, prescindir de él para reducir el problema a uno de *clustering* sobre embeddings supondría desperdiciar información estructural valiosa y aceptar la opacidad del modelo del lenguaje a cambio de poco.

Esa distinción permite también identificar de manera explícita el hueco en el estado del arte que justifica la existencia de este proyecto. Por un lado, las herramientas clásicas de navegación basada en etiquetas, ejemplificadas por la línea Clavy, ofrecen un modelo determinista y trazable, pero se han mantenido predominantemente en el escenario de colecciones homogéneas, sin primitivas explícitas para la navegación transversal entre colecciones de distinta naturaleza ni para la combinación de contextos de exploración contruidos por separado. Por otro lado, las herramientas recientes basadas en modelos del lenguaje son flexibles ante colecciones no estructuradas, pero renuncian al modelo relacional explícito y, con él, a la posibilidad de operaciones deterministas sobre el espacio de navegación. Entre ambos extremos queda un espacio escasamente cubierto de forma satisfactoria, el de un motor de exploración que mantenga el rigor determinista de la primera tradición, que extienda el modelo plano de etiquetas con relaciones tipadas entre colecciones y con operaciones de conjuntos sobre contextos independientes, y que exponga estas capacidades a través de una API web de manera que sean efectivamente utilizables desde una interfaz interactiva. La propuesta cuyo desarrollo se detalla en los capítulos siguientes pretende, precisamente, ocupar ese espacio.

Arquitectura y Metodología

Este capítulo centraliza el conjunto de decisiones de diseño técnico y estructural que sirven como fundamentos del proyecto. Comenzamos formalizando el **motor de navegación** que articula todo el sistema: la metodología, la arquitectura por capas y la persistencia que se describen después son, en el fondo, decisiones de soporte para esa pieza conceptual. Esta primera sección recoge el camino que llevó del prototipo previo al modelo definitivo y presenta dicho modelo con una notación formal, para que los capítulos siguientes puedan referirse a él sin ambigüedad. A continuación, exponemos los marcos metodológicos utilizados para el manejo del propio ciclo de vida del software, definiendo las herramientas, prácticas y técnicas de ingeniería aplicadas. Luego, seguimos con el desglose de la arquitectura del sistema, delimitando expresamente la separación de responsabilidades a través de las distintas capas de la aplicación: la capa de control y exposición de la API, la capa de persistencia y acceso a datos, y finalmente, el mecanismo de integración con la interfaz de usuario. El principal objetivo de esta arquitectura fue garantizar un sistema modular, escalable, mantenible y flexible en la integración de base de datos.

3.1. Motor de navegación

El motor de navegación es la pieza central sobre la que descansa el resto del sistema: la metodología elegida para construirlo, la arquitectura en capas que lo aloja y la capa de persistencia que lo materializa se entienden mejor cuando se conoce primero qué pretende modelar. Esta sección reconstruye, en este orden, la historia y la teoría del motor: un primer apartado describe el prototipo previo que sirvió de banco de pruebas y las lecciones que se extrajeron de él; un segundo apartado decanta esas lecciones en un modelo formal cuyos conjuntos, estado, operaciones y semántica fijan el vocabulario que utiliza el resto de la memoria.

3.1.1. Del prototipo al modelo formal

Antes de plantear la arquitectura definitiva del sistema, se desarrolló un prototipo previo¹ cuyo objetivo era validar las intuiciones iniciales sobre el modelo de navegación. La meta no era construir un producto, sino explorar empíricamente qué primitivas necesitaba el motor y qué decisiones de diseño se sostenían cuando se las llevaba al código.

El prototipo se implementó en **Java 17** sobre **Maven**, sin servidor de aplicaciones ni base de datos: era una aplicación de línea de comandos que cargaba un dataset reducido (`test_pms.json`) en memoria y permitía recorrerlo de forma interactiva. Sobre ese dataset — compuesto por colecciones de tipo **People**, **Movies** y **Studios** relacionadas entre sí — el prototipo ofrecía un subconjunto del catálogo de operaciones que terminaría incorporando el sistema final: seleccionar la colección activa, listar las entidades visibles, consultar el detalle de una entidad, aplicar filtros sobre metadatos (con la variante negativa), aplicar filtros de referencia entre colecciones, navegar por enlaces relacionales y guardar o restaurar estados ya visitados.

Aunque el prototipo logró validar la idea, también expuso limitaciones que motivaron rediseñarlo desde cero. La más importante fue de naturaleza *conceptual*: el prototipo no distinguía con precisión entre el estado de navegación del usuario y los datos del dominio, mezclando responsabilidades que en el diseño final quedaron separadas en capas distintas (modelo, acceso a datos y controladores). Esta confusión generó deuda técnica temprana que habría sido necesario pagar antes de poder implementar mecanismos más complejos — como la navegación transversal o las operaciones de unión — y desaconsejaba promocionar el prototipo a sistema final.

A esta limitación de fondo se sumaron carencias prácticas que también empujaron hacia un rediseño: la carga en memoria del dataset hacía inviable escalar a catálogos del tamaño de los reales y obligaba a recorrer estructuras **HashMap** anidadas para resolver cualquier consulta, sin un lenguaje declarativo (como SQL) en el que expresar los filtros acumulados; faltaban además primitivas que después se revelaron centrales, como los filtros *negativos* sobre referencias y la operación de *unión* entre exploraciones independientes; y la interfaz por línea de comandos resultaba opaca para entender las trayectorias de navegación.

La consecuencia metodológica fue clara: el siguiente paso no podía ser “arreglar el prototipo”, sino *formalizar* qué era exactamente lo que estaba modelando, con independencia de la implementación. La subsección siguiente recoge esa formalización: define los conjuntos sobre los que opera el motor, el estado, la estructura de los enlaces, el contexto de exploración, las operaciones primitivas y la semántica del estado. El sistema final — descrito en el resto del capítulo y en el capítulo de desarrollo — es una realización fiel de ese modelo: el código del backend (**State**, **StateManager**, **Link** y **NavigationDAO**) reproduce uno a uno los elementos formales que se introducen a continuación.

¹<https://github.com/hibjan/TFG-prototype>

3.1.2. Modelo formal de navegación

Conjuntos base

El motor opera sobre un universo de datos estructurado en datasets, colecciones y entidades. Fijamos los siguientes conjuntos:

- $\mathcal{D}$: conjunto de *datasets*. Cada dataset agrupa colecciones cargadas desde una misma fuente.
- $\mathcal{C}$: conjunto de *colecciones*.
- $\mathcal{E}$: conjunto de *entidades*.
- $\mathcal{K}$: conjunto de *claves de metadato*.
- $\mathcal{V}$: conjunto de *valores de metadato*.
- $\mathcal{R}$: conjunto de *razones*; cada razón etiqueta un tipo de relación dirigida entre entidades.

Las funciones que estructuran la pertenencia entre conjuntos son:

$ds : \mathcal{C} \rightarrow \mathcal{D}$	(dataset al que pertenece una colección)
$coll : \mathcal{E} \rightarrow \mathcal{C}$	(colección a la que pertenece una entidad)
$meta : \mathcal{E} \rightarrow \mathcal{P}(\mathcal{K} \times \mathcal{V})$	(pares atributo-valor de una entidad)
$Ref \subseteq \mathcal{E} \times \mathcal{E} \times \mathcal{R}$	(tripletas de referencias tipadas activas)

La sesión de un usuario explora un único dataset; fijamos por tanto un dataset $d_0 \in \mathcal{D}$ y denotamos $\mathcal{C}_{d_0} = \{c \in \mathcal{C} \mid ds(c) = d_0\}$.

Estado de exploración

Un *estado* es una tupla

$$s = \langle c, F^+, F^-, R^+, R^-, L \rangle$$

donde:

- $c \in \mathcal{C}_{d_0}$ es la *colección activa*.
- $F^+, F^- : \mathcal{C}_{d_0} \times \mathcal{K} \rightarrow \mathcal{P}(\mathcal{V})$ son los *filtros de metadatos*, positivos y negativos, indexados por colección y clave.
- $R^+, R^- : \mathcal{C}_{d_0} \times \mathcal{C}_{d_0} \times \mathcal{R} \rightarrow \mathcal{P}(\mathcal{E})$ son los *filtros de referencia*, positivos y negativos, indexados por colección activa, colección referenciada y razón.
- $L = [\ell_1, \dots, \ell_n]$ es una secuencia ordenada de *links* (definidos a continuación).

Denotamos por $\mathcal{S}$ el conjunto de estados posibles. Esta tupla se corresponde literalmente con la clase **State** del backend: el campo c es `currentCollectionId`; los mapas F^+, F^-, R^+, R^- son `mfilters`, `notMfilters`, `rfilters` y `notRfilters`; y L es la lista `links`. El dataset d_0 se almacena además en el campo `datasetId`.

Link

Un *link* representa una transición entre colecciones registrada como parte del historial de navegación del estado. Es una tupla

$$\ell = \langle d, c_{\text{from}}, r, s' \rangle$$

donde:

- $d \in \{\text{fwd}, \text{bwd}\}$ es la dirección del enlace.
- $c_{\text{from}} \in \mathcal{C}_{d_0}$ es la *colección origen* de la transición, es decir, la colección activa antes de aplicar el cambio.
- $r \in \mathcal{R}$ es la razón seguida.
- $s' \in \mathcal{S}$ es un *snapshot* (copia inmutable) del estado anterior a la transición.

Esta tupla corresponde con la clase **Link**: el campo d se materializa en el booleano **forward**, y s' es la copia profunda del estado capturada en el momento de crear el enlace.

Contexto de exploración

El contexto de una sesión es una tupla

$$\Sigma = \langle s_{\text{cur}}, H, U, T \rangle$$

donde:

- $s_{\text{cur}} \in \mathcal{S}$ es el *estado activo*.
- H es una pila LIFO de estados (**historyStack**), utilizada por **save** y **restore**.
- $U = [s_1, \dots, s_k]$ es la lista de estados marcados para unión (**unionSet**).
- T es una pila auxiliar de links *forward* ya aplicados (**linkStack**), que permite a **goback** recuperar la colección origen del último enlace.

Operaciones primitivas

Para cada operación se indica su signatura, su precondition y su postcondition. Las operaciones marcadas con $\rightsquigarrow$ actúan sobre el estado activo s_{cur} del contexto; las marcadas con $\Rightarrow$ modifican el contexto completo. Por brevedad, cuando se modifica una función como F^+ , escribimos $F^+[(c, k) \mapsto V]$ para denotar la función igual a F^+ salvo en (c, k) , donde toma el valor V .

1. **ADDMFILTER**(s, k, v) $\rightsquigarrow s'$
 - Signatura: $\mathcal{S} \times \mathcal{K} \times \mathcal{V} \rightarrow \mathcal{S}$.
 - Precondition: $k \in \mathcal{K}, v \in \mathcal{V}$.

- Postcondición: s' es idéntico a s salvo que $F^{+'} = F^+[(c, k) \mapsto F^+(c, k) \cup \{v\}]$, donde c es la colección activa de s .
2. ADDNOTMFILTER(s, k, v) $\rightsquigarrow s'$
- Signatura: $\mathcal{S} \times \mathcal{K} \times \mathcal{V} \rightarrow \mathcal{S}$.
 - Postcondición: análoga a la anterior actualizando F^- en lugar de F^+ .
3. ADDRFILTER(s, c', r, e) $\rightsquigarrow s'$
- Signatura: $\mathcal{S} \times \mathcal{C}_{d_0} \times \mathcal{R} \times \mathcal{E} \rightarrow \mathcal{S}$.
 - Precondición: $\text{coll}(e) = c'$.
 - Postcondición: $R^{+'}(c, c', r) = R^+(c, c', r) \cup \{e\}$; el resto de R^+ no cambia. La operación ADDNOTRFILTER es análoga actualizando R^- .
4. LINK(s, c', r) $\rightsquigarrow s'$
- Signatura: $\mathcal{S} \times \mathcal{C}_{d_0} \times \mathcal{R} \rightarrow \mathcal{S}$.
 - Precondición: existe al menos una entidad $e \in \llbracket s \rrbracket$ y una entidad $e' \in \mathcal{E}$ con $\text{coll}(e') = c'$ tales que $(e, e', r) \in \text{Ref}$.
 - Postcondición: s' tiene la misma información de filtros que s y además c' pasa a ser la colección activa, $L' = L :: \langle \text{fwd}, c, r, s \rangle$.
- El contexto registra adicionalmente el nuevo link en la pila T .
5. GOBACK(Σ) $\Rightarrow \Sigma'$
- Signatura: contexto $\rightarrow$ contexto.
 - Precondición: T no está vacía; sea $\ell = \langle \text{fwd}, c_{\text{prev}}, r_{\text{prev}}, s_{\text{snap}} \rangle$ el elemento en la cima de T .
 - Postcondición: $T' = \text{pop}(T)$; el estado activo s'_{cur} se obtiene de s_{cur} cambiando la colección activa a c_{prev} y añadiendo a L un link $\langle \text{bwd}, c, r_{\text{prev}}, s_{\text{cur}} \rangle$, donde c es la colección activa de s_{cur} .
6. SAVE(Σ) $\Rightarrow \Sigma'$ y RESTORE(Σ) $\Rightarrow \Sigma'$
- SAVE apila una copia inmutable de s_{cur} en H .
 - RESTORE desapila el último estado guardado y lo restaura como s_{cur} , siempre que $|H| > 1$.
7. UNION(Σ, c') $\Rightarrow \Sigma'$
- Signatura: contexto $\times \mathcal{C}_{d_0} \rightarrow$ contexto.
 - Postcondición: $U' = U :: s_{\text{cur}}$; $H' = \emptyset$; s'_{cur} se reinicializa como estado limpio con colección activa c' (es decir, sin filtros y con $L = []$).

Semántica: resultados de un estado

Lo más importante del modelo es decidir *qué conjunto de entidades representa un estado*. Definimos una función semántica

$$\llbracket \cdot \rrbracket : \mathcal{S} \longrightarrow \mathcal{P}(\mathcal{E})$$

de manera recursiva sobre la lista de links. Empezamos por los predicados de filtrado, que actúan sobre una entidad e con colección $c = \text{coll}(e)$:

$$\begin{aligned} \Phi^+(s, e) &\equiv \forall (k, V) \in F^+(c, \cdot) \ \exists v \in V : (k, v) \in \text{meta}(e), \\ \Phi^-(s, e) &\equiv \forall (k, V) \in F^-(c, \cdot) \ \forall v \in V : (k, v) \notin \text{meta}(e), \\ \Psi^+(s, e) &\equiv \forall (c', r, E) \in R^+(c, \cdot, \cdot) \ \forall e' \in E : (e, e', r) \in \text{Ref}, \\ \Psi^-(s, e) &\equiv \forall (c', r, E) \in R^-(c, \cdot, \cdot) \ \forall e' \in E : (e, e', r) \notin \text{Ref}. \end{aligned}$$

(La notación $F^+(c, \cdot)$ designa el conjunto de pares (k, V) con $V = F^+(c, k) \neq \emptyset$; análogamente $R^+(c, \cdot, \cdot)$ designa las tripletas (c', r, E) con $E = R^+(c, c', r) \neq \emptyset$.)

Caso base. Si $L = []$,

$$\llbracket s \rrbracket = \{ e \in \mathcal{E} \mid \text{coll}(e) = c \wedge \Phi^+(s, e) \wedge \Phi^-(s, e) \wedge \Psi^+(s, e) \wedge \Psi^-(s, e) \}.$$

Caso recursivo. Si $L = L' :: \ell$ con $\ell = \langle d, c_{\text{from}}, r, s' \rangle$, sea $s_{L'}$ el estado obtenido de s reemplazando L por L' . Entonces

$$\llbracket s \rrbracket = \{ e \in \llbracket s_{L'} \rrbracket \mid \exists e'' \in \llbracket s' \rrbracket : \Theta_d(e, e'', r) \},$$

donde

$$\Theta_{\text{fwd}}(e, e'', r) \equiv (e'', e, r) \in \text{Ref}, \quad \Theta_{\text{bwd}}(e, e'', r) \equiv (e, e'', r) \in \text{Ref}.$$

Esta definición recursiva es exactamente la que se materializa en la sobrecarga recursiva `appendFilterClauses` de `NavigationDAO`: la rama no recursiva delega en `appendStateFilters` ($\Phi^\pm$ y $\Psi^\pm$ se traducen en cláusulas `EXISTS` y `NOT EXISTS` sobre las tablas `metadata` y `reference`); la rama recursiva añade, por cada link ℓ , una subconsulta sobre `reference` que liga las entidades del nivel actual con las del estado anterior, eligiendo `target_id` o `source_id` según ℓ sea *forward* o *backward*. Las secciones 4.3 y 4.4 del capítulo de desarrollo describen esa traducción a SQL con el código a la vista, y la sección 4.5 hace lo propio para la unión.

Semántica de la unión

El operador de unión combina el estado activo con todos los estados guardados en U . Su semántica es la unión conjuntista de los resultados individuales:

$$\llbracket \Sigma \rrbracket_U = \llbracket s_{\text{cur}} \rrbracket \cup \bigcup_{s \in U} \llbracket s \rrbracket.$$

En el código, esta semántica se realiza en `NavigationDAO.getUnionEntities`, que emite una única consulta SQL en la que cada estado de $U \cup \{s_{\text{cur}}\}$ contribuye con un bloque de condiciones unidas mediante `OR`, exactamente la traducción de la unión definida arriba (sección 4.5).

3.2. Metodología de Ingeniería del Software

Este Trabajo de Fin de Grado se ha organizado por principios de Ingeniería de Software fundamentados en el paradigma ágil. Como se estableció en el plan de trabajo, hemos optado por **Scrum** (Hanser, 2026) como referencia, adaptando sus características a las particularidades de un equipo reducido de dos personas. La adaptación resultó esencial para mantener la disciplina y estructura ofrecida por el modelo ágil sin introducir una sobrecarga innecesaria.

Por eso la aplicación de **Scrum** se centró principalmente en ciclos de desarrollo iterativos (*sprints*) de corta duración, donde se planteaban objetivos concretos y alcanzables. En general, toda iteración coincidía con un incremento funcional del sistema, con excepción de las fases iniciales, en lo cual la mayor parte del esfuerzo se mantuvo en una especificación inicial del motor de navegación. Este abordaje permitió ejecutar una validación progresiva de los componentes críticos del proyecto. Específicamente, se ha notado un gran impacto en los *milestones*: motor de navegación en Java, la integración con la base de datos TMDB y la posterior exposición de funcionalidades mediante una API REST.

Desarrollo iterativo e incremental

A cambio de definir completa y decididamente la arquitectura desde el principio con un enfoque lineal y secuencial, como la metodología cascada, el sistema se ha desarrollado de forma incremental. En sus primeras iteraciones, hemos priorizado la validación del propio modelo de navegación y la estructura de datos adyacente, al mismo tiempo que en fases posteriores hemos incorporado progresivamente la capa de acceso a datos, la API y el *front-end web*. Dicho tratamiento resultó significativamente relevante debido a la naturaleza exploratoria del proyecto, ya que permitió adaptar el diseño del motor de navegación al mismo tiempo que se probaba con datos reales.

Gestión de requisitos y backlog

Las funcionalidades más complejas iniciales, como el sistema de filtrado relacional, la gestión de estados de navegación y la combinación de resultados, se desarrollaron mediante periodos de trabajo intensivo centrados en la implementación y validación de cada componente. En cambio, el resto de tareas de menor complejidad o con menor dependencia técnica se fueron abordando progresivamente siguiendo la prioridad establecida en el *backlog*.

Los criterios de priorización y la planificación de las tareas se realizaron en coordinación con la tutoría del proyecto, llevando a cabo revisiones periódicas y ajustes continuos. De esta forma, los objetivos de cada iteración se adaptaban en función de los resultados obtenidos y de los obstáculos técnicos que iban apareciendo durante el desarrollo.

El *backlog* del proyecto se gestionó en GitHub Projects² y recoge la priorización

²<https://github.com/users/hibjan/projects/5/views/1>

descrita, como se observa en la Figura 3.1.

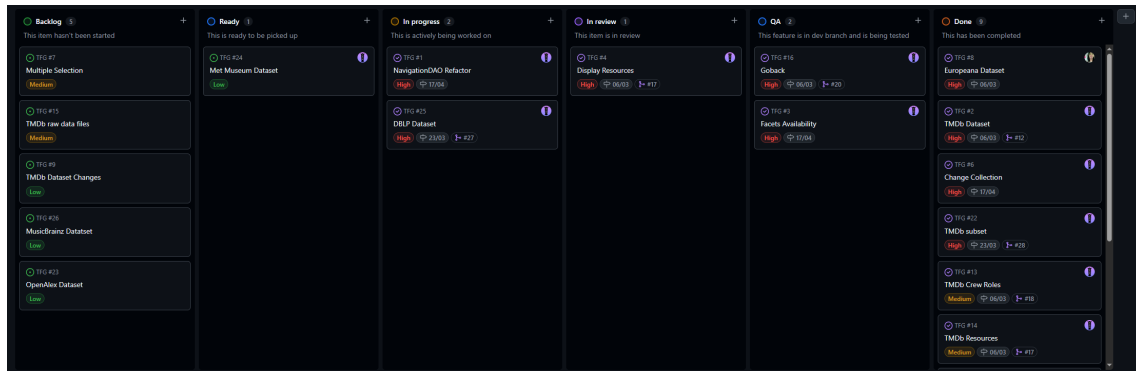


Figura 3.1: Captura del backlog del proyecto en GitHub Projects.

Trabajo colaborativo en un equipo reducido

Hemos adaptado las prácticas de **Scrum** a un entorno de colaboración directa y continua, ya que el proyecto sería desarrollado por un equipo de dos personas. En lugar de llevar a cabo reuniones formales, se optó por mantener una comunicación frecuente entre los integrantes, manteniendo siempre una división clara y acordada de responsabilidades, fundamental para la paralelización del desarrollo sin perder coherencia en el diseño global. Este método de comunicación y organización facilitó una evolución más eficiente y dinámica del software, con una alta capacidad de reacción frente los problemas técnicos.

Control de versiones y gestión del código fuente

En virtud de la necesidad de mantener la integridad del código y facilitar el trabajo colaborativo, hemos utilizado Git como sistema de control de versiones distribuido. El repositorio principal se alojó en GitHub³ y se empleó una estrategia de ramificación sencilla : una rama principal (*main*) para el código estable y funcional, y ramas de características (*feature branches*) para el desarrollo de nuevos componentes antes de su integración final. Con esa metodología, hemos podido trabajar en paralelo sin conflictos significativos manteniendo siempre una versión funcional del sistema.

Buenas prácticas de programación y diseño

Durante el desarrollo hemos dado especial atención a la aplicación de buenas prácticas de programación y diseño software. Se siguieron principios de separación de responsabilidades (*Separation of Concerns*) y modularidad, lo cual está reflejado en la organización del código en distintos paquetes (por ejemplo, **controller**, **model**, **dao**, **filter**, etc.). Dicha estructura tiende a facilitar la legibilidad del código, su mantenimiento y la posibilidad de ampliación futura sin necesidad de realizar modificaciones significativas o profundas en su arquitectura.

³<https://github.com/hibjan/TFG>

3.3. Arquitectura del Sistema

Para permitir la exposición de la lógica del motor de exploración y su respectivo consumo por parte de aplicaciones, hemos diseñado una arquitectura fundamentada en el modelo Cliente-Servidor. Desde el lado del servidor (*backend*), la aplicación favorece el desacoplamiento al basarse en capas bien definidas. Esta sección se dedica a detallar la capa de control, responsable de enrutar las distintas peticiones HTTP, y la capa de filtros, encargada de controlar las políticas de acceso y seguridad.

La Figura 3.2 resume las capas que constituyen el sistema y su flujo principal de comunicación, desde el cliente web hasta la base de datos relacional.

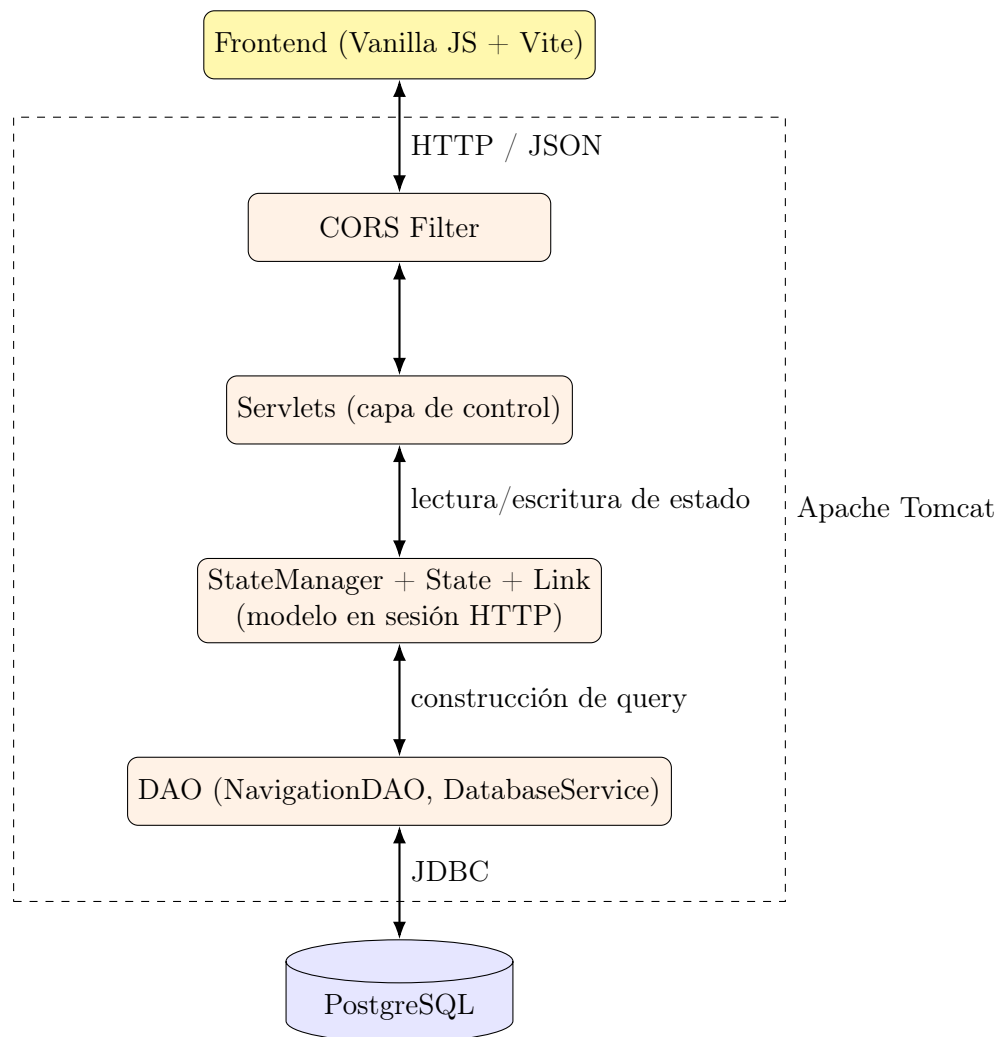


Figura 3.2: Arquitectura general del sistema en capas.

Diseño de la API y Controladores

Hemos optado por implementar la interfaz de comunicación entre el cliente web y el motor de navegación usando una API de estilo REST, siguiendo la especificación de *Servlets* de Java (**HttpServlet**) (Hunter y Crawford, 2001). Esta elección viene de priorizar un control directo sobre las peticiones y respuestas HTTP frente al

uso de frameworks web de más alto nivel (como Spring Boot), que aunque facilitan el desarrollo de aplicaciones web tradicionales, introducen una mayor complejidad estructural y una capa adicional de abstracción que no resulta necesaria para los objetivos de este proyecto.

El uso directo de `HttpServlet` ha sido necesaria para implementar una capa de comunicación más compacta, donde cada controlador se encarga exclusivamente de recibir los parámetros de la petición, delegar la lógica en el motor de navegación y generar la respuesta correspondiente en formato JSON. Emplear esa estrategia fue importante para mantener una separación explícita entre la lógica de navegación y la capa de acceso web. Además permitió un control más preciso de la gestión de sesiones, el formato de las respuestas y el flujo de las peticiones HTTP.

Para ejecutar la aplicación se ha utilizado Apache Tomcat (Brittain y Darwin, 2007) como contenedor de *Servlets*. Nos hemos inclinado por Tomcat frente a otras opciones principalmente por la naturaleza del proyecto. Tomcat es un contenedor considerablemente ligero, enfocado en la ejecución de aplicaciones basadas en *Servlets* y JSP, lo que lo convierte en una solución adecuada para trabajar directamente con la API de Java sin generar dependencias adicionales.

Por motivos semejantes, hemos descartado servidores de aplicaciones completos como GlassFish, ya que requieren tecnologías adicionales como EJB y gestión avanzada de recursos. Alternativas como Jetty también permiten ejecutar *Servlets*, pero éste está más orientado a una integración embebida dentro de otras aplicaciones/frameworks. Un contenedor independiente, sencillo de configurar y con respaldo de experiencia en entornos académicos y de desarrollo hace que Tomcat resulte la opción más estable encontrada.

Como contrapartida, esta decisión implica un mayor esfuerzo de implementación manual, especialmente en tareas como el enrutamiento de peticiones, la serialización de datos o la gestión de errores. Sin embargo, este *trade-off* se ha asumido de forma consciente, ya que el objetivo principal del proyecto no es el desarrollo de una aplicación web en sí, sino el diseño e implementación de un motor de navegación sobre colecciones de datos, siendo la API web únicamente un mecanismo para exponer dicha funcionalidad.

Gestión del Estado de Navegación (Enfoque *Stateful*)

El manejo del contexto del usuario no es trivial, ya que, a diferencia de aplicaciones web tradicionales basadas en peticiones independientes, la naturaleza de nuestro proyecto implica una navegación que consiste en una exploración progresiva, donde cada acción del usuario modifica un estado acumulativo que condiciona las operaciones posteriores.

Las arquitecturas REST normalmente trabajan con sistemas *stateless* (sin estado), donde cada petición del cliente debe contener toda la información necesaria para ser procesada. En nuestro caso, dicha naturaleza es altamente iterativa y acumulativa, de manera que hacer que cada petición esté completamente encapsulada y atomizada puede suponer un problema.

Requerir que el cliente envíe en cada petición el historial completo de navegación, las listas de filtros aplicados, las operaciones realizadas y los saltos relacionales

supondría una sobrecarga de red inasumible. Este estado no puede considerarse independiente entre peticiones, sino que constituye una estructura dinámica que evoluciona a lo largo de la sesión de usuario. Además, requerir la petición completa añadiría una complejidad excesiva en el *frontend*, aumentando el tamaño de cada *request*, empeorando el rendimiento y dificultando considerablemente la escalabilidad.

Por este motivo, se ha optado por un diseño *stateful*, en el que el contexto de navegación se mantiene en el servidor. Al inicio de la interacción, se crea una sesión de usuario (`HttpSession`) en la que se almacena una instancia de `StateManager`, encargada de gestionar el estado actual y las transiciones entre estados del usuario.

De este modo, los distintos controladores pueden operar sobre un contexto compartido y persistente, aplicando las acciones del usuario (como añadir filtros, explorar relaciones o combinar resultados) de forma centralizada en el *backend*. Este enfoque permite simplificar la lógica del cliente, mejorar la eficiencia de las comunicaciones y mantener una coherencia estricta con el modelo de navegación propuesto.

Componentes transversales de la arquitectura

El sistema contiene una serie de componentes transversales, más allá de la capa de controladores, que proporcionan funcionalidades comunes que permiten separar claramente la lógica de dominio y los aspectos técnicos propios de la infraestructura web.

Como se ha mencionado en la sección anterior, la gestión de sesiones es un elemento fundamental dentro de la arquitectura, pues es el responsable por mantener el contexto de navegación del usuario entre peticiones distintas. A través del mecanismo de `HttpSession`, el sistema asocia cada usuario con una instancia de `StateManager`, lo que garantiza la persistencia del estado de navegación y posibilita un modelo de interacción basado en estados.

Por otro lado, la serialización de datos en formato JSON empleada de manera sistemática funciona como el mecanismo clave para el intercambio de información entre el *backend* y el *frontend*. Este abordaje se ha hecho necesario principalmente para desacoplar ambas capas, lo que facilita la evolución independiente de la interfaz de usuario y del motor de navegación como tal.

Adicionalmente, hemos implementado un filtro encargado de gestionar aspectos técnicos de la comunicación HTTP, como el intercambio de recursos entre distintos orígenes - política CORS (*Cross-Origin Resource Sharing*). Este componente intercepta las peticiones entrantes antes de que alcancen los controladores y añade las cabeceras necesarias para permitir la comunicación entre cliente y servidor cuando se ejecutan en dominios o puertos distintos, necesario para evitar las restricciones por defecto de seguridad del navegador. De este modo, se evita introducir lógica técnica adicional en los controladores y se mantiene su enfoque en la lógica de dominio.

3.4. Persistencia y Acceso a Datos

La capa de persistencia y su respectivo diseño constituyen la base fundamental de la arquitectura del sistema, puesto que el motor de navegación tiene una relación de dependencia directa de la estructura y disponibilidad de los datos sobre los que opera. Como el sistema no se limita a recuperación de la información, en este proyecto los datos no se consumen directamente, sino que pasan por una transformación y reorganización que permiten la exploración mediante un modelo de navegación relacional.

Para posibilitar dicha exploración y mantener un nivel razonable de independencia del dominio, hemos decidido desacoplar completamente la lógica del motor de las fuentes de datos concretas. Para ello, hemos definido un modelo interno de representación que abstrae las entidades y relaciones relevantes del dominio, permitiendo trabajar de forma uniforme independientemente del origen de los datos. Este enfoque es particularmente relevante dado que el sistema integra conjuntos de datos heterogéneos, los cuales deben ser "traducidos" bajo una misma estructura conceptual.

Por esa misma razón, hemos optado por organizar el acceso a datos a través de una capa dedicada de abstracción (*DAO*), que se encarga de aislar la lógica de acceso, traducción y recuperación de la información. Esta separación es lo que garantiza mantener la independencia entre la lógica de navegación y los mecanismos de persistencia, permitiendo al mismo tiempo una evolución del propio sistema y posible cambios en las fuentes de datos o incluso en su forma de almacenamiento.

Modelo de datos y representación interna

El sistema se base en un modelo de datos interno diseñado específicamente para soportar la navegación sobre colecciones complejas, permitiendo representar relaciones cualitativamente distintas independiente del dominio asociado. Decidimos no operar directamente sobre las estructuras proporcionadas por las fuentes de datos externas, y sí definir una representación propia basada en entidades y relaciones, adaptada a las necesidades del motor de navegación.

En dicho modelo, los elementos del dominio se representan como entidades independientes, caracterizadas por un conjunto de atributos (**metadatos**), mientras que las interacciones entre dichas entidades se expresan mediante relaciones explícitamente manifestadas. Esta separación permite tratar de forma homogénea tanto el filtrado por atributos (por ejemplo, características propias de una entidad) como la exploración relacional (navegación entre entidades conectadas).

Esta representación se materializa en un esquema relacional almacenado en PostgreSQL y compuesto por cinco tablas: **datasets**, **collections**, **entities**, **metadata** y **reference**. Los *datasets* agrupan colecciones, cada colección contiene entidades y cada entidad lleva asociado un número arbitrario de filas de metadatos. Para evitar fijar el conjunto de atributos por anticipado (requisito incompatible con la naturaleza heterogénea de las fuentes integradas), la tabla **metadata** sigue un patrón clave/valor que asocia cada entidad con tantos pares (**key**, **value**) como atributos posea. De manera análoga, la tabla **reference** materializa las relaciones entre

entidades como filas de primera clase, con un identificador de origen, uno de destino y una *razón* que cualifica el tipo de vínculo. Esta uniformidad permite que la incorporación de nuevos tipos de atributos o nuevas clases de relaciones no requiera modificar el esquema. El detalle del modelo relacional y de las restricciones de integridad asociadas se presenta en el Capítulo 4.

Al mismo tiempo, esta representación actúa como una capa de abstracción sobre las fuentes de datos, unificando conjuntos heterogéneos bajo una estructura común e independiente de su formato u origen. De este modo, el motor de navegación puede operar de forma desacoplada, garantizando tanto la extensibilidad del sistema como la coherencia en las operaciones realizadas sobre los datos.

Integración y adquisición de datos

Dado que el sistema utiliza y transforma fuentes de datos heterogéneas, una de las decisiones arquitectónicas principales ha sido definir un proceso de integración que permita desacoplar la adquisición de datos de su representación interna. En lugar de depender directamente de la estructura y organización de cada fuente, se ha establecido un mecanismo de transformación que adapta los datos externos al modelo común basado en entidades y relaciones descrito anteriormente.

Este proceso de integración implica la extracción de la información relevante de cada fuente de datos, su normalización y su posterior adaptación a las estructuras internas del sistema. De este modo, se garantiza que, independientemente del formato, granularidad o semántica original de los datos, todos ellos puedan ser tratados de forma uniforme por el motor de navegación.

Una de las consideraciones más importantes en este diseño ha sido la coexistencia de distintos conjuntos de datos con características complementarias. Antes de privilegiar una única fuente como referencia principal, hemos diseñado el sistema para permitir la incorporación de nuevas fuentes, siempre que estas puedan ser mapeadas al modelo interno. Este enfoque posibilita la extensibilidad del sistema y evita dependencias estrictas con tecnologías o fuentes concretas.

Asimismo, se ha optado por un modelo de integración en el que la responsabilidad de adaptación recae en la capa de acceso a datos, evitando trasladar esta complejidad al motor de navegación. De este modo, el núcleo del sistema opera exclusivamente sobre estructuras homogéneas, manteniendo una separación clara entre la lógica de dominio y los detalles específicos de cada fuente de datos.

Capa de acceso a datos (DAO)

Para mantener una separación eficiente entre la lógica de navegación y los mecanismos de acceso a datos, hemos definido una capa específica de abstracción basada en el patrón *Data Access Object* (DAO). La capa actúa como intermediaria entre el modelo interno del sistema y las distintas fuentes de datos, encapsulando toda la lógica necesaria para la recuperación y adaptación de la información.

La principal responsabilidad de los componentes DAO es proporcionar una interfaz uniforme de acceso a los datos, independiente de su origen. De este modo, el motor de navegación no interactúa de manera directa con las fuentes externas,

sino que opera sobre estructuras ya adaptadas al modelo interno. Hemos tomado esta decisión para aislar completamente el núcleo del sistema de detalles específicos (como el formato de los datos, los mecanismos de acceso o, mismamente, las particularidades de cada fuente).

Además, esta capa facilita la integración de múltiples conjuntos de datos, ya que cada fuente puede ser gestionada mediante su propia implementación de acceso, de manera compleamente independiente, sin afectar al resto del sistema. Esto resulta especialmente relevante en un contexto donde hay una coexistencia de datos heterogéneos, permitiendo su incorporación de manera progresiva, controlada y sin grandes dependencias.

Esta decisión también es relevante para hacer la mantenibilidad y evolución del sistema de manera flexible. Al centralizar la lógica de acceso a datos en una capa específica, es posible realizar cambios en las fuentes de información, en sus formatos o en las estrategias de recuperación sin necesidad de modificar el motor de navegación o la capa de control. Por lo tanto, se garantiza una arquitectura modular, robusta, preparada para futuras extensiones y la incorporación de otras colecciones de datos.

Estrategias de gestión de datos

Como mencionado anteriormente, hemos priorizado el diseño del sistema a partir de una separación clara entre dos niveles de gestión de datos: el estado de navegación del usuario y los datos del dominio. Esta distinción responde a criterios distintos de persistencia, mutabilidad y coste de acceso, y determina la arquitectura general del sistema.

Los datos del dominio — entidades, metadatos y relaciones — se almacenan de forma persistente en una base de datos relacional. Esta decisión facilita que el sistema soporte datasets de tamaño potencialmente grandes sin imponer restricciones sobre la memoria disponible, y delega en el motor de base de datos la responsabilidad de ejecutar las operaciones de filtrado y recuperación de forma razonable.

El estado de navegación, en cambio, es ligero, momentáneo y específico de cada usuario: recoge los filtros acumulados, el historial de transiciones entre colecciones y los conjuntos de resultados en construcción. Hemos decidido mantenerlo en memoria dentro de la sesión HTTP por su naturalidad, dado su ciclo de vida acotado y su reducido tamaño, además de evitar serializar y deserializar el contexto de navegación en cada petición.

La consecuencia más relevante de esta separación es que el sistema no materializa subconjuntos intermedios de resultados: en cada petición, el estado en memoria se traduce directamente a una consulta sobre la base de datos. Esto simplifica el modelo de datos a costa de trasladar la complejidad al proceso de construcción dinámica de queries, que debe reflejar fielmente el contexto de navegación acumulado. Esta tensión — entre simplicidad del modelo y complejidad de las consultas — es una de las decisiones de diseño que hemos estimado que tendría un gran impacto en la implementación, y se aborda en detalle en el capítulo siguiente.

Capítulo 4

Desarrollo e Implementación

Este capítulo presenta la implementación del sistema, trasladando al código y al esquema de base de datos las decisiones de diseño expuestas en el Capítulo 3. Su organización sigue un orden ascendente, de los componentes que sostienen al resto a las extensiones que aportan funcionalidad específica.

La exposición comienza por el modelo de datos (Sección 4.1), que sustenta tanto la persistencia en PostgreSQL como la representación en memoria del estado de exploración. Sobre esa base, la clase `State` y su gestor, `StateManager` (Sección 4.2), articulan el contexto de un usuario en cada momento de la sesión. La Sección 4.3 describe los mecanismos de filtrado clásico y relacional, junto con la construcción dinámica de las consultas SQL parametrizadas que los trasladan al motor de base de datos. A partir de ese mecanismo común se introducen las dos extensiones que constituyen el aporte diferencial del sistema: la navegación transversal entre colecciones (Sección 4.4) y las operaciones de unión sobre contextos independientes (Sección 4.5). El capítulo cierra con una revisión de los obstáculos surgidos durante el desarrollo y las soluciones adoptadas en cada caso (Sección 4.8).

4.1. Modelo de datos

La aplicación maneja dos representaciones complementarias de la información. La primera es un esquema relacional persistente, alojado en PostgreSQL, en el que se almacenan las entidades importadas de las fuentes externas (TMDB, DBLP, etc.) junto con sus metadatos y vínculos. El detalle del pipeline de integración con cada fuente (descarga, procesado e ingesta) se documenta en el Apéndice B. La segunda es una representación en memoria del contexto de exploración de cada usuario, descrita en la Sección 4.2, que se traduce dinámicamente a consultas sobre el esquema persistente.

Esta sección se centra en la primera capa. Antes de detallar el esquema de tablas conviene partir de su contraparte conceptual, ya que el modelo Entidad-Relación captura las abstracciones que el esquema relacional se limita a codificar.

4.1.1. Modelo Entidad-Relación

El modelo conceptual se articula en torno a cuatro entidades fuertes (*Dataset*, *Collection*, *Entity* y *Metadata*) y una relación N–N reflexiva sobre *Entity*, dotada de atributo propio, que materializa los vínculos tipados entre entidades.

Un *Dataset* representa una fuente de datos completa, como el catálogo de TMDB o un volcado de DBLP, y agrupa varias *Collections*, que son los tipos de entidad expuestos por esa fuente. En el caso de TMDB, esas colecciones son *Movies*, *People*, *Studios*, *TV Series* y *Networks*. Cada *Entity* pertenece a una única *Collection*, y a esa entidad se asocian dos conjuntos abiertos de información: por un lado, una colección de *Metadata* formada por pares clave-valor que describen atributos del dominio (género, año de estreno, idioma original, etc.); por otro, un número arbitrario de *references* hacia otras entidades, etiquetadas con una *reason* que indica el tipo de relación que las une (*Director*, *Actor*, *Network*, etc.). La Figura 4.1 resume estas entidades y las cardinalidades entre ellas.

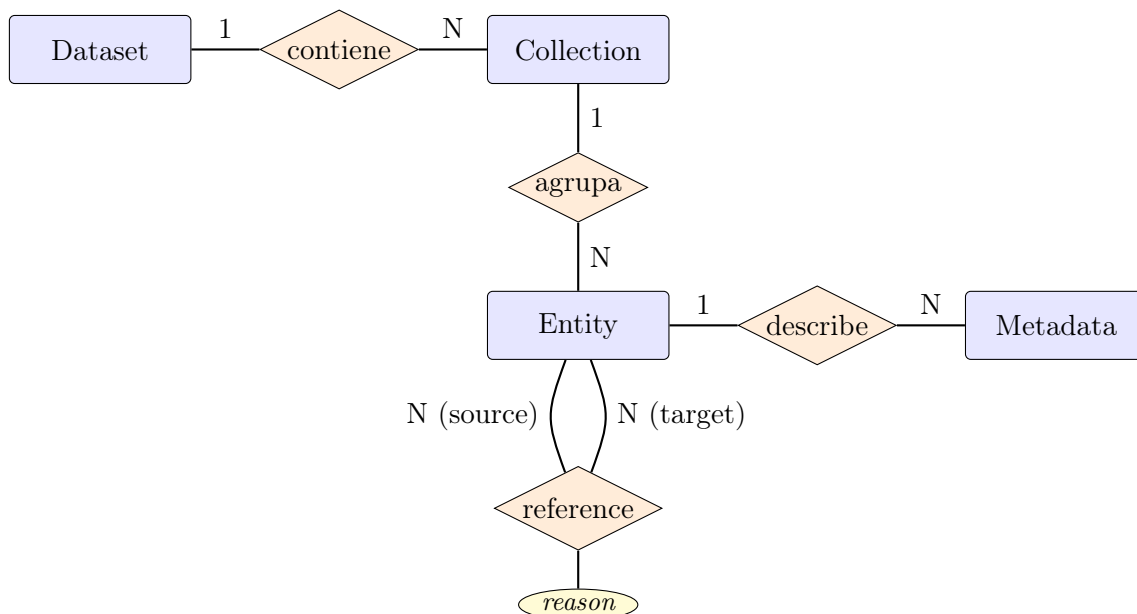


Figura 4.1: Diagrama Entidad-Relación del modelo de datos persistente.

La relación *reference* merece una observación específica. No es una asociación N–N pura, sino una relación con atributo propio, ya que cada vínculo lleva consigo la *reason* que lo etiqueta. Este matiz, aparentemente menor en la fase conceptual, condiciona directamente la forma del esquema relacional: una relación con atributo no admite la simplificación habitual a una tabla de cruce y exige una tabla independiente, como se verá a continuación.

4.1.2. Modelo relacional

La traducción del modelo conceptual a PostgreSQL distribuye estas entidades en cinco tablas: una por cada entidad fuerte y una adicional para la relación *reference*, que por las razones recién expuestas requiere existencia propia. La Figura 4.2 muestra el esquema implementado, con las claves primarias subrayadas y las claves

foráneas en *itálica*. Para no recargar el diagrama, las restricciones de unicidad y los índices secundarios se omiten en la figura y se comentan más adelante, ya que forman parte de las decisiones de diseño que motivan este esquema..

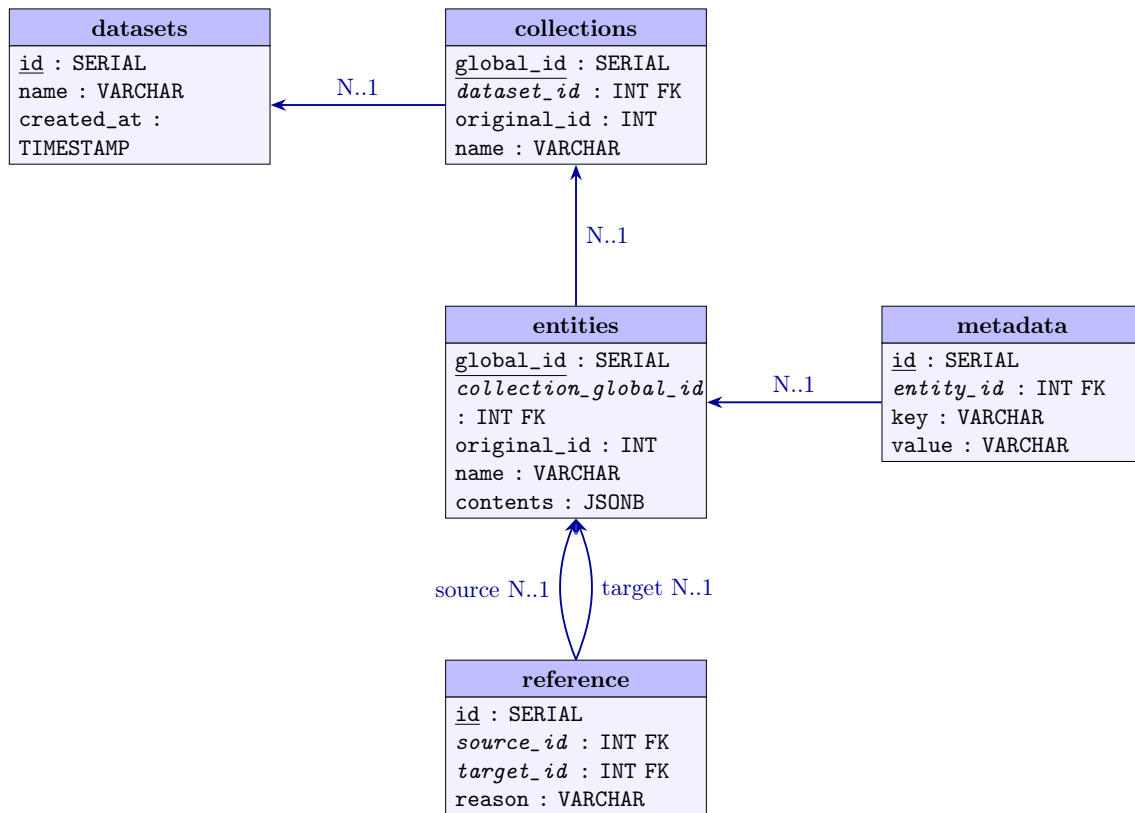


Figura 4.2: Modelo relacional implementado en PostgreSQL.

Más allá del paso mecánico de entidades a tablas, el esquema incorpora tres decisiones de diseño que conviene comentar de forma explícita, ya que responden a requisitos concretos del sistema y no a la traducción E-R por defecto.

Clave dual *global_id* / *original_id*. Tanto *collections* como *entities* mantienen dos identificadores. El primero, *global_id*, es una clave primaria autogenerada que garantiza la unicidad global dentro de la base de datos y se utiliza de manera uniforme como clave foránea desde el resto del esquema. El segundo, *original_id*, preserva el identificador heredado de la fuente externa y mantiene la trazabilidad con esa fuente, lo que permite reconciliar la información ante reimportaciones del mismo dataset sin romper las referencias internas. Las restricciones de unicidad (UQ(*dataset_id*, *original_id*) en *collections* y UQ(*collection_global_id*, *original_id*) en *entities*) imponen que el identificador original sea único *dentro* de su contexto, dejando que distintos datasets reutilicen los mismos identificadores numéricos sin colisión.

Columna *contents* de tipo JSONB. Cada entidad almacena, junto a sus metadatos normalizados, una copia del documento original en formato JSONB. Esta redundancia controlada cumple dos funciones complementarias. Por un lado, permite a la vista de detalle acceder a campos no indexados, como sinopsis, URLs de imagen u otros atributos del dominio que no se utilizan como facetas exploratorias, sin acoplar el esquema a cada particularidad de cada fuente. Por otro, mantiene

desacoplada la ingesta respecto al esquema relacional: si una fuente nueva expone campos adicionales, basta con incluirlos en `contents` sin necesidad de migrar la base de datos.

Índices orientados a la consulta exploratoria. Los índices definidos sobre `metadata(key, value)`, `reference(source_id, reason)` y `reference(target_id, reason)` no son casuales. Reflejan directamente la forma de las subconsultas `EXISTS` que el `NavigationDAO` genera al traducir el estado del usuario a SQL, mecanismo descrito en detalle en la Sección 4.3. Indexar `metadata` por la pareja (`key, value`) permite resolver eficientemente filtros del tipo “género = Acción”. Indexar la tabla `reference` por las dos columnas más selectivas en cada dirección, `source_id, reason` para el filtrado relacional directo y `target_id, reason` para la navegación inversa, cubre los dos sentidos en que el sistema atraviesa las relaciones entre colecciones.

4.2. Representación del contexto de exploración

Para comprender las decisiones de diseño del modelo de representación interno, es útil partir de un ejemplo concreto: la estructura del dataset de The Movie Database (TMDb), que fue la primera y principal fuente de datos utilizada durante el desarrollo. TMDb organiza su información en torno a cinco tipos de entidades: películas, series de televisión, personas, productoras y cadenas de televisión. Cada tipo de entidad posee atributos propios — como el género o la fecha de estreno en el caso de las películas, o el género y la fecha de nacimiento — y mantiene relaciones directas con entidades de otros tipos. Así, una película puede estar producida por una o varias productoras, contar con actores y miembros de equipo técnico de la colección de personas, una serie puede estar asociada a una cadena de televisión concreta, etc. O sea, al final tenemos una base de datos suficientemente compleja, con atributos específicos al dominio de cada entidad, y posibles relaciones de navegación, con aristas que conectan entidades de igual o distinta naturaleza.

Estas relaciones son bidireccionales en el dataset procesado: si una película referencia a una persona como *Director*, esa persona referencia a su vez a la película bajo la razón *Director*. Esto permite recorrer el grafo de relaciones en ambas direcciones desde cualquier punto de partida.

Al analizar esta estructura se infirieron las dos abstracciones fundamentales del modelo interno. Por un lado, los **metadatos**: atributos propios de cada entidad, representados como pares clave-valor, sobre los que el usuario puede establecer condiciones de inclusión o exclusión. Por otro lado, las **referencias**: vínculos entre entidades de distintas colecciones, etiquetados con un tipo de relación (*reason*), que permiten tanto filtrar por asociación como navegar de una colección a otra siguiendo esos vínculos. Esta distinción — metadatos para describir, referencias para relacionar — es la que articula toda la representación del contexto de exploración.

Estas dos abstracciones, metadatos y referencias, se materializan en código a través de tres clases que dan forma al modelo en memoria. La Figura 4.3 las resume y muestra cómo se relacionan entre sí, sirviendo de mapa antes de entrar en el detalle de cada una. La clase `State` encapsula los filtros activos y el historial de transiciones

de una sesión; **StateManager** coordina el estado activo, la pila de historial y los contextos guardados para unión; **Link**, por último, captura una transición concreta entre colecciones.

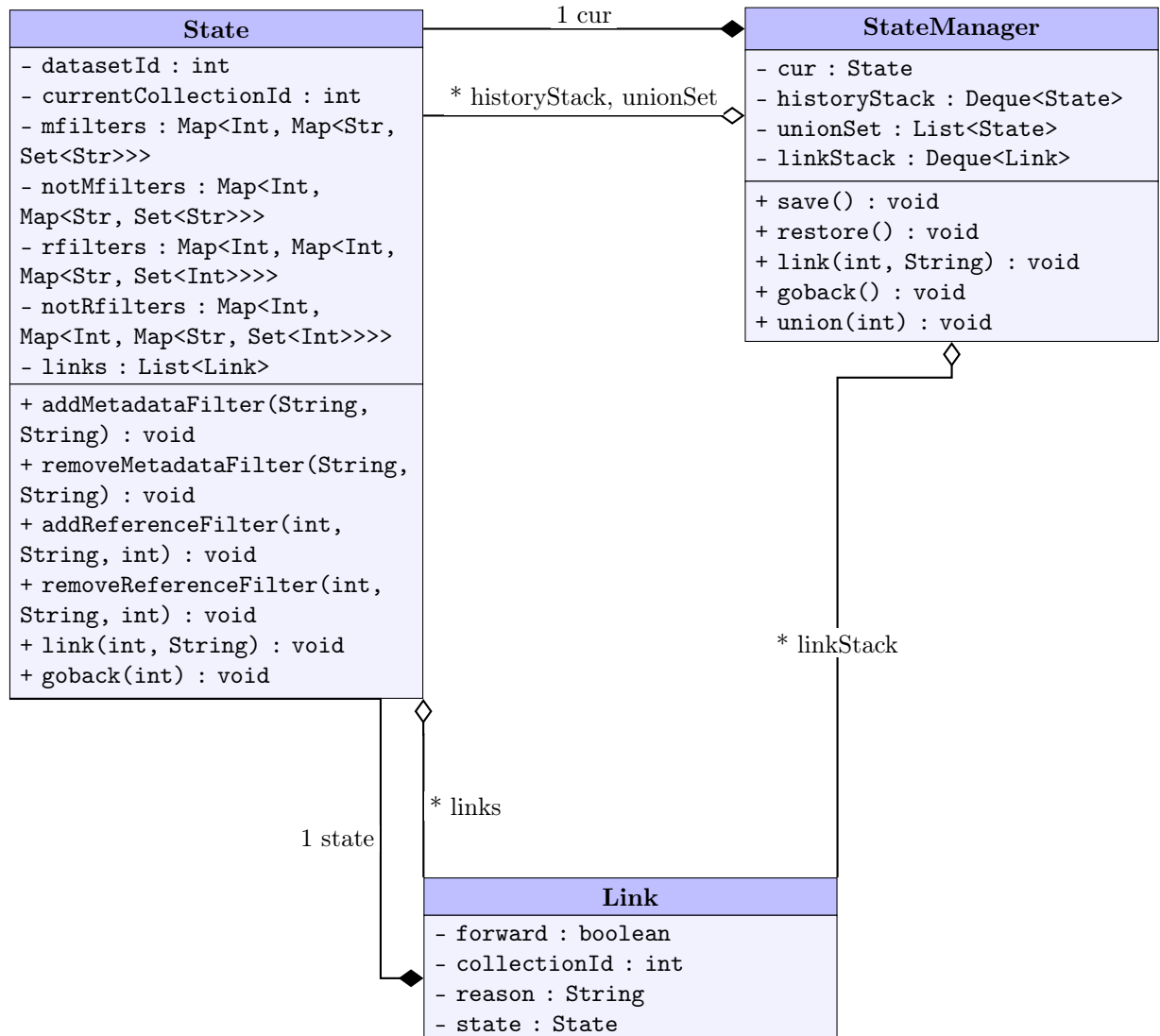


Figura 4.3: Diagrama UML de clases del modelo de exploración en memoria. **State** en el momento de la transición.

4.2.1. La clase State: filtros y contexto acumulado

La clase **State** representa el contexto de exploración de un usuario en un momento dado. Encapsula, por un lado, los filtros activos sobre la colección actual y, por otro, el historial de transiciones realizadas entre colecciones durante la sesión actual.

Los filtros se organizan en cuatro estructuras **HashMap** anidadas. Los filtros de metadatos (**mfilters**) mapean cada atributo a un conjunto de valores seleccionados, expresando que las entidades resultantes deben satisfacer al menos uno de dichos valores por atributo. Su contraparte negativa (**notMfilters**) excluye las entidades

que coincidan con los valores indicados. Los filtros de referencia (**rfilters**) operan sobre relaciones entre colecciones: para cada colección referenciada y tipo de relación, almacenan el conjunto de identificadores de entidades con los que debe existir vínculo. De igual modo, **notRfilters** excluye las entidades que presenten dichos vínculos. Todos estos mapas están indexados por el identificador de la colección activa, de forma que el mismo objeto **State** puede acumular filtros correspondientes a distintas colecciones visitadas a lo largo de la navegación.

Junto a los filtros, **State** mantiene una lista de objetos **Link**. Cada **Link** registra una transición entre colecciones: almacena la dirección de la transición (bidireccional), la colección de origen, el tipo de relación seguida y una copia del estado en el momento de realizar la transición. Esta lista constituye el historial de navegación y es lo que permite reconstruir, a la hora de ejecutar una consulta, el camino completo que el usuario ha recorrido entre colecciones.

State implementa **Serializable** y facilita un constructor de copia que realiza una copia profunda de todas sus estructuras internas, necesaria para preservar instantáneas del estado en distintos puntos de la sesión.

A modo de ejemplo concreto, la Figura 4.4 muestra cómo se ve un **State** tras una secuencia típica de operaciones: el usuario parte de la colección *Movies* en el dataset TMDb, añade un filtro de metadatos *Genres = Action* y a continuación navega hacia *People* siguiendo la relación *Director*. La colección activa ha pasado a *People*, pero el estado conserva tanto los filtros aplicados previamente sobre *Movies* como una entrada en **links** que registra la transición y guarda una instantánea del estado en el instante del salto.

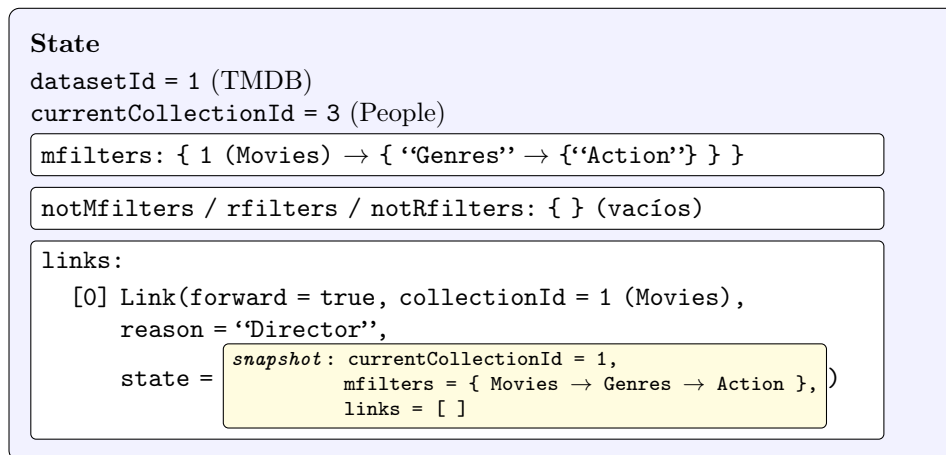


Figura 4.4: Instantánea de un **State** tras filtrar películas por *Genres = Action* y navegar a *People* por la relación *Director*. La colección activa es ya *People*, pero los filtros aplicados sobre *Movies* y la transición que cruzó al usuario quedan registrados en **mfilters** y **links**, respectivamente.

4.2.2. StateManager: gestión del ciclo de vida del estado

StateManager es el punto de entrada a través del cual los controladores interactúan con el estado de navegación. Se instancia al inicio de la sesión, cuando el usuario selecciona un dataset y una colección de partida, y se almacena como atributo de la

sesión HTTP bajo la clave `navState`.

Internamente, `StateManager` coordina tres estructuras con ciclos de vida distintos. La primera es el estado activo (`cur`), que representa el contexto de exploración en curso y sobre el que se aplican todas las operaciones de filtrado y navegación. La segunda es una pila de historial (`historyStack`), una `Deque` que el `NavigationServlet` alimenta invocando `save()` antes de cada acción, lo que permite revertir el estado mediante `restore()`. La tercera es el `unionSet`, una lista de estados congelados que acumula los contextos que el usuario ha marcado para combinar mediante unión; al invocar `union()`, el estado activo se añade al `unionSet` y `cur` se reinicializa con un estado limpio para continuar la exploración de manera independiente.

A estas tres estructuras se suma una pila auxiliar, `linkStack`, que registra los `Link` hacia delante creados por `link()` y que `goback()` consume en orden inverso para saber qué transición revertir. A diferencia de las anteriores, `linkStack` no representa contexto de exploración por sí misma: el historial efectivo, el que el `NavigationDAO` traduce a SQL, vive dentro de cada `State` en su campo `links`.

Es responsabilidad íntegra de `StateManager` la gestión de coherencia entre estas tres estructuras. Por ejemplo, al ejecutar `link()` o `union()`, es `StateManager` quien decide cuándo realizar una copia profunda del estado activo antes de cambiarlo, garantizando que los estados almacenados en el historial y en el `unionSet` no se vean afectados por operaciones posteriores. La API pública de `StateManager` refleja el vocabulario de acciones disponibles desde el frontend — `addMetadataFilter`, `removeReferenceFilter`, `link`, `goback`, `save`, `restore` y `union` — delegando en `State` las operaciones sobre filtros, y aplicando directamente la lógica de coordinación cuando la operación afecta a más de una de sus estructuras internas.

4.2.3. API REST: endpoints expuestos al cliente

El modelo de exploración descrito hasta aquí permanecería inerte sin un mecanismo que lo expusiese al exterior. Esa función la cumple una API HTTP basada en *servlets* anotados con `@WebServlet`, en la que cada *servlet* se ocupa de un recurso bien delimitado y delega la lógica en `StateManager` o en `NavigationDAO` según corresponda.

La traducción del modelo a esta API sigue una asimetría intencionada entre escrituras y lecturas. Todas las acciones que modifican el estado de exploración se canalizan a través de un único *endpoint*, `/api/navigation`, que recibe en el cuerpo de la petición un campo `action` cuyo valor discrimina entre las operaciones disponibles: añadir y retirar filtros de metadatos y de referencia, navegar por un enlace, deshacer una transición, restaurar el estado previo a una unión, abrir un nuevo bloque de unión o cambiar de colección. De este modo, el vocabulario público de `StateManager` se proyecta directamente sobre la superficie HTTP del backend, sin necesidad de multiplicar *endpoints* por cada operación. Las consultas, por el contrario, sí justifican *endpoints* independientes, ya que cada una devuelve una representación distinta del estado actual: las entidades visibles, el detalle de una entidad concreta, las facetas de metadatos, las facetas de referencia o los enlaces disponibles.

La correspondencia entre cada URL y la clase `*Servlet` que la atiende es direc-

ta: `DatasetsServlet` resuelve `/api/datasets`, `MetadataFacetsServlet` resuelve `/api/facets/metadata`, y así sucesivamente. Todas las peticiones que requieren contexto de exploración recuperan el `StateManager` asociado a la sesión mediante `req.getSession()` y devuelven HTTP 400 cuando no existe un estado activo, lo que garantiza que ningún *endpoint* opere sobre un cliente que aún no se ha inicializado.

El conjunto completo de *endpoints* expuestos por el backend, junto con sus parámetros principales y la acción que desencadenan, se resume en la Tabla 4.1.

Método	URL	Parámetros	Acción
GET	<code>/api/datasets</code>	—	Lista los <i>datasets</i> disponibles
GET	<code>/api/collections</code>	<code>datasetId</code>	Lista las colecciones de un <i>dataset</i>
GET	<code>/api/session</code>	—	Devuelve el estado de la sesión activa
POST	<code>/api/session</code>	<code>datasetId</code> , <code>collectionId</code> (cuerpo JSON)	Inicializa una sesión sobre un <i>dataset</i> y una colección
GET	<code>/api/entities</code>	<code>page</code> , <code>size</code>	Entidades visibles tras aplicar los filtros activos (paginado)
GET	<code>/api/entity</code>	<code>id</code> , <code>collectionId</code> (opcional)	Detalle de una entidad: metadatos, recursos y referencias
GET	<code>/api/facets/metadata</code>	—	Facetas de metadatos y filtros activos
GET	<code>/api/facets/references</code>	—	Facetas de referencia y filtros activos
GET	<code>/api/facets/links</code>	—	Relaciones disponibles desde la colección actual
POST	<code>/api/navigation</code>	<code>action</code> + parámetros propios de la acción	Aplica una acción sobre el estado: filtros (<code>add_mfilter</code> , <code>rm_mfilter</code> , <code>add_not_mfilter</code> , <code>rm_not_mfilter</code> , <code>add_rfilter</code> , <code>rm_rfilter</code> , <code>add_not_rfilter</code> , <code>rm_not_rfilter</code>), navegación (<code>link</code> , <code>goback</code>), unión (<code>union</code> , <code>change</code>) y restauración (<code>restore</code>)
GET	<code>/api/union</code>	<code>page</code> , <code>size</code>	Entidades del conjunto de unión (paginado)
GET	<code>/api/resource/*</code>	<code>ruta</code> como <code>pathInfo</code>	Sirve archivos del directorio de <i>uploads</i> (imágenes y recursos asociados a entidades)

Tabla 4.1: Endpoints REST expuestos por el backend.

4.3. Filtrado clásico y relacional

En TMDb, la exploración de un dataset raramente se agota filtrando por atributos propios de una colección, por cuenta del tamaño y complejidad de la colección.

Un caso de uso habitual es, por ejemplo, encontrar películas de un género concreto dirigidas por una persona específica, o excluir series producidas por una determinada compañía. El primer tipo de condición opera sobre los metadatos de la entidad; el segundo, sobre sus relaciones con entidades de otra colección. Esta distinción entre *filtrado clásico* — sobre atributos — y *filtrado relacional* — sobre vínculos — motivó el diseño de dos mecanismos de filtrado independientes y de naturaleza distinta pero que se aplican de forma conjunta en cada consulta.

Ambos mecanismos admiten además una variante por la negativa: es posible no solo incluir entidades que cumplan alguna condición, sino también excluir explícitamente las que la cumplan. La combinación de inclusiones y exclusiones, tanto sobre metadatos como sobre referencias, define el espacio de resultados que el usuario está explorando en cada momento.

4.3.1. Construcción dinámica de queries SQL

El núcleo de la capa de acceso a datos es la traducción del estado de navegación a una query SQL ejecutable. Esta traducción se ensambla de manera incremental: en cada petición, el `NavigationDAO` construye la query de forma incremental añadiendo cláusulas `AND` sobre una consulta base que selecciona entidades de la colección activa.

El `appendFilterClauses` es el método central de este proceso, que recibe el `StringBuilder` de la query en construcción, la lista de parámetros y el estado actual, y le añade todas las condiciones derivadas de los filtros acumulados. Para el caso más simple — sin historial de navegación entre colecciones — delega directamente en `appendStateFilters`, que itera sobre los cuatro mapas de filtros del estado (`mfilters`, `notMfilters`, `rfilters`, `notRfilters`) y genera una subconsulta `EXISTS` o `NOT EXISTS` por cada condición activa. Cuando el estado contiene además un historial de *links*, `appendFilterClauses` actúa de forma recursiva, generando subconsultas anidadas que incorporan el contexto de cada transición previa; este mecanismo se describe en detalle en la sección 4.4.

Todas las queries se construyen con parámetros posicionales (?), nunca interpolando valores directamente en el SQL. Los valores se acumulan en una lista `params` y se asignan posteriormente mediante `stmt.setObject()`, lo que delega en el `PreparedStatement` de JDBC la responsabilidad de escapar los valores antes de enviarlos al motor, para prevenir inyecciones SQL. El coste de filtrado se traslada íntegramente a PostgreSQL, sin materializar subconjuntos intermedios de resultados en memoria.

4.3.2. Filtros de metadatos

Los filtros de metadatos permiten restringir los resultados de una colección en función de los atributos clave-valor asociados a sus entidades. En el contexto de TMDB, por ejemplo, esto equivale a operaciones como seleccionar únicamente películas del género *Action*, o excluir aquellas con idioma original distinto del inglés.

Cada filtro de inclusión se traduce en una subconsulta `EXISTS` sobre la tabla `metadata`. Si el usuario ha seleccionado múltiples valores para el mismo atributo, se genera una cláusula `EXISTS` independiente por cada valor, lo que impone

que la entidad deba satisfacer *todos* ellos simultáneamente. Los filtros de exclusión (`notMfilters`) siguen la misma estructura pero con `NOT EXISTS`, como muestra el Código 4.1.

Código 4.1: Filtros de metadatos

```
1 for (var entry : state.getMfilters().entrySet()) {
2     String attr = entry.getKey();
3     for (String val : entry.getValue()) {
4         sql.append(" AND EXISTS (SELECT 1 FROM metadata " + m
5             + " WHERE " + m + ".entity_id = " + e + ".
6                 global_id"
7             + " AND " + m + ".key = ? AND " + m + ".value = ?)
8                 ");
9         params.add(attr);
10        params.add(val);
11    }
12 }
```

Esta elección implica una semántica de conjunción estricta entre valores del mismo atributo, lo cual puede parecer contraintuitivo cuando los valores son mutuamente excluyentes — como ocurre con **Genres** en TMDb, donde una película puede pertenecer a varios géneros a la vez. En ese caso, seleccionar *Action* y *Drama* devuelve las películas que tienen *ambos* géneros, no las que tienen alguno de los dos. Esta semántica se mantuvo de forma deliberada para garantizar consistencia con el comportamiento del resto de filtros del sistema, y la describiremos como parte de los obstáculos encontrados en la sección 4.8.

4.3.3. Filtros de referencia

Los filtros de referencia permiten restringir los resultados de una colección en función de sus vínculos con entidades concretas de otra colección. En TMDb, esto cubre casos como mostrar únicamente las películas en las que una persona determinada participó como *Director*, o excluir películas con un *Actor* específico.

Cada filtro de referencia está identificado por tres elementos: la colección referenciada, el tipo de relación (*reason*) y el identificador de la entidad concreta. Al igual que con los metadatos, se genera una subconsulta `EXISTS` por cada combinación activa, realizando un `JOIN` entre la tabla `reference`, la tabla `entities` y la tabla `collections` para verificar la existencia de dicho vínculo, tal y como recoge el Código 4.2.

Los filtros de exclusión (`notRfilters`) utilizan `NOT EXISTS` con la misma estructura. Todos los filtros activos — de metadatos y de referencia, en sus variantes positiva o negativa — se aplican conjuntamente sobre la misma consulta base, de forma que el resultado final es la intersección de todas las condiciones establecidas por el usuario en ese momento.

Código 4.2: Filtros de referencia

```

1 sql.append(" AND EXISTS ("
2   + "SELECT 1 FROM reference " + r
3   + " JOIN entities " + re + " ON " + r + ".target_id = " +
4     re + ".global_id"
5   + " JOIN collections " + rc + " ON " + re + ".
6     collection_global_id = "
7     rc + ".global_id"
8   + " WHERE " + r + ".source_id = " + e + ".global_id"
9   + " AND " + rc + ".original_id = ?"
10  + " AND " + rc + ".dataset_id = ?"
11  + " AND " + r + ".reason = ?"
12  + " AND " + re + ".original_id = ?) ");

```

4.3.4. Facetas: cómputo dinámico de valores disponibles

Las facetas son el mecanismo por el que el sistema informa al usuario de qué valores siguen siendo relevantes dado el contexto de filtrado actual: qué géneros tienen al menos una película entre los resultados visibles, qué directores aparecen en esas películas, o a qué cadenas están asociadas las series que quedan tras aplicar los filtros. Este cómputo es siempre dinámico: se recalcula en cada petición a partir del conjunto de entidades que satisfacen el estado actual.

El sistema distingue dos tipos de facetas, ambas servidas por `FacetsServlet` a través del endpoint `GET /api/facets`:

Las **facetas de metadatos** se obtienen mediante `getFacets()`, que construye una subconsulta con los identificadores de las entidades actualmente visibles — aplicando todos los filtros del estado — y sobre ella ejecuta un `GROUP BY` sobre la tabla `metadata`, según se aprecia en el Código 4.3.

Código 4.3: Cómputo dinámico de facetas

```

1 String sql =
2   "SELECT m.key, m.value, COUNT(*) as cnt "
3   + "FROM metadata m "
4   + "WHERE m.entity_id IN (" + subquery + ") "
5   + "GROUP BY m.key, m.value "
6   + "ORDER BY m.key, cnt DESC";

```

El resultado de la query se recorre fila a fila y se agrupa en un `LinkedHashMap` indexado por atributo (`key`). Por cada fila, se construye un mapa con tres campos: el valor del atributo (`value`), su frecuencia entre las entidades visibles (`count`), y un booleano `active` que indica si ese valor ya está seleccionado como filtro en el estado actual, que lo consulta mediante `isFilterActive()`. El uso de `LinkedHashMap` preserva el orden de inserción, que viene determinado por el `ORDER BY m.key, cnt DESC` de la query: los atributos aparecen en orden alfabético y, dentro de cada atributo, los valores se listan de mayor a menor frecuencia. Este orden es el que el frontend recibe y renderiza directamente, sin necesidad de reordenar en el cliente.

Las **facetas de referencia** se obtienen mediante `getReferenceFacets()`, que sobre la misma subconsulta de entidades visibles realiza un `JOIN` con la tabla `reference` para descubrir qué entidades de otras colecciones están vinculadas a los resultados actuales, agrupadas por colección y tipo de relación. El resultado expone, por ejemplo, qué personas concretas aparecen como *Director* entre las películas visibles, junto con el número de películas en las que aparecen.

Junto a las facetas, el mismo endpoint devuelve los *links* disponibles — obtenidos mediante `getAvailableLinks()` — que indican a qué otras colecciones es posible navegar desde el conjunto actual, y los filtros activos en ese momento (`activeFilters`), permitiendo al frontend reflejar el estado de selección sin necesidad de mantener dicha información en el cliente.

4.4. Navegación transversal

Hasta ahora, hemos descrito los mecanismos de filtrado que operan dentro de una única colección: dadas las entidades de *Movies*, se restringe el conjunto según atributos propios o según vínculos con entidades concretas de otras colecciones. Entretanto, el sistema permite también *trasladarse* de una colección a otra siguiendo una relación, manteniendo y llevando el contexto de filtrado acumulado. Este mecanismo es lo que se denomina *navegación transversal*.

Damos un ejemplo concreto en TMDb: el usuario está explorando películas con el filtro de género *Action* activo. Desde ese contexto, decide navegar hacia la colección *People* siguiendo la relación *Actor*, para ver qué actores aparecen en esas películas de acción. Tras el salto, la colección activa pasa a ser *People*, pero el sistema recuerda que solo interesan las personas vinculadas al subconjunto de películas que cumplan el filtro establecido anteriormente. El historial de transiciones compone, por tanto, parte del contexto de exploración con responsabilidad expresa, no un simple registro de navegación.

4.4.1. El mecanismo de link y goback

La transición entre colecciones se realiza mediante la operación `link`, que recibe el identificador de la colección destino y el tipo de relación a seguir. Su implementación en `StateManager`, reproducida en el Código 4.4, tiene cuatro efectos.

Código 4.4: Operación link

```
1 public void link(int targetCollectionId, String reason) {
2     Link newLink = new Link(true, this.cur.
3         getCurrentCollectionId(),
4         reason, new State(this.cur));
5     this.cur.link(targetCollectionId, reason);
6     this.cur.getLinks().add(newLink);
7     this.linkStack.push(newLink);
8 }
```

Primero, se construye un nuevo objeto `Link` marcado como transición hacia delante (`forward = true`). Este objeto almacena la colección de origen, el tipo de relación seguida y una copia profunda del estado activo en el momento de la transición, preservando así los filtros que estaban activos antes del salto. Segundo, se actualiza la colección activa en `cur` a la colección destino. Tercero, el nuevo `Link` se añade a la lista de transiciones del propio `State` (`cur.getLinks()`), que es la que el `NavigationDAO` consume al construir las queries. Cuarto, ese mismo `Link` se apila en `linkStack`, la pila auxiliar que mantiene `StateManager` para conocer cuál es la última transición revertible en una eventual operación de `goback`.

La Figura 4.5 resume la secuencia de mensajes entre el cliente, el `NavigationServlet` y el modelo en memoria al ejecutar una operación de `link`. Se observa que el servlet invoca primero `save()` (volcando una copia del estado actual a `historyStack`) y a continuación `link()`, dentro de la cual `StateManager` actualiza tanto la colección activa de `cur` como sus listas internas `links` y `linkStack`.

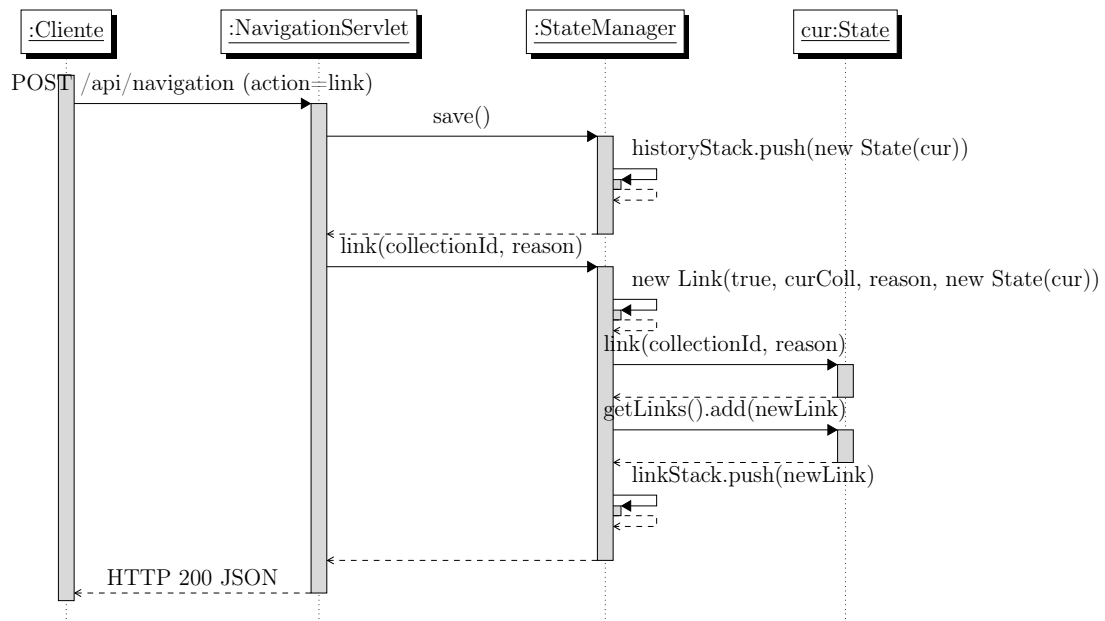


Figura 4.5: Secuencia de mensajes durante la operación `link`. `save()` guarda una copia profunda del estado en `historyStack`; a continuación `link()` crea el nuevo `Link` con instantánea del estado previo, actualiza `cur` y lo apila tanto en `cur.getLinks()` como en `linkStack`.

La operación `goback` revierte la última transición hacia delante realizada en la sesión. Su implementación se apoya en `linkStack`: extrae de la pila el último `Link` apilado por `link()`, construye un nuevo `Link` con dirección hacia atrás (`forward = false`) que registra esa reversión en la lista de transiciones de `cur`, y restaura como colección activa la colección de origen almacenada en el `Link` extraído. El Código 4.5 reproduce el método completo.

El hecho de que `goback` registre un nuevo `Link` en `cur.getLinks()` en lugar de eliminar el último tiene una consecuencia relevante: ese historial es siempre creciente y nunca se borra. Esto significa que la query SQL resultante debe recorrer toda la cadena de transiciones acumuladas, incluidos los retrocesos, para reconstruir fielmente

Código 4.5: Operación goback

```

1 public void goback() {
2     Link last = this.linkStack.pop();
3     Link backLink = new Link(false, this.cur.
4         getCurrentCollectionId(),
5         last.getReason(), new State(this.
6             cur));
7     this.cur.goback(last.getCollectionId());
8     this.cur.getLinks().add(backLink);
9 }

```

el contexto de exploración. La pila `linkStack`, en cambio, sí pierde la entrada con cada `goback`, pero su rol es estrictamente operacional: indicar cuál es la próxima reversión legal y nada más.

Esta asimetría queda reflejada en la Figura 4.6: `linkStack.pop()` consume la cabeza de la pila, mientras que el nuevo `Link` de retroceso se añade únicamente a `cur.getLinks()`, dejando la cadena efectiva del historial monótonamente creciente.

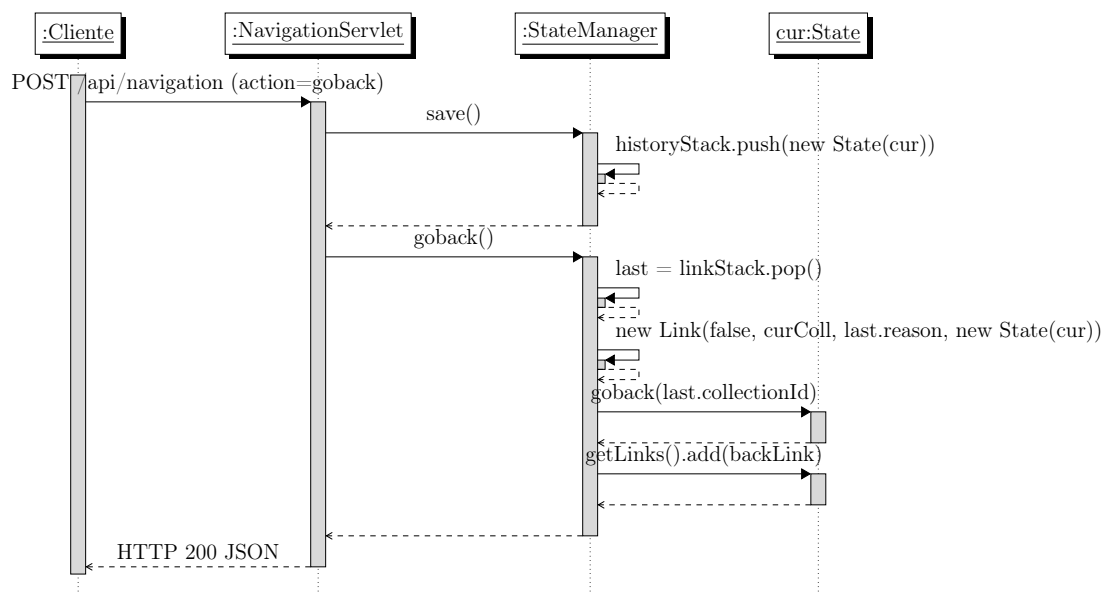


Figura 4.6: Secuencia de mensajes durante la operación `goback`. `linkStack` sí se reduce con `pop()`, pero el historial efectivo en `cur.getLinks()` crece con un nuevo `Link` marcado como retroceso (`forward = false`), preservando toda la cadena de transiciones para reconstruir el contexto en SQL.

4.4.2. Subconsultas anidadas: traducción del historial de *links* a SQL

Cuando el estado contiene un historial de transiciones, `appendFilterClauses` no puede limitarse a aplicar los filtros del estado activo: debe también expresar en SQL la restricción que impone cada transición puesta anteriormente. Para ello utiliza una sobrecarga recursiva que recorre la lista de transiciones del estado (`state.getLinks()`)

desde el último *Link* hacia el primero, generando en cada nivel una subconsulta anidada.

La estructura de la recursión tiene la siguiente definición: para cada nivel de profundidad, se aplican primero los filtros del estado en ese nivel mediante `appendStateFilters`, y a continuación se genera una cláusula `AND ... IN (SELECT ...)` que restringe las entidades del nivel actual a aquellas que, a través de la relación registrada en el *Link*, están vinculadas a entidades del nivel siguiente. La dirección del *Link* determina qué columna de la tabla `reference` actúa como origen y cuál como destino, como se observa en el Código 4.6.

Código 4.6: Subconsultas anidadas

```

1  if (link.isForward()) {
2      sql.append(" AND " + e + ".global_id IN ("
3          + "SELECT " + r_link + ".target_id FROM reference " +
4              r_link
5          + " JOIN entities " + next_e + " ON " + r_link + ".
6              source_id = "
7          + next_e + ".global_id"
8          + " JOIN collections " + next_c + " ON " + next_e
9          + ".collection_global_id = " + next_c + ".global_id"
10         + " WHERE " + next_c + ".dataset_id = ? AND " + next_c
11         + ".original_id = ? AND " + r_link + ".reason = ? ");
12 } else {
13     sql.append(" AND " + e + ".global_id IN ("
14         + "SELECT " + r_link + ".source_id FROM reference " +
15             r_link
16         + " JOIN entities " + next_e + " ON " + r_link + ".
17             target_id = "
18         + next_e + ".global_id"
19         + " JOIN collections " + next_c + " ON " + next_e
20         + ".collection_global_id = " + next_c + ".global_id"
21         + " WHERE " + next_c + ".dataset_id = ? AND " + next_c
22         + ".original_id = ? AND " + r_link + ".reason = ? ");
23 }
24 appendFilterClauses(sql, params, link.getState(), linkList,
25     index - 1, depth + 1);
26 sql.append(") ");

```

Cada llamada recursiva abre una subconsulta que se cierra después de que la llamada interna haya completado su propio nivel, resultando en una estructura de paréntesis anidados que refleja correctamente la cadena de transiciones del usuario. Los identificadores de tabla (`e`, `e1`, `e2`, `r10`, `r11`...) se generan dinámicamente a partir del parámetro `depth` para evitar colisiones entre niveles.

Retomando el ejemplo anterior — películas de acción, navegación hacia *People* por la relación *Actor* — la query resultante para obtener los actores visibles adopta la forma mostrada en el Código 4.7.

La subconsulta interior opera sobre la colección *Movies* con sus filtros activos, y la consulta exterior recupera únicamente las personas de *People* que aparecen como

Código 4.7: Query SQL resultante de la navegación transversal

```

1 SELECT DISTINCT e.original_id, e.name
2 FROM entities e
3 JOIN collections c ON e.collection_global_id = c.global_id
4 WHERE c.dataset_id = ? AND c.original_id = ?    -- People
5 AND e.global_id IN (
6     SELECT rl0.source_id FROM reference rl0
7     JOIN entities e1 ON rl0.target_id = e1.global_id
8     JOIN collections c1 ON e1.collection_global_id = c1.
9         global_id
10    WHERE c1.dataset_id = ? AND c1.original_id = ?    -- Movies
11    AND rl0.reason = 'Actor'
12    AND EXISTS (SELECT 1 FROM metadata m0    -- Genres = Action
13                WHERE m0.entity_id = e1.global_id
14                AND m0.key = ? AND m0.value = ?)

```

origen de una relación *Actor* hacia alguna de esas películas.

4.5. Operaciones de conjuntos

Los mecanismos de filtrado y navegación transversal descritos hasta ahora operan siempre sobre un único contexto de exploración activo. Sin embargo, hay casos de uso que requieren combinar los resultados de dos o más contextos contruidos de manera independiente. Un posible ejemplo concreto en TMDb: el usuario quiere obtener todas las personas que han trabajado como *Director* en películas de acción o como *Actor* en series de ciencia ficción. No sabemos si hay intersección entre ambos conjuntos, y construir ambas condiciones simultáneamente dentro de un único estado sería impracticable, y posiblemente imposible con la semántica del filtrado descrito en la sección 4.3.

Para cubrir este caso, el sistema incorpora una operación de unión, donde el usuario puede guardar el contexto actual, iniciar una nueva exploración desde cero y, al consultar los resultados, obtener las entidades que satisfacen cualquiera de los contextos acumulados hasta entonces.

4.5.1. Construcción del unionSet

La operación `union` en `StateManager` guarda el contexto actual para una unión posterior y reinicializa el estado activo desde cero. El Código 4.8 reproduce su implementación, que delega la parte de reinicio en el método auxiliar `change`.

Primero, se añade el estado activo actual al `unionSet` sin realizar ninguna copia: `cur` va a ser reemplazado inmediatamente, por lo que no hay riesgo de que modificaciones posteriores alteren la entrada guardada, que ya contiene su propio historial de transiciones dentro de `cur.getLinks()`. A continuación se invoca `change`, que vacía la `historyStack` y reinicializa `cur` con un `State` limpio sobre la colección

Código 4.8: Operación union

```

1 public void union(int collectionId) {
2     this.unionSet.add(this.cur);
3     change(collectionId);
4 }
5
6 public void change(int collectionId) {
7     this.historyStack.clear();
8     this.cur = new State(this.datasetId, collectionId);
9 }

```

indicada. Ese nuevo estado arranca con su propia lista de transiciones vacía, lo que desvincula por completo el contexto recién iniciado del anterior.

El `unionSet` puede acumular un número arbitrario de entradas repitiendo esta operación. Cada entrada es un estado completo e independiente, con su propia colección activa, sus propios filtros y su propio historial de transiciones.

La Figura 4.7 ilustra la secuencia de mensajes correspondiente. A diferencia de `link` y `goback`, la operación no toca `State` directamente: el estado anterior se añade tal cual a `unionSet` (sin copia profunda, dado que `cur` va a ser reemplazado), y `change()` instala un `State` recién creado como nuevo contexto activo.

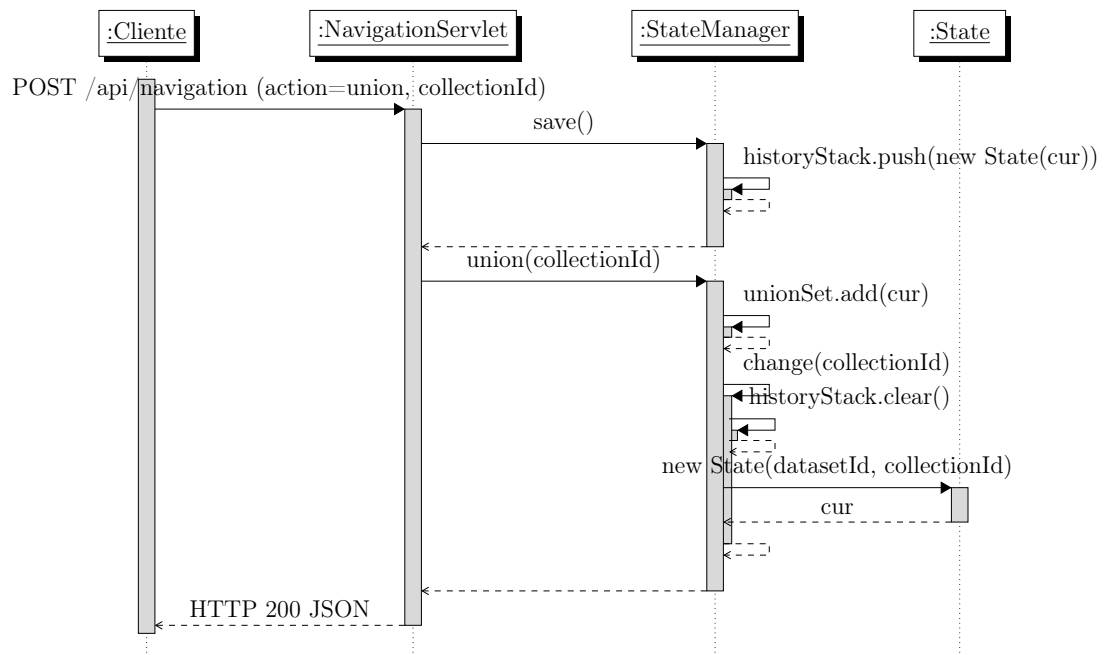


Figura 4.7: Secuencia de mensajes durante la operación `union`. El estado activo pasa a `unionSet` sin copia profunda y `change()` vacía `historyStack` antes de reinicializar `cur` con un `State` limpio sobre la colección indicada. La pila `linkStack` no se modifica.

4.5.2. Resolución de la unión en base de datos

La consulta del `unionSet` se realiza a través del endpoint `GET /api/union`, servido por `UnionServlet`, que delega en los métodos `getUnionEntities()` y `countUnionEntities()` del `NavigationDAO`.

Ambos métodos construyen una única query que itera sobre todas las entradas del `unionSet` y concatena un bloque de condiciones por entrada, separados por `OR`, como ilustra el Código 4.9.

Código 4.9: Query de unión

```
1 for (int i = 0; i < unionSet.size(); i++) {  
2     State state = unionSet.get(i);  
3     if (i > 0) sql.append(" OR ");  
4     sql.append(" (c.dataset_id = ? AND c.original_id = ? ");  
5     params.add(state.getDatasetId());  
6     params.add(state.getCurrentCollectionId());  
7     appendFilterClauses(sql, params, state);  
8     sql.append(" ) ");  
9 }
```

Cada bloque es estructuralmente idéntico a la query que se generaría para ese estado en una consulta normal, aplicando `appendFilterClauses` sobre el estado correspondiente, lo que incluye sus filtros de metadatos, sus filtros de referencia y, si procede, las subconsultas anidadas derivadas de su historial de transiciones. La unión entre bloques se expresa con `OR` a nivel de la cláusula `WHERE`, y el `DISTINCT` en el `SELECT` garantiza que las entidades presentes en más de un bloque no aparezcan duplicadas en los resultados.

Una consecuencia de este diseño es que la unión puede combinar entradas sobre colecciones distintas. En el ejemplo anterior, el primer bloque opera sobre *People* filtrado por películas de acción con relación *Director*, y el segundo también sobre *People* pero filtrado por series de ciencia ficción con relación *Actor*. Que ambas entradas compartan colección activa no es un requisito del sistema: el campo `collection_id` devuelto por `getUnionEntities()` indica a qué colección pertenece cada entidad del resultado, permitiendo al frontend distinguirlas cuando fuera necesario.

4.6. Desarrollo del frontend

Concluida la descripción del backend, queda por presentar la capa que el usuario percibe directamente. El frontend se concibió como una superficie de presentación deliberadamente delgada, cuya única responsabilidad consiste en renderizar el estado actual del servidor y en traducir las acciones del usuario en peticiones HTTP contra los *endpoints* descritos en la Tabla 4.1. El cliente no mantiene una copia del modelo de datos ni implementa una lógica propia de navegación: el `StateManager` del servidor es la única fuente de verdad, y la interfaz se limita a presentar lo que recibe y a invocar el *endpoint* adecuado ante cada acción del usuario.

Tecnología y filosofía

Para una capa con esa responsabilidad acotada, recurrir a un *framework* reactivo como React, Vue o Angular habría supuesto un sobrecoste injustificado. El cliente está implementado, por tanto, en JavaScript moderno con módulos *ES*, sin dependencias de interfaz. Tres razones sostienen esta decisión. En primer lugar, su lógica se reduce a peticiones HTTP y a manipulación directa del DOM, sin estructuras de datos derivadas que justifiquen un sistema reactivo. En segundo lugar, la gestión del estado vive íntegramente en el servidor a través de `StateManager`, por lo que no es necesaria una capa equivalente en el cliente. Por último, mantener el código en JavaScript estándar facilita la lectura y la evolución del proyecto, especialmente en un contexto académico en el que la base de código debe poder revisarse sin requerir conocimientos específicos de un *framework* concreto.

La única dependencia de desarrollo es **Vite**, que actúa como servidor de desarrollo durante la iteración, con recarga en caliente, y como empaquetador de producción al generar el *bundle* estático mediante `npm run build`.

Estructura

Sobre esa base mínima de herramientas, el árbol de directorios del cliente se organiza de forma plana y predecible:

- `index.html`: punto de entrada HTML. Declara las regiones visuales (selección de *dataset*, selección de colección y pantalla de navegación) y los contenedores donde el *script* inyecta dinámicamente facetas, entidades y filtros activos.
- `src/main.js`: punto de entrada JavaScript. Orquesta la inicialización, mantiene un objeto `state` con la información mínima necesaria para renderizar (identificadores y nombres de la colección actual, historial de *links* o página activa, entre otros) y registra los *listeners* sobre los controles de la interfaz.
- `src/api.js`: cliente HTTP, descrito más adelante.
- `src/utils.js`: utilidades genéricas, como la selección de nodos por `id` o el escape de HTML.
- `src/components/`: piezas de interfaz reutilizables. Cada archivo encapsula un componente independiente, con una API mínima: combobox con búsqueda incremental para las facetas (`Combobox.js`), *chips* que representan los filtros activos (`FilterTags.js`), ventana modal de detalle de entidad (`Modal.js`), control de paginación (`PaginationHandler.js`), reordenamiento de paneles por *drag and drop* (`PanelDragOrder.js`), *placeholders* de carga (`Skeleton.js`) y notificaciones efímeras (`Toast.js`).
- `src/styles/`: hojas de estilo (`base.css`, `layout.css` y `main.css`).

Comunicación con el backend

A todas estas piezas y a la orquestación de `main.js` les da soporte una capa transversal: el cliente HTTP centralizado en `src/api.js`. Esta pieza expone una única función, `api(endpoint, options)`, construida sobre `fetch`, que fija `mode: 'cors'`, `credentials: 'include'` y la cabecera `Content-Type: application/json`, serializa automáticamente el cuerpo de las peticiones POST y lanza una excepción si la respuesta no es satisfactoria. El uso de `credentials: 'include'` es lo que garantiza que la cookie `JSESSIONID` viaje en cada petición, permitiendo al backend recuperar el `StateManager` asociado a la sesión.

Coherente con la decisión arquitectónica que sitúa el estado en el servidor, el cliente no duplica esa información. Tras cada acción, ya sea aplicar un filtro, navegar por un enlace, retroceder o añadir al conjunto de unión, se invoca una función `refreshAll()` que vuelve a llamar a los *endpoints* de facetas, entidades y unión, y repinta la interfaz con los datos recibidos. Esta estrategia, alineada con el espíritu HATEOAS discutido en el Capítulo 2, simplifica el razonamiento sobre coherencia: el servidor es la única fuente de verdad, y la interfaz refleja en todo momento el resultado que aquel devuelve.

4.7. Ejemplo ilustrativo (recorrido end-to-end)

Las secciones anteriores han presentado por separado las piezas que componen el motor de navegación: el `State` acumulado, las transformaciones que aplica `StateManager`, la traducción a SQL realizada por `NavigationDAO` y el flujo HTTP que conecta el frontend con todo lo anterior. La presente sección integra esas piezas en un ejemplo ilustrativo, ejecutado sobre un dataset deliberadamente reducido. El objetivo es mostrar cómo cada acción primitiva del API (`add_mfilter`, `add_not_mfilter`, `add_rfilter`, `link`, `goback` y `union`) modifica de manera observable el conjunto de entidades visibles, y permitir al lector seguir paso a paso la evolución conjunta del estado interno del sistema y de la consulta SQL que esa acción provoca.

Para que cada operación quede plenamente apreciada, el ejemplo se organiza en tres flujos independientes. Cada flujo arranca desde el estado inicial sobre el dataset y demuestra una categoría distinta de operaciones: el primero acota progresivamente un conjunto dentro de una sola colección, el segundo viaja entre colecciones aplicando filtros en destino que repercuten en el conjunto de origen, y el tercero combina contextos contruidos por separado sobre colecciones diferentes. Dentro de cada flujo, todos los pasos exhiben la acción invocada por el usuario y el conjunto que resulta, mientras que el `State` completo, la consulta SQL y la captura de la interfaz se presentan al final como síntesis del recorrido.

El dataset que sustenta el ejemplo se distribuye junto al código fuente, en el archivo `scripts/test/example_dataset.json`, siguiendo el mismo formato (un objeto JSON con los campos `collections` y `objects`) que el resto de *datasets* de prueba presentes en `scripts/test/`. Las capturas que acompañan a cada flujo se obtuvieron ejecutando el sistema sobre ese dataset y reproduciendo manualmente la secuencia de acciones correspondiente.

4.7.1. Mini-dataset reproducible

El dataset agrupa tres colecciones cuyos tamaños se han elegido deliberadamente asimétricos: cuatro entidades en *Movies*, cuatro en *People* y dos en *Studios*. La asimetría persigue que los filtros y las navegaciones produzcan resultados informativos, ni vacíos ni saturados, sobre conjuntos lo bastante pequeños como para que el lector pueda comprobar mentalmente cada paso. La Tabla 4.2 y la Tabla 4.3 reflejan, respectivamente, el *dataset* y las colecciones que lo componen, tal y como aparecen físicamente en la base de datos.

id	name	created_at
1	tmdb-toy	2025-01-01

Tabla 4.2: Tabla **datasets** del mini-dataset.

global_id	dataset_id	original_id	name
1	1	1	Movies
2	1	2	People
3	1	3	Studios

Tabla 4.3: Tabla **collections**: tres colecciones bajo el *dataset* **tmdb-toy**.

Las entidades concretas se reparten por colección en las Tablas 4.4, 4.5 y 4.6. Por brevedad se omite la columna **contents**, cuyo único uso relevante en este recorrido es el nombre de la entidad, ya reflejado en la columna **name**.

global_id	collection_global_id	original_id	name
10	1	27205	Inception
11	1	157336	Interstellar
12	1	24428	The Avengers
13	1	49026	The Dark Knight Rises

Tabla 4.4: Tabla **entities** restringida a la colección *Movies*.

global_id	collection_global_id	original_id	name
20	2	525	Christopher Nolan
21	2	6193	Leonardo DiCaprio
22	2	3223	Robert Downey Jr.
23	2	3894	Christian Bale

Tabla 4.5: Tabla **entities** restringida a la colección *People*.

global_id	collection_global_id	original_id	name
30	3	174	Warner Bros. Pictures
31	3	420	Marvel Studios

Tabla 4.6: Tabla **entities** restringida a la colección *Studios*.

A continuación, la Tabla 4.7 recoge los metadatos vinculados a cada entidad. Las películas se anotan con género (**Genres**) y año (**Year**), las personas con su lugar de nacimiento (**Birthplace**) y los estudios con su año de fundación (**Founded**). Son precisamente estos pares clave/valor los que darán soporte a los filtros **add_mfilter** y **add_not_mfilter** en las escenas posteriores.

id	entity_id	key	value
1	10	Genres	Action
2	10	Genres	Sci-Fi
3	10	Year	2010
4	11	Genres	Sci-Fi
5	11	Genres	Drama
6	11	Year	2014
7	12	Genres	Action
8	12	Year	2012
9	13	Genres	Action
10	13	Genres	Drama
11	13	Year	2012
12	20	Birthplace	London
13	21	Birthplace	Los Angeles
14	22	Birthplace	New York
15	23	Birthplace	Haverfordwest
16	30	Founded	1923
17	31	Founded	1993

Tabla 4.7: Tabla **metadata**: pares clave/valor por entidad.

Cierra el dataset la Tabla 4.8, que enumera las aristas almacenadas en la tabla **reference** mediante las columnas *source_id*, *target_id* y *reason*. Estas filas son las que sostienen tanto a **add_rfilter** como a **link**. Por las decisiones de diseño descritas en la Sección 4.1, cada relación se persiste de forma explícita en sus dos sentidos. La arista *Director* entre *Inception* y *Christopher Nolan*, por ejemplo, aparece tanto como (10 $\rightarrow$ 20) como (20 $\rightarrow$ 10), lo que permite al motor recorrerla con la misma eficiencia en cualquier dirección de navegación.

id	source_id	target_id	reason	sentido
1	10	20	Director	Inception → Nolan
2	10	21	Actor	Inception → DiCaprio
3	10	30	Production	Inception → Warner
4	11	20	Director	Interstellar → Nolan
5	11	30	Production	Interstellar → Warner
6	12	22	Actor	Avengers → Downey
7	12	31	Production	Avengers → Marvel
8	13	20	Director	TDKR → Nolan
9	13	23	Actor	TDKR → Bale
10	13	30	Production	TDKR → Warner
11	20	10	Director	Nolan ← Inception
12	20	11	Director	Nolan ← Interstellar
13	20	13	Director	Nolan ← TDKR
14	21	10	Actor	DiCaprio ← Inception
15	22	12	Actor	Downey ← Avengers
16	23	13	Actor	Bale ← TDKR
17	30	10	Production	Warner ← Inception
18	30	11	Production	Warner ← Interstellar
19	30	13	Production	Warner ← TDKR
20	31	12	Production	Marvel ← Avengers

Tabla 4.8: Tabla **reference**: aristas dirigidas entre entidades. La columna *sentido* es meramente explicativa.

4.7.2. Vista esquemática del dataset

Las tablas anteriores son indispensables para reproducir el dataset, pero su forma tabular oculta la estructura relacional que subyace al ejemplo. Para complementarlas, la Figura 4.8 ofrece una vista alternativa en la que cada entidad recibe una ficha propia con sus metadatos y sus referencias salientes. Conviene tenerla a mano durante el recorrido posterior, pues constituye un mapa visual de las conexiones que las escenas van explotando una tras otra.

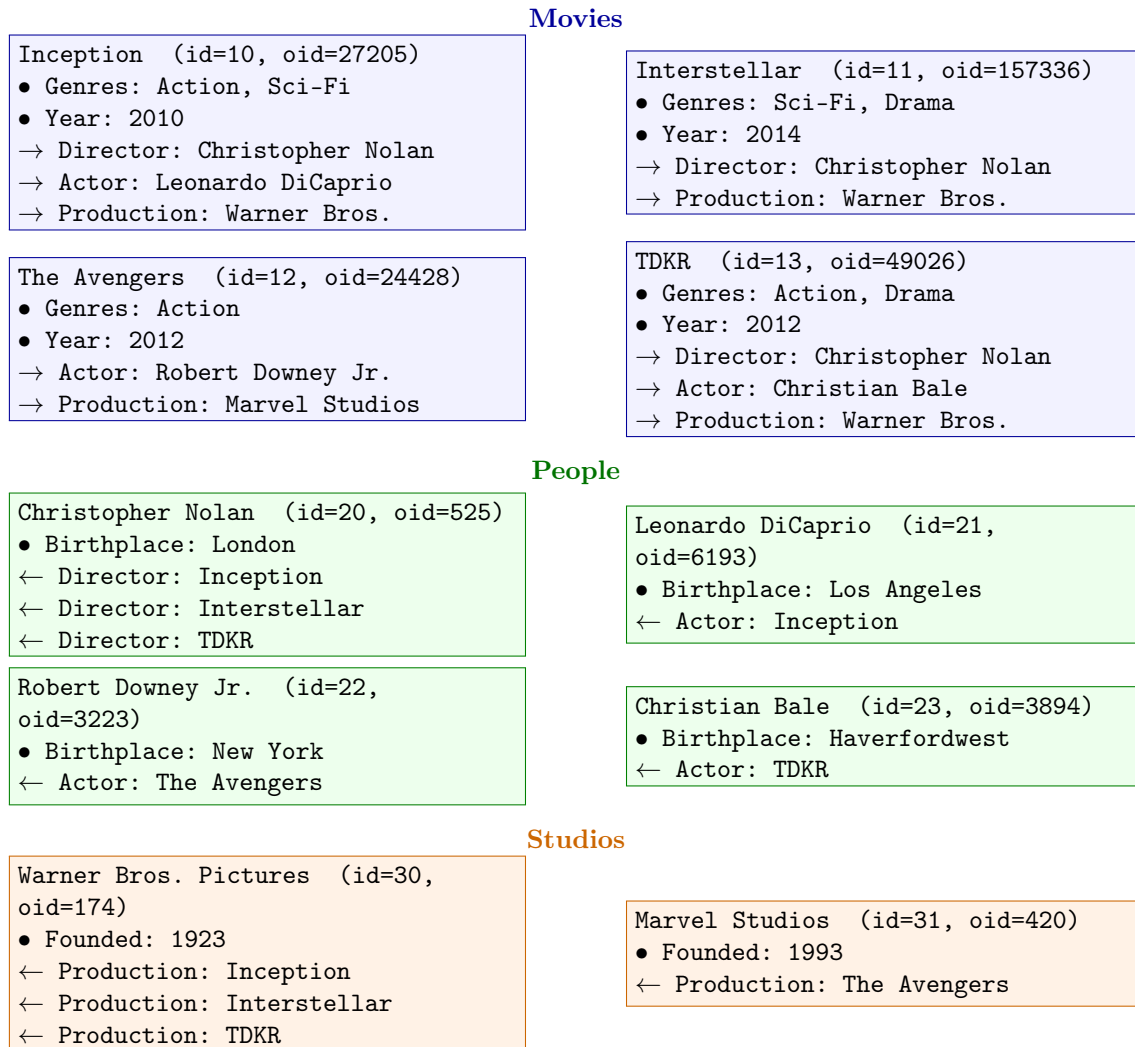


Figura 4.8: Fichas por entidad del mini-dataset, agrupadas por colección y coloreadas en consecuencia: *Movies* en azul, *People* en verde y *Studios* en naranja. Los puntos resumen los metadatos; las flechas → marcan referencias salientes y ← referencias entrantes.

4.7.3. Estado inicial

Cada uno de los tres flujos que siguen arranca desde el estado inicial del sistema: el frontend acaba de abrir la colección *Movies* sobre el dataset descrito, no hay ningún filtro activo y la cuadrícula central enumera las cuatro películas (Figura 4.9). Internamente, el `StateManager` mantiene un único `State` sin `links` ni `unionSet`, y todos los mapas de filtros están vacíos. A partir de ese punto, cada flujo aplica una secuencia distinta de acciones para ilustrar una categoría diferente de operaciones.

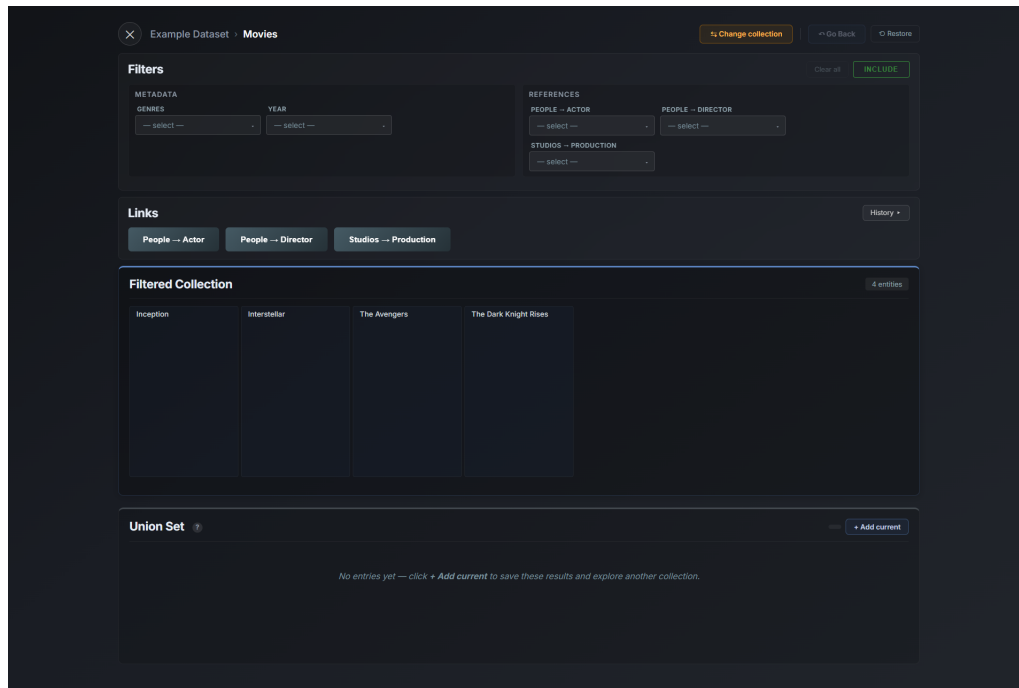


Figura 4.9: Estado inicial del frontend, antes de aplicar ninguna acción: colección *Movies* abierta y las cuatro entidades visibles.

4.7.4. Flujo 1: acotación dentro de una colección

El primer flujo demuestra los tres operadores de filtrado disponibles sobre una misma colección, encadenados de modo que cada uno reduzca de manera observable el conjunto de entidades visibles. Es la forma de exploración más cercana al funcionamiento de un buscador clásico, y cubre las acciones `add_mfilter`, `add_not_mfilter` y `add_rfilter`.

Paso 1.1. Sobre las cuatro películas iniciales, el usuario invoca `addMetadataFilter` con la clave "Genres" y el valor "Action". El conjunto se reduce a tres entidades: $\{Inception, The Avengers, The Dark Knight Rises\}$. *Interstellar* queda fuera, ya que sus únicos géneros son *Sci-Fi* y *Drama* (Figura 4.10).

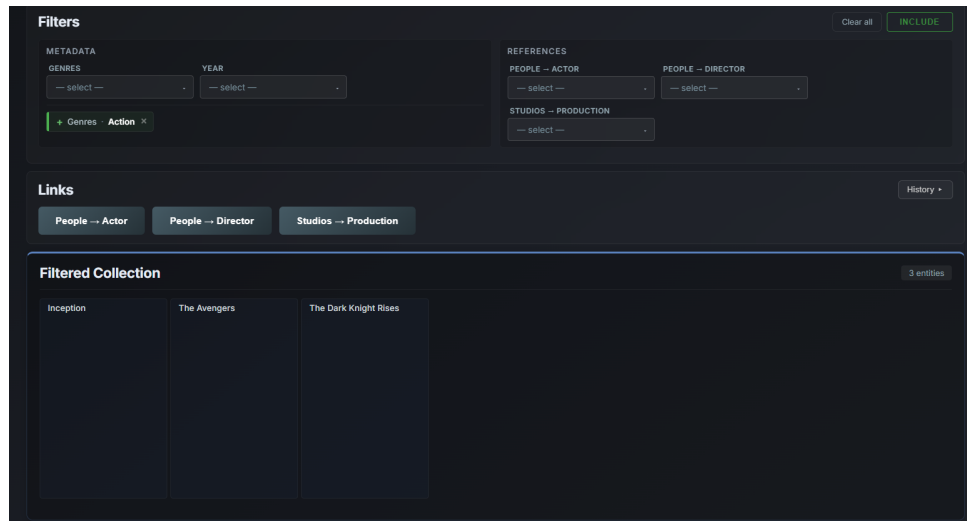


Figura 4.10: Flujo 1, Paso 1.1. Tras aplicar *Genres = Action*, el panel de facetas muestra un único chip activo y la cuadrícula enumera tres películas.

Paso 1.2. Para descartar también las películas dramáticas, el usuario añade un filtro de exclusión: `addNotMetadataFilter` con clave "Genres" y valor "Drama". *The Dark Knight Rises*, que combina *Action* y *Drama*, desaparece, dejando dos entidades visibles: *{Inception, The Avengers}* (Figura 4.11).

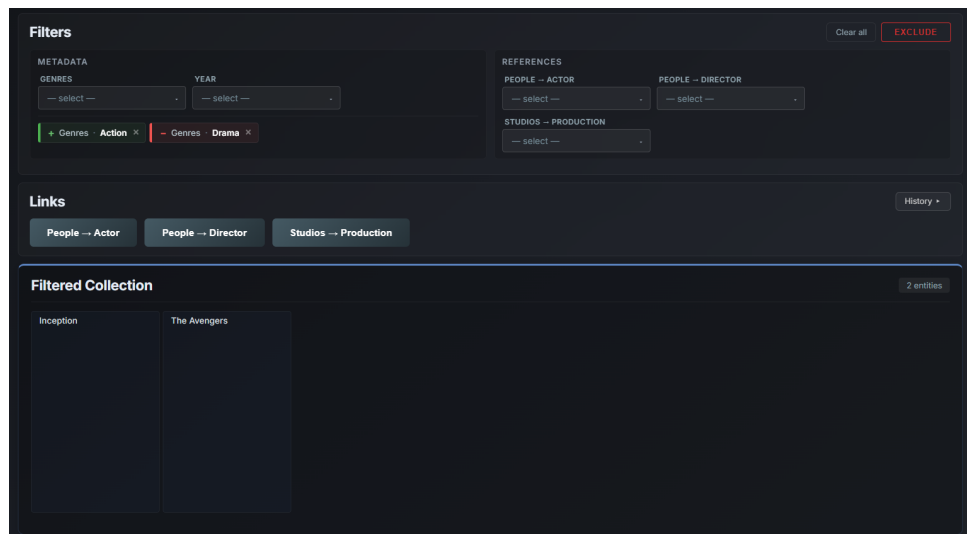


Figura 4.11: Flujo 1, Paso 1.2. El panel de facetas combina un chip positivo (*Genres = Action*) y otro negativo (*Genres ≠ Drama*); en la cuadrícula quedan dos películas.

Paso 1.3. Finalmente, el usuario filtra por estudio productor con `addReferenceFilter`, pasando como colección referenciada el 3 (*Studios*), como motivo "Production" y como identificador externo el 420 (*Marvel Studios*). *Inception*, producida por *Warner Bros.*, queda descartada. El resultado final es *{The Avengers}*, una única entidad de las cuatro iniciales (Figura 4.12).

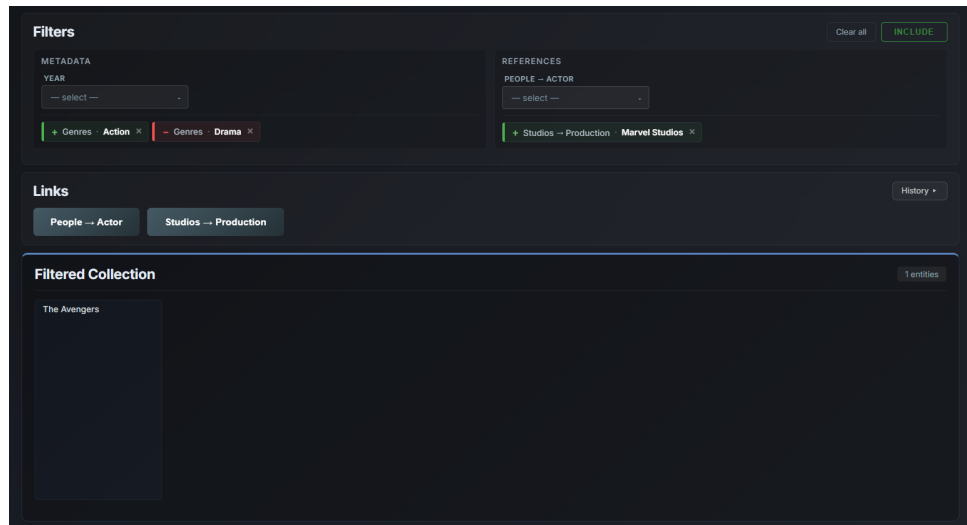


Figura 4.12: Flujo 1, Paso 1.3. Tres chips activos en el panel de facetas y *The Avengers* como único resultado en la cuadrícula central.

State. Tras los tres pasos, el `cur` acumula los tres filtros sobre *Movies* y permanece desligado de cualquier historial de navegación (Figura 4.13).

```
cur.currentCollectionId = Movies (1)
mfilters = { 1 → { Genres → {Action} } }
notMfilters = { 1 → { Genres → {Drama} } }
rfilters = { 1 → { 3 → { Production → {420} } } } notRfilters = { }
linkList = [] unionSet = []
```

Figura 4.13: Estado final del Flujo 1, con un filtro inclusivo, uno de exclusión y uno de referencia activos.

SQL. Como la lista de `links` sigue vacía, la sobrecarga simple de `appendFilterClauses` se limita a invocar a `appendStateFilters` en el nivel exterior. La consulta resultante combina un `EXISTS`, un `NOT EXISTS` y un segundo `EXISTS` que cruza la tabla `reference`, tal como recoge el Código 4.10.

Código 4.10: SQL acumulada al final del Flujo 1.

```
1 SELECT DISTINCT e.original_id, e.name
2 FROM entities e
3 JOIN collections c ON e.collection_global_id = c.global_id
4 WHERE c.dataset_id = 1 AND c.original_id = 1
5     AND EXISTS      (SELECT 1 FROM metadata m0
6                       WHERE m0.entity_id = e.global_id
7                             AND m0.key = 'Genres' AND m0.value = '
8                             Action')
9     AND NOT EXISTS  (SELECT 1 FROM metadata m0
10                      WHERE m0.entity_id = e.global_id
11                            AND m0.key = 'Genres' AND m0.value = '
12                            Drama')
13     AND EXISTS      (SELECT 1 FROM reference r0
14                       JOIN entities re0 ON r0.target_id = re0.
15                           global_id
16                       JOIN collections rc0 ON re0.
17                           collection_global_id = rc0.global_id
18                       WHERE r0.source_id = e.global_id
19                             AND rc0.original_id = 3 AND rc0.dataset_id
20                             = 1
21                             AND r0.reason = 'Production' AND re0.
22                             original_id = 420)
23 ORDER BY e.name LIMIT 40 OFFSET 0
```

4.7.5. Flujo 2: navegación entre colecciones

El segundo flujo parte de nuevo del estado inicial y muestra cómo se encadenan `link` y `goback` alrededor de un filtro intermedio para responder preguntas que ningún filtro local podría plantear. Cada uno de los tres pasos admite una lectura semántica clara: al saltar a *People* estamos preguntando qué personas son actrices en alguna película visible, al filtrar dentro de *People* restringimos esa pregunta a las que además cumplen una propiedad concreta, y al volver a *Movies* obtenemos, en sentido inverso, las películas cuyo reparto incluye precisamente a esas personas.

Paso 2.1. Sobre las cuatro películas iniciales, y sin ningún filtro activo sobre *Movies*, el usuario sigue la relación *Actor* invocando `link` con destino 2 (*People*) y motivo "Actor". Al partir de una colección sin filtros, la pregunta que el sistema responde es la más amplia posible en este sentido: ¿qué personas figuran como actrices de alguna película del catálogo? El motor recorre las aristas *Actor* desde cada película hacia su reparto, y la vista pasa de las cuatro películas iniciales a las tres personas que aparecen como actores: {*Leonardo DiCaprio*, *Robert Downey Jr.*, *Christian Bale*}. *Christopher Nolan*, que sólo figura en el dataset como director, queda lógicamente fuera (Figura 4.14). Internamente, el `StateManager` apila un `Link` con `forward = true`, conserva una copia del `State` previo sobre *Movies* y desplaza `cur.currentCollectionId` a *People*.

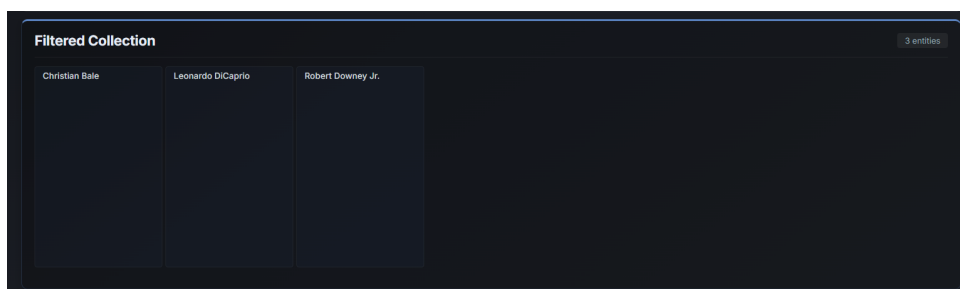


Figura 4.14: Flujo 2, Paso 2.1. El *breadcrumb* indica *Movies* → *People* (*Actor*) y la cuadrícula contiene los tres actores presentes en alguna película.

Paso 2.2. Ya en *People*, el usuario añade un filtro sobre el nuevo `cur` mediante `addMetadataFilter` con clave "Birthplace" y valor "Los Angeles". La pregunta del paso anterior queda refinada: ¿qué personas son actrices de alguna película y, además, nacieron en Los Angeles? El conjunto se restringe a {*Leonardo DiCaprio*}, único actor del dataset con esa ciudad como lugar de nacimiento (Figura 4.15). Conviene observar que el filtro se almacena en el `State` de *People*, no en el de *Movies*, pero queda registrado dentro del `linkList` y volverá a tener efecto en el paso siguiente.

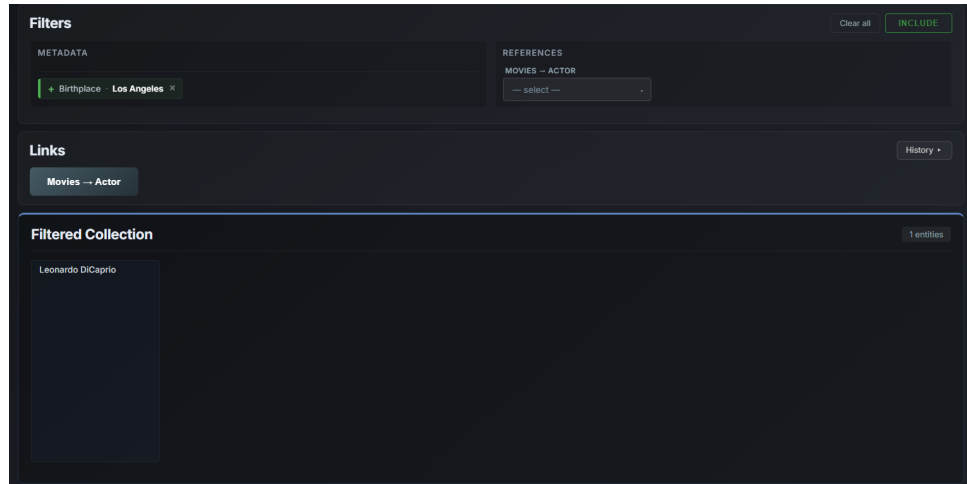


Figura 4.15: Flujo 2, Paso 2.2. El filtro *Birthplace = Los Angeles* aparece como chip activo en el panel y deja una sola persona en la cuadrícula.

Paso 2.3. Para regresar a *Movies*, el usuario invoca `goback()`. Como se detalló en la Sección 4.4, esta operación no elimina el *Link* anterior, sino que apila un segundo *Link* con `forward = false` sobre el primero. El `cur` vuelve a *Movies* y la pregunta se invierte respecto al paso 2.1: ahora interrogamos qué películas tienen en su reparto a alguna de esas personas filtradas, es decir, a algún actor nacido en Los Angeles. La generación de SQL recorre la cadena *Movies* $\rightarrow$ *People* $\rightarrow$ *Movies* arrastrando el filtro *Birthplace* guardado en el *Link* intermedio, y devuelve las películas cuyo reparto incluye a alguna de esas personas. El resultado es una única película, $\{Inception\}$, en la que actúa *DiCaprio* (Figura 4.16). Sin el filtro intermedio sobre *People*, el viaje de ida y vuelta habría devuelto las cuatro películas originales, esto es, todas las que cuentan con algún actor en el dataset.

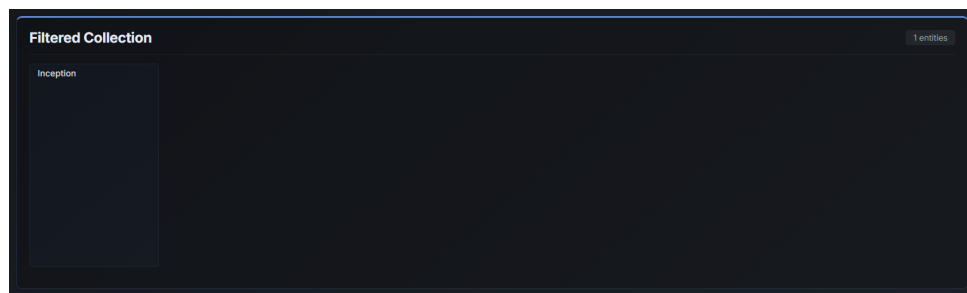


Figura 4.16: Flujo 2, Paso 2.3. El *breadcrumb* extendido (*Movies* $\rightarrow$ *People* (*Actor*) $\rightarrow$ *Movies* (*Actor*)) y *Inception* como único resultado reflejan la cadena completa de transiciones.

State. El `linkList` contiene ahora dos entradas, una hacia adelante y otra de retroceso (Figura 4.17). El filtro de *Birthplace*, aunque ya no aparezca activo en el `cur` actual (que está sobre *Movies*), vive dentro de la copia profunda del **State** sobre *People* guardada en L_1 , y por eso interviene en la SQL del retroceso.

SQL. La sobrecarga recursiva de `appendFilterClauses` recorre el `linkList` de fuera hacia adentro y emite dos niveles de subconsulta anidada. La rama `backward` de L_1 usa `r10.source_id` y, en su interior, materializa el filtro de *Birthplace* aplicado


```
cur.currentCollectionId = Movies (1)
mfilters = {} notMfilters = {} rfilters = {} notRfilters = {}
linkList = [ L0, L1 ]
L0 = Link(forward=true, from=Movies, reason="Actor", state=(Movies sin
filtros))
L1 = Link(forward=false, from=People, reason="Actor", state=(People +
Birthplace=LA))
unionSet = []
```

Figura 4.17: Estado final del Flujo 2. El filtro aplicado en *People* reside dentro del *state* de **L₁** y participa en la SQL recursiva del retroceso.

durante el Paso 2.2. La rama interna corresponde al **forward** original de **L₀**, sin filtros sobre *Movies* (Código 4.11).

El recorrido completo de las dos transiciones, junto con el resultado final de una sola película, queda reflejado en la Figura 4.16. La diferencia con el inicio del flujo es notable: se ha pasado de cuatro entidades a una mediante un filtro aplicado en otra colección.

Código 4.11: SQL acumulada al final del Flujo 2: el filtro *Birthplace = Los Angeles* aplicado en *People* restringe el conjunto visible de *Movies*.

```
1 SELECT DISTINCT e.original_id, e.name
2 FROM entities e
3 JOIN collections c ON e.collection_global_id = c.global_id
4 WHERE c.dataset_id = 1 AND c.original_id = 1
5     AND e.global_id IN (
6         SELECT rl0.source_id FROM reference rl0
7         JOIN entities e1 ON rl0.target_id = e1.global_id
8         JOIN collections c1 ON e1.collection_global_id = c1.
9             global_id
10        WHERE c1.dataset_id = 1 AND c1.original_id = 2
11        AND rl0.reason = 'Actor'
12        AND EXISTS (SELECT 1 FROM metadata m1
13            WHERE m1.entity_id = e1.global_id
14            AND m1.key = 'Birthplace' AND m1.value = '
15                Los Angeles')
16        AND e1.global_id IN (
17            SELECT rl1.target_id FROM reference rl1
18            JOIN entities e2 ON rl1.source_id = e2.global_id
19            JOIN collections c2 ON e2.collection_global_id = c2.
20                global_id
21            WHERE c2.dataset_id = 1 AND c2.original_id = 1
22            AND rl1.reason = 'Actor'
23        )
24    )
25 ORDER BY e.name LIMIT 40 OFFSET 0
```

4.7.6. Flujo 3: unión de contextos independientes

El tercer flujo parte una vez más del estado inicial y explora la última operación del API. La acción **union** permite guardar un contexto construido sobre una colección, comenzar otro independiente sobre otra colección distinta y combinar ambos en una sola respuesta.

Paso 3.1. Sobre las cuatro películas iniciales, el usuario aplica **addMetadataFilter** con clave "Genres" y valor "Sci-Fi". El conjunto visible se reduce a $\{Inception, Interstellar\}$ (Figura 4.18).

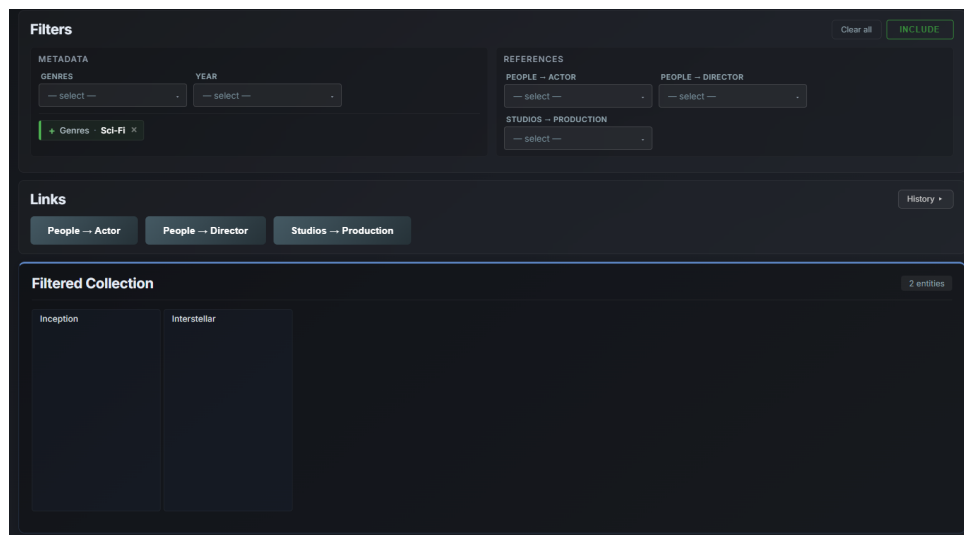


Figura 4.18: Flujo 3, Paso 3.1. *Genres = Sci-Fi* como chip activo y dos películas en la cuadrícula.

Paso 3.2. A continuación, el usuario invoca **union(2)** para guardar el contexto recién construido y abrir uno nuevo sobre *People*. La operación traslada el **cur** actual al **unionSet** y lo sustituye por un **State** en blanco sobre *People*. La cuadrícula pasa a mostrar las cuatro personas del dataset, ya que el nuevo contexto no tiene aún filtros activos, mientras que el panel de unión refleja por primera vez el contexto guardado (Figura 4.19).

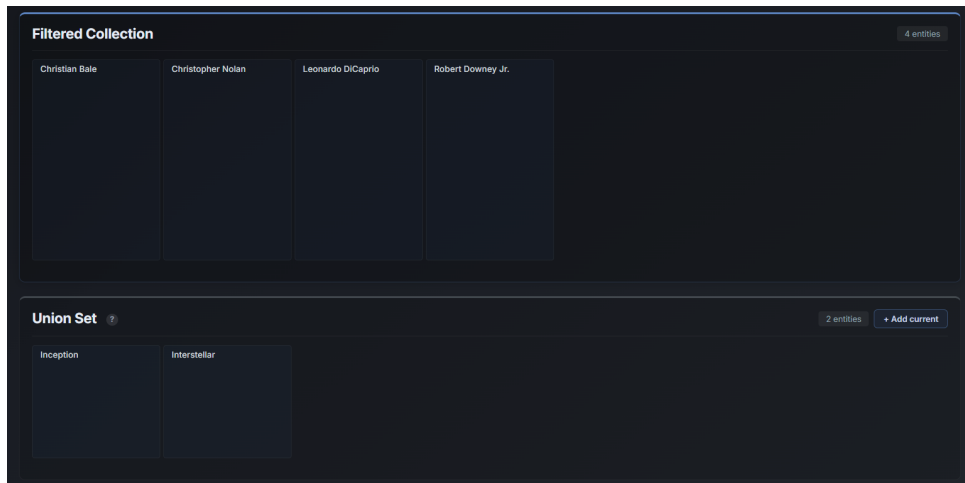


Figura 4.19: Flujo 3, Paso 3.2. La interfaz expone simultáneamente el panel de unión con el contexto recién guardado (*Movies + Sci-Fi*) y la cuadrícula sobre el nuevo cur vacío en *People*.

Paso 3.3. Sobre *People*, el usuario aplica `addMetadataFilter` con clave "Birthplace" y valor "Haverfordwest". El conjunto se restringe a $\{Christian\ Bale\}$ (Figura 4.20).

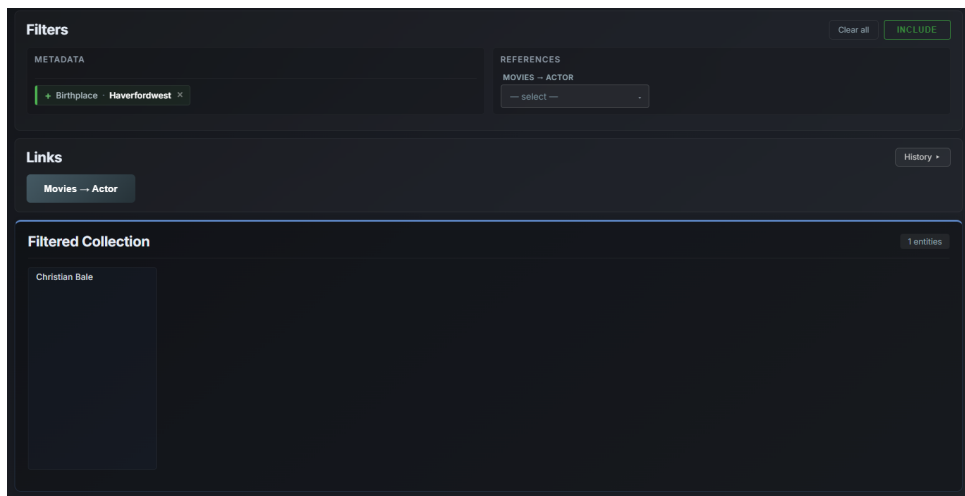


Figura 4.20: Flujo 3, Paso 3.3. El chip *Birthplace = Haverfordwest* reduce a una sola persona la cuadrícula de *People*, mientras que el panel de unión sigue conteniendo el contexto guardado en el paso anterior.

Paso 3.4. Por último, el usuario invoca de nuevo `union(1)` para añadir también este segundo contexto al `unionSet` y comenzar uno nuevo sobre *Movies*. Tras este paso, el `unionSet` contiene dos entradas heterogéneas: un `State` sobre *Movies* con el filtro *Sci-Fi*, y otro sobre *People* con el filtro *Birthplace = Haverfordwest* (Figura 4.21).

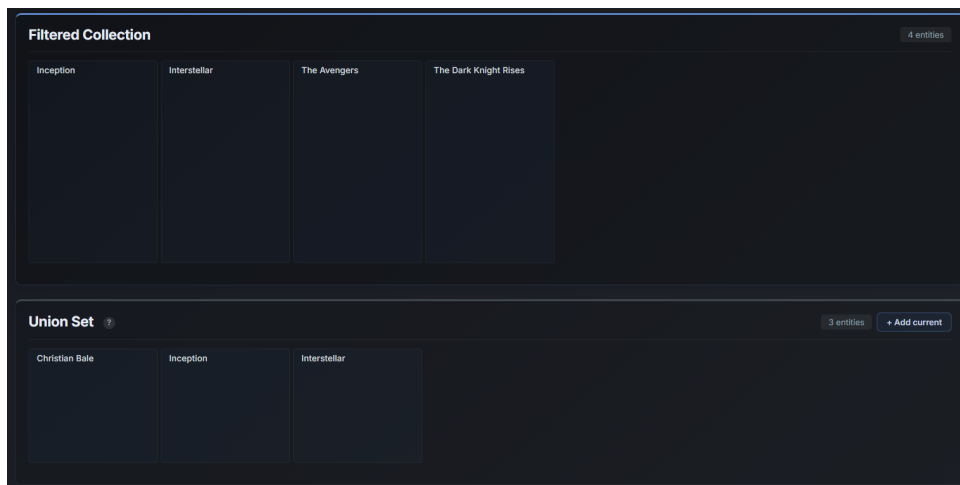


Figura 4.21: Flujo 3, Paso 3.4. La vista de unión combina los tres resultados procedentes de las dos colecciones (dos películas en azul y una persona en verde), siguiendo el código cromático introducido en la Figura 4.8.

State. La Figura 4.22 muestra el contenido del `unionSet` al final del flujo. El nuevo `cur`, limpio sobre *Movies*, no influye en la respuesta del endpoint `/api/union`, que opera exclusivamente sobre las entradas del `unionSet`.

```
unionSet = [
  < Movies (1), mfilters = { Genres → {Sci-Fi} } >,
  < People (2), mfilters = { Birthplace → {Haverfordwest} } >
]
cur.currentCollectionId = Movies (1)
mfilters = {} notMfilters = {} rfilters = {} notRfilters = {}
linkList = []
```

Figura 4.22: Estado final del Flujo 3. El `unionSet` retiene los dos contextos sobre colecciones distintas; el `cur` actual está vacío.

SQL. Al solicitar `GET /api/union`, el método `getUnionEntities(unionSet, page, size)` construye un único `SELECT` con un bloque `WHERE` por entrada del `unionSet`, separados por `OR`. Cada bloque vuelve a invocar `appendFilterClauses` con su propio `State`, de modo que las dos colecciones del `unionSet` dan lugar a dos ramas independientes en la condición (Código 4.12). La proyección añade `c.original_id AS collection_id` para que el frontend pueda distinguir el origen de cada entidad en la respuesta combinada.

Resultado. El bloque 1 aporta `{Inception, Interstellar}`. El bloque 2 aporta `{Christian Bale}`. La respuesta combina las tres entidades en una única lista, con su `collection_id` explícito en cada fila para que la interfaz pueda renderizarlas según su tipo.

Código 4.12: SQL emitida por `getUnionEntities` con dos contextos de colecciones distintas en el `unionSet`.

```
1 SELECT DISTINCT e.original_id, e.name, c.original_id AS
   collection_id
2 FROM entities e
3 JOIN collections c ON e.collection_global_id = c.global_id
4 WHERE
5     ( c.dataset_id = 1 AND c.original_id = 1
6       AND EXISTS (SELECT 1 FROM metadata m0
7                   WHERE m0.entity_id = e.global_id
8                       AND m0.key = 'Genres' AND m0.value = 'Sci-Fi
9                       ')
10    OR
11     ( c.dataset_id = 1 AND c.original_id = 2
12       AND EXISTS (SELECT 1 FROM metadata m0
13                   WHERE m0.entity_id = e.global_id
14                       AND m0.key = 'Birthplace' AND m0.value = '
15                       Haverfordwest')
16 ORDER BY e.name LIMIT 40 OFFSET 0
```

4.8. Obstáculos encontrados y soluciones implementadas

A lo largo del desarrollo surgieron obstáculos de naturaleza diversa: algunos conceptuales, relacionados con la definición del modelo de datos y la lógica de navegación; otros técnicos, derivados de las particularidades del entorno de ejecución. Esta sección recoge los más relevantes, describiendo el problema encontrado y la solución adoptada en cada caso.

Confusión conceptual en el prototipo inicial

El primer obstáculo fue de carácter conceptual y afectó al punto de partida del desarrollo. El modelo navegacional que articula el sistema — colecciones, referencias, filtros acumulados, transiciones — no tiene un equivalente directo en los sistemas de exploración de datos más habituales, lo que dificultó establecer desde el principio una representación interna clara.

La historia de ese aprendizaje y las lecciones del prototipo previo se relatan ya en la sección 3.1.1 del capítulo de arquitectura, donde además se formaliza el modelo definitivo. A los efectos de este capítulo basta con dejar constancia del coste técnico que tuvo aquella confusión: una deuda temprana que fue necesario resolver antes de poder implementar mecanismos más complejos como la navegación transversal o las operaciones de unión, y que motivó la separación estricta de responsabilidades entre modelo (`State`, `StateManager`), capa de acceso a datos (`NavigationDAO`) y

controladores que se describe a lo largo de este capítulo.

Tamaño de las colecciones y trabajo con subconjuntos

TMDB expone colecciones de tamaño considerable: solo en películas y series, el catálogo completo supera con holgura las decenas de miles de entidades. Trabajar con el dataset completo durante el desarrollo habría introducido tiempos de carga, poblado de base de datos y ejecución de queries que habrían dificultado la iteración rápida.

La solución adoptada fue incorporar un mecanismo de filtrado por popularidad directamente en el script de procesamiento, controlado por las variables `FILTER_TOP_K_POPULAR` y `TOP_K_COUNT`, configuradas como se muestra en el Código 4.13.

Código 4.13: Configuración del filtro por popularidad

```
1 FILTER_TOP_K_POPULAR = True
2 TOP_K_COUNT = 5000
```

Con este mecanismo activo, el script ordena las películas y series por su campo `popularity` de la API de TMDB y retiene únicamente las `TOP_K_COUNT` más populares antes de generar el dataset de salida. Esto permitió trabajar durante el desarrollo con un subconjunto representativo y manejable, manteniendo la opción de escalar al dataset completo simplemente desactivando el filtro o aumentando el límite. La motivación de operar con un subconjunto y los detalles del script de procesado se desarrollan en el Apéndice B.

Valores numéricos y de fecha en los metadatos: la decisión de usar rangos

Un obstáculo temprano en el diseño del filtrado fue la naturaleza de ciertos campos de metadatos en TMDB: presupuesto, recaudación, duración, fecha de estreno. Almacenarlos como valores exactos en la tabla `metadata` y filtrar por igualdad exacta habría resultado inútil desde el punto de vista exploratorio: es improbable que un usuario quiera películas con exactamente 157 minutos de duración o con un presupuesto de exactamente 80.000.000 de dólares.

La solución fue discretizar estos valores en rangos durante el procesamiento, antes de insertarlos en la base de datos. Para valores numéricos, la función `bucket_range` agrupa cada valor en un intervalo de tamaño configurable, según la implementación recogida en el Código 4.14.

Código 4.14: Función `bucket_range` para discretizar valores numéricos

```
1 def bucket_range(value, bucket_size):
2     bucket_start = int(num // bucket_size) * bucket_size
3     return f"{format_number(bucket_start)}-{format_number(
        bucket_start + bucket_size - 1)}"
```

Para fechas, se aplica una lógica análoga con granularidad configurable por año, mes y día, definida en `format.json` por colección. De este modo, el valor almacenado en `metadata` no es 157 sino 150-179, lo que permite al sistema de filtrado operar con igualdad de cadenas sobre rangos semánticamente útiles, sin necesidad de soportar operadores de comparación numérica en las queries. La alternativa de soportar expresiones regulares en los filtros de metadatos fue considerada y descartada: la complejidad añadida en la construcción de queries y la dificultad de exponerlo de forma intuitiva al usuario no compensaban el beneficio, siendo los rangos una solución más simple y suficientemente expresiva para los casos de uso identificados. Para los casos en que un usuario necesita combinar varios rangos, la operación de unión cubre esa necesidad de forma más predecible.

CORS y la comunicación entre frontend y backend

El sistema está organizado como dos procesos independientes: el frontend se sirve estáticamente y realiza peticiones al backend desplegado en Tomcat. Esta separación implica que las peticiones HTTP se originan desde un origen distinto al del servidor de la API, lo que activa la política de mismo origen (*same-origin policy*) del navegador y bloquea las respuestas por defecto.

El obstáculo no fue solo técnico sino también de comprensión: el mecanismo CORS (*Cross-Origin Resource Sharing*) no es transparente, y sus manifestaciones —peticiones bloqueadas sin mensaje de error claro, o peticiones `OPTIONS` inesperadas— no resultan inmediatamente evidentes si no se ha trabajado previamente con arquitecturas de frontend y backend separados. Fue necesario investigar el protocolo para entender que el navegador envía primero una petición de preflight con método `OPTIONS` antes de la petición real, y que el servidor debe responder con las cabeceras adecuadas para que el navegador autorice la comunicación.

La solución fue implementar un filtro de servlet, `CORSFilter`, que intercepta todas las peticiones mediante la anotación `@WebFilter("//*")` y añade las cabeceras necesarias en cada respuesta, tal y como se reproduce en el Código 4.15.

Código 4.15: Filtro `CORSFilter` para gestionar las cabeceras CORS

```
1 response.setHeader("Access-Control-Allow-Origin", origin);
2 response.setHeader("Access-Control-Allow-Credentials", "true")
3   ;
4 response.setHeader("Access-Control-Allow-Methods",
5   "GET, POST, PUT, DELETE, OPTIONS");
6 if ("OPTIONS".equalsIgnoreCase(request.getMethod())) {
7   response.setStatus(HttpServletResponse.SC_OK);
8   return;
9 }
10 chain.doFilter(req, res);
```

Un detalle relevante fue la combinación de `Access-Control-Allow-Credentials: true` con el reflejo dinámico del origen en lugar de un comodín `*`: los navegadores rechazan el uso de `*` cuando la petición incluye credenciales (en este caso, la cookie de

sesión), por lo que el filtro lee el encabezado `Origin` de la petición y lo devuelve como valor explícito. Complementariamente, fue necesario configurar el `CookieProcessor` en `context.xml` con `sameSiteCookies="None"` para que la cookie de sesión se enviara correctamente en peticiones cross-origin. La configuración exacta del contenedor de servlets en los modos de desarrollo y de producción, junto con el empaquetado del backend tras un mismo origen mediante Nginx en producción, se describe en el Apéndice C.

El flujo completo de una petición HTTP, desde el *preflight* CORS hasta la respuesta JSON, se ilustra en la Figura 4.23. El diagrama distingue las dos fases que el navegador desencadena de manera automática para una petición cross-origin con credenciales: primero un `OPTIONS` que `CORSFilter` corta en cuanto inyecta las cabeceras, y a continuación la petición real, que recorre toda la cadena (filtro → servlet → DAO → JDBC → PostgreSQL) antes de devolver el cuerpo JSON.

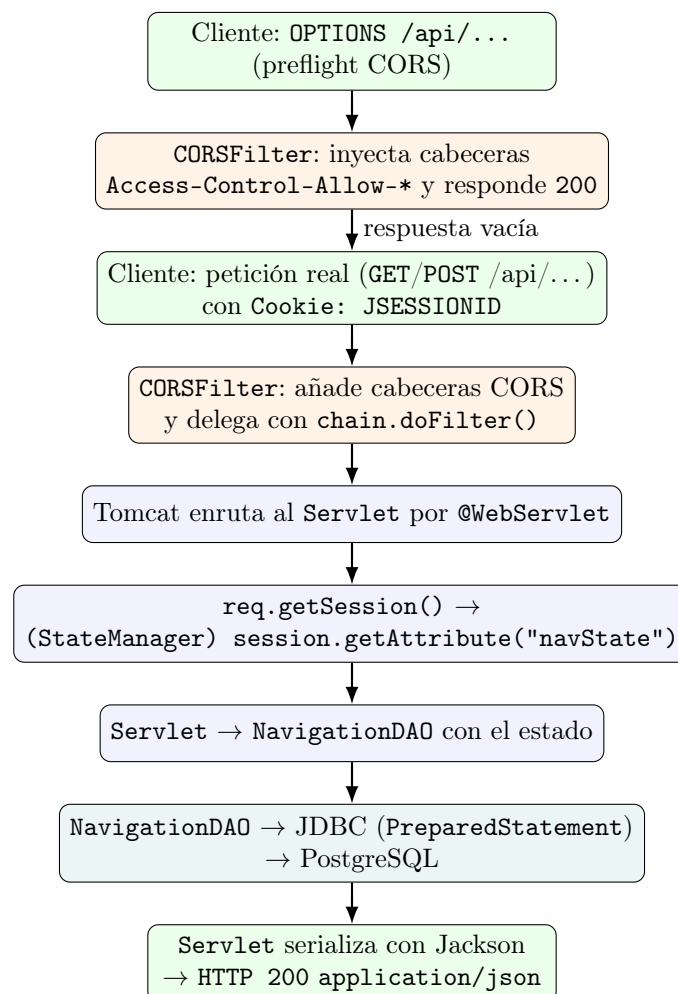


Figura 4.23: Flujo completo de una petición HTTP en el sistema. La fase de *preflight* (parte superior) se resuelve íntegramente en `CORSFilter`, sin alcanzar al servlet. La petición real (parte inferior) atraviesa el filtro, el routing por anotación, la recuperación del `StateManager` en la sesión HTTP, la consulta a través de `NavigationDAO` y JDBC, y termina con una respuesta JSON serializada por Jackson.

El diseño de goback y sus implicaciones

La operación `goback` fue uno de los puntos del diseño que requirió más iteraciones. La dificultad radicó en que revertir una transición entre colecciones no es simplemente volver a la colección anterior: es volver al estado exacto que existía antes de la transición, incluyendo todos los filtros que estaban activos en ese momento.

Un primer enfoque consistió en eliminar el último `Link` del historial al ejecutar `goback`, restaurando la colección activa a la del enlace eliminado. Este enfoque resultó problemático porque el historial perdía información: si el usuario navegaba hacia adelante, retrocedía y volvía a navegar, la cadena de transiciones no reflejaba fielmente el recorrido real, y las subconsultas generadas por `appendFilterClauses` producían resultados incorrectos.

La solución adoptada fue convertir `goback` en una operación aditiva sobre el historial: en lugar de borrar el último `Link`, añade uno nuevo a la lista de transiciones del estado con la dirección invertida (`forward = false`), preservando así el historial completo de forma creciente. Para saber qué transición debe revertirse en cada `goback`, `StateManager` mantiene una pila auxiliar, `linkStack`, que solo registra los enlaces hacia delante creados por `link()`; `goback` extrae el último elemento de esa pila para conocer la colección de origen y la razón del salto que se va a deshacer, dejando el historial real, residente en `cur.getLinks()`, intacto y monótonamente creciente. La implementación final coincide con la mostrada en el Código 4.5.

La consecuencia de este diseño, descrita en la sección 4.4, es que `appendFilterClauses` debe recorrer toda la cadena acumulada, incluidos los retrocesos, para construir correctamente las subconsultas anidadas. El coste es una query potencialmente más larga conforme el usuario navega y retrocede, pero garantiza que el contexto de exploración se reconstruye de forma exacta en cada petición.

Capítulo 5

Pruebas y Validación

*“El primer principio es que no debes engañarte a ti mismo, y
tú eres la persona más fácil de engañar.”*

— Richard Feynman

Este capítulo recoge la validación del sistema desde dos perspectivas complementarias. La primera, de carácter funcional, comprueba que el motor de exploración se comporta correctamente sobre los dos dominios de prueba seleccionados (TMDB y DBLP) y se aborda brevemente en la sección 5.1, ya que las decisiones técnicas correspondientes se han descrito en el capítulo 4. La segunda, que ocupa el grueso del capítulo, es una evaluación formal de usabilidad con participantes externos, presentada en la sección 5.2 y desarrollada en las secciones 5.3 (resultados cuantitativos), 5.4 (hallazgos cualitativos), 5.5 (iteraciones de interfaz derivadas) y 5.6 (casos de uso validados). La sección 5.7 cierra el capítulo reconociendo de manera explícita los límites del estudio.

5.1. Validación funcional sobre TMDB y DBLP

La validación funcional del motor se ha realizado de manera incremental durante el desarrollo, sobre dos volcados de catálogos públicos de naturaleza estructural muy distinta: TMDB (*The Movie Database*), que reúne películas, series y personas con metadatos descriptivos y relaciones de participación, y DBLP, una base bibliográfica que reúne publicaciones científicas, autores e instituciones. La elección deliberada de dos dominios separados ha permitido comprobar empíricamente que las primitivas del motor (filtrado facetado, navegación transversal mediante *links* y operaciones de conjunto) generalizan sin necesidad de modificaciones sustanciales del modelo relacional, y constituye uno de los argumentos centrales del trabajo. Las decisiones de diseño y los obstáculos encontrados durante esta integración se detallan en el capítulo 4 y no se reproducen aquí. El resto del presente capítulo se concentra en la evaluación con usuarios, donde reside el grueso de la evidencia empírica recogida.

5.2. Evaluación de usabilidad con usuarios

5.2.1. Objetivos del estudio

La evaluación se diseñó con cinco objetivos explícitos que guían tanto el diseño del estudio como la lectura posterior de los resultados:

- **Facilidad de aprendizaje** en la primera toma de contacto con el sistema, sin formación ni documentación previa.
- **Eficiencia** al realizar tareas comunes de búsqueda y navegación, medida en tiempo y en cantidad de asistencia requerida.
- **Comprensión de los conceptos clave** sobre los que se asienta el modelo de navegación: filtros de inclusión y exclusión, *links* entre colecciones, conjunto unión, historial visual y la distinción entre *Restore* y *Go Back*.
- **Recuperación ante errores** y manejo correcto de las acciones destructivas (*Exit* y *Change collection*), que reinician parte del estado de la sesión.
- **Satisfacción subjetiva**, recogida mediante el cuestionario estandarizado SUS (*System Usability Scale*) y un bloque de preguntas abiertas de cierre.

5.2.2. Metodología

El estudio sigue un protocolo formal de *think-aloud* acompañado de observación directa y de cuestionarios pre y post sesión. Cada participante realiza, en orden fijo, catorce tareas planteadas como escenarios realistas, sin instrucciones procedimentales: se enuncia el objetivo y el participante decide cómo resolverlo. Durante cada tarea, el facilitador anota tiempo, completitud (*Éxito / Parcial / Fallo*), fluidez de navegación, estrategia seguida, nivel de asistencia requerido y satisfacción aparente. Tras cada tarea se recoge la métrica **SEQ** (*Single Ease Question*, escala de 1 a 7) y un comentario abierto. Al final de la sesión se administra el cuestionario **SUS** (diez ítems en escala Likert de 1 a 5, con la fórmula clásica de cálculo que produce una puntuación entre 0 y 100) y un bloque de ocho preguntas abiertas que recogen percepciones globales y propuestas de mejora. El guion completo del estudio, incluyendo texto literal a leer, hojas de consentimiento, plantillas de observación y notas metodológicas, se reproduce íntegro en el Apéndice A.

Cada sesión tuvo una duración aproximada de entre 32 y 37 minutos, con media en torno a 34 minutos, en línea con la estimación previa de 35-45 minutos. Las sesiones se realizaron de manera presencial entre el 7 y el 14 de mayo de 2026, sobre la misma versión del sistema y la misma colección inicial para asegurar comparabilidad. Cuatro de las cinco sesiones fueron grabadas con permiso explícito del participante, y las grabaciones se eliminaron una vez codificadas las observaciones.

5.2.3. Perfil de los participantes

Se reclutaron cinco participantes con perfiles diversificados, siguiendo la recomendación de Nielsen según la cual cinco evaluadores descubren aproximadamente el 85 % de los problemas de usabilidad observables en un sistema. La diversidad demográfica y disciplinar buscada quedó representada como muestra el cuadro 5.1: tres rangos de edad distintos, cinco ocupaciones diferentes, dos personas con experiencia en catalogación o bases de datos y tres sin ella, y una distribución equilibrada de la comodidad auto-percibida con herramientas digitales nuevas (rango de 2 a 5 sobre 5).

ID	Edad	Ocupación	Catalogación	Comodidad
P01	18–24	Estudiante de Derecho y Filosofía	No	3
P02	18–24	Estudiante de Ingeniería Informática	Sí (en estudios)	5
P03	25–34	Médico	No	2
P04	18–24	Programador	Sí (profesional)	4
P05	55+	Profesional administrativo	No	3

Tabla 5.1: Perfil demográfico y de experiencia previa de los cinco participantes. La columna *Comodidad* recoge la respuesta a la pregunta sobre comodidad para aprender una herramienta nueva por cuenta propia, en escala de 1 a 5.

5.3. Resultados cuantitativos

5.3.1. Tasa de éxito y completitud

De las setenta tareas realizadas en total (catorce tareas × cinco participantes), sesenta y cinco se completaron con éxito sin asistencia y dentro del tiempo límite, lo que arroja una tasa global de éxito del **93 %**. Las cinco situaciones restantes se reparten entre dos parciales (T4 y T12 en P01, ambas resueltas con asistencia ligera) y dos fallos (T2 en P03, derivado de una mala interpretación inicial de la tarea y no de la interfaz; y T4 en P05, donde el participante no consiguió cambiar al modo de exclusión). El gráfico 5.1 desglosa estas categorías por tarea.

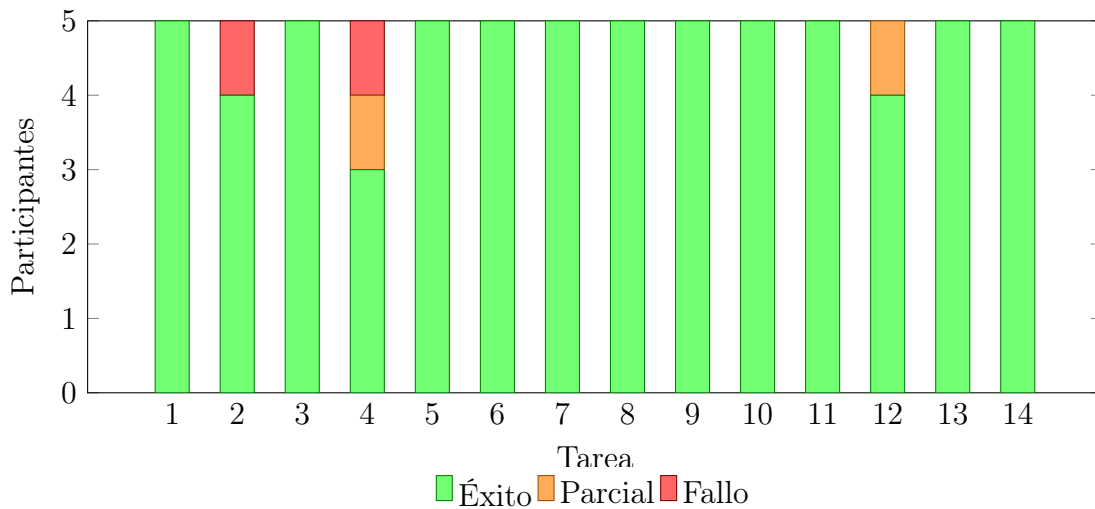


Figura 5.1: Distribución de la completitud por tarea. Las tareas T4 (filtros de exclusión) y T12 (reordenación de paneles) concentran la totalidad de las situaciones que no resultaron en éxito limpio.

La concentración de incidencias en T4 y T12 anticipa los hallazgos cualitativos discutidos más adelante: el paso del modo *Include* a *Exclude* es el punto donde más participantes tropiezan, y el comportamiento del arrastre de paneles cuando hay *scroll* vertical es el segundo foco de fricción.

5.3.2. Tiempo por tarea

El cuadro 5.2 resume los tiempos por tarea en segundos. Empleamos la mediana como medida central (más robusta que la media para muestras pequeñas) y reportamos el mínimo y el máximo para revelar la dispersión.

T	Tarea	Mediana	Mín.	Máx.	N éxito
1	Empezar a navegar	15	5	21	5/5
2	Filtrar por metadatos	16	7	45	4/5
3	Combinar varios filtros	30	11	77	5/5
4	Filtros de exclusión	73	49	148	3/5
5	Filtros de referencia	5	2	55	5/5
6	Quitar filtros	27	11	35	5/5
7	Seguir un <i>link</i>	7	2	110	5/5
8	Ver el historial de navegación	19	5	30	5/5
9	Volver atrás un paso	6	2	25	5/5
10	Ver el detalle de una entidad	20	7	65	5/5
11	Guardar en <i>Union</i> y cambiar	44	30	75	5/5
12	Reordenar paneles	33	6	45	4/5
13	Cambiar de colección (sin <i>Union</i>)	13	5	45	5/5
14	Salir del modo navegación	10	3	10	5/5

Tabla 5.2: Tiempos por tarea (segundos) y número de éxitos limpios sobre cinco intentos. Los tiempos elevados en T4 reflejan tanto la dificultad intrínseca como la asistencia ligera que requirieron varios participantes.

Tres patrones merecen comentario explícito. Primero, T4 (filtros de exclusión) destaca con una mediana de 73 segundos, casi triple que la siguiente tarea más costosa, y un máximo de 148 segundos cercano al límite de tiempo. Segundo, T7 (seguir un *link*) presenta una varianza atípica (mediana de 7 segundos pero máximo de 110): los dos participantes que tardaron más no encontraron el panel de *links* de inmediato porque éste quedaba por debajo del primer pliegue de la pantalla, lo cual conecta con un hallazgo cualitativo recurrente sobre el tamaño de la interfaz. Tercero, las tareas que ejercen el modelo de navegación recuperable (T9 *Go Back*, T13 *Change* y T14 *Exit*) resultan rápidas y consistentes, lo que sugiere que la diferenciación visual ámbar/azul/gris de la barra superior funciona en términos de eficiencia, aunque más adelante veremos que la diferenciación conceptual entre *Go Back* y *Restore* sigue siendo borrosa para algunos participantes.

5.3.3. Facilidad percibida (SEQ)

Para cada tarea, los participantes respondieron a la pregunta *Single Ease Question* en escala de 1 (muy difícil) a 7 (muy fácil). El gráfico 5.2 muestra la media por tarea. Los valores quedan, en general, por encima de 6 (es decir, en el rango positivo), con la única excepción notable de T4, que cae a 3,4 y se sitúa por debajo del umbral de 4 que suele considerarse como frontera entre tareas percibidas como aceptables y problemáticas. T11 (*Union*) y T12 (reordenar paneles) ocupan una zona intermedia en torno a 5, coherente con la fricción detectada en sus tiempos.

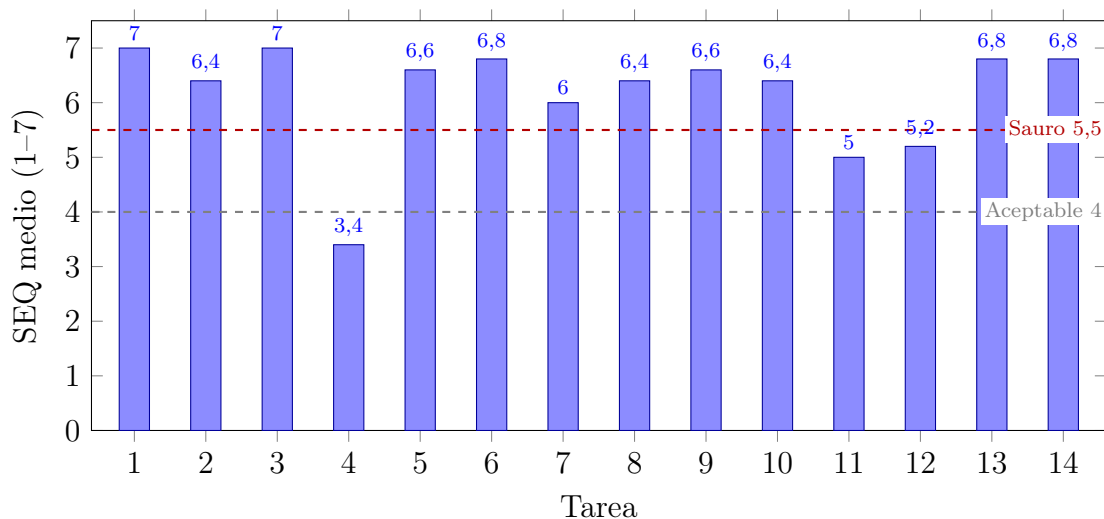


Figura 5.2: Facilidad percibida (SEQ) media por tarea. Las líneas discontinuas marcan el umbral de aceptación clásico ($SEQ \geq 4$) y la referencia de Sauro para tareas «fáciles» ($SEQ \geq 5,5$). Sólo T4 cae por debajo del primer umbral.

5.3.4. Satisfacción global (SUS)

Las puntuaciones SUS individuales y la media del grupo aparecen en el gráfico 5.3. Las cinco puntuaciones quedan por encima del umbral clásico de 68 que separa los sistemas «aceptables» de los que no lo son, y cuatro de las cinco superan

el umbral de 80 que se asocia con sistemas «excelentes». La media del grupo, **85**, se sitúa en el primer cuartil de la distribución de referencia de Sauro y Lewis, lo que constituye uno de los resultados más sólidos del estudio.

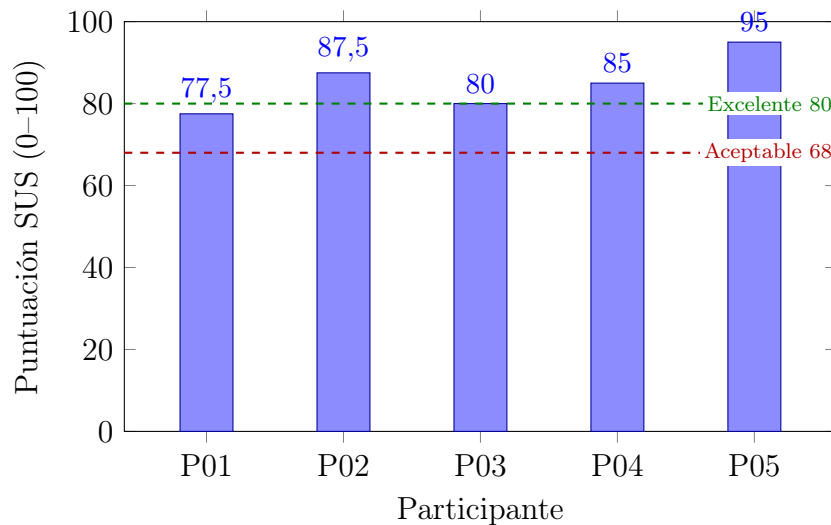


Figura 5.3: Puntuación SUS por participante. La media del grupo es 85, con todos los valores por encima del umbral de aceptación (68) y cuatro de cinco por encima del umbral de excelencia (80).

5.4. Hallazgos cualitativos y problemas detectados

El cruce de las observaciones del facilitador, los comentarios espontáneos en *think-aloud* y las respuestas a las preguntas abiertas de cierre permite agrupar los problemas detectados en cinco familias, ordenadas en el cuadro 5.3 por gravedad estimada según la escala de Nielsen y por frecuencia de aparición.

Problema	Gravedad	Frecuencia	Tareas
Toggle <i>Include/Exclude</i> poco descubrible	3 (mayor)	5/5	T4
Arrastre de paneles sin <i>auto-scroll</i> cuando hay desbordamiento	2 (menor)	3/5	T12
Distinción <i>Restore</i> vs <i>Go Back</i>	2 (menor)	2/5	T9, T13
Visibilidad de los paneles inferiores (<i>links</i> , entidades)	2 (menor)	2/5	T7, T10
Concepto de <i>Union Set</i> sin pistas	1 (cosmético)	2/5	T11

Tabla 5.3: Catálogo de incidencias detectadas, ordenadas por gravedad. La gravedad sigue la escala clásica de Nielsen (0 = sin problema, 1 = cosmético, 2 = menor, 3 = mayor, 4 = catastrófico) y la frecuencia indica sobre cuántos de los cinco participantes apareció.

El **toggle de inclusión y exclusión** es el problema más nítido del estudio. Los cinco participantes intentaron, en algún momento de T4, cambiar el sentido del filtro pulsando directamente sobre la etiqueta verde con «+», esperando que mutase a la versión roja con «-». El control real (un botón segmentado en la cabecera del

panel de Filtros) sólo se descubría tras varios segundos de exploración o, en el caso de P05, no se descubría en absoluto. Las tres respuestas a la pregunta de cierre sobre lo más confuso del sistema mencionan literalmente este punto, y P01 lo expresa con claridad: «piensa que sería más intuitivo poder dar click al "+" de tal manera que se convierta en un "-"». Adicionalmente, la previsualización por *hover* introducida para anticipar la acción del clic generó confusión: al pasar el ratón el texto cambiaba pero, al hacer clic, el ratón seguía sobre el botón y el cambio aparente de estado se confundía con la previsualización en curso. La combinación de un toggle separado del filtro y una previsualización ambigua acumula dos fuentes de fricción sobre la misma acción.

El **arrastre de paneles** (T12) funcionó correctamente cuando el panel de origen y el de destino cabían en la pantalla, pero falló sistemáticamente cuando el destino estaba fuera del *viewport*: tres participantes intentaron arrastrar un panel hacia arriba sin que la página realizase *auto-scroll*, lo que rompió la metáfora de manipulación directa. Dos de ellos consiguieron completar la tarea por una vía alternativa, pero P01 quedó en parcial. Es un fallo de implementación del *drag-and-drop* más que un fallo de diseño, pero el efecto sobre la percepción es desproporcionado.

La **distinción entre *Restore* y *Go Back*** sigue siendo problemática para una minoría significativa. Aunque la diferenciación visual de la barra superior (ámbar / azul / gris) sí se percibe como una pista de uso, el modelo conceptual no se deriva sin pistas: P01 y P05 invierten el papel de los dos botones cuando se les pide explicarlos al final de la sesión. P02 y P03 los explican correctamente, lo que sugiere que el problema afecta sobre todo a participantes con menor familiaridad con interfaces de exploración.

La **visibilidad de los paneles inferiores** se manifestó como una serie de pequeñas fricciones en T7 y T10. Dos participantes (P01, P02) no descubrieron de inmediato la existencia del panel de *links* ni del de entidades porque ambos quedaban por debajo del primer pliegue de la pantalla en su configuración inicial, e intentaron resolver la tarea con los controles visibles. P02 sintetizó el problema durante la sesión: «siente que la interfaz es un poco grande y hay que ir moviéndose por ella».

El **concepto de *Union Set*** es el caso más matizado. Cuatro de cinco participantes lo entendieron en el momento del uso (T11 se completa con éxito en los cinco casos), pero al pedirles que lo explicasen con sus propias palabras al final de la sesión, dos no consiguieron articular su semántica más allá de «se han guardado cosas». Esto sugiere que la interfaz comunica bien la operación inmediata pero no necesariamente el modelo subyacente. P02, con formación en bases de datos, lo asoció directamente con la unión matemática.

Las preguntas abiertas dejaron también un conjunto de **solicitudes de funcionalidad** recurrentes que conviene registrar aunque excedan el alcance del trabajo: búsqueda por texto sobre los nombres de los metadatos y sobre las entidades (P01), previsualizaciones ricas con miniaturas en el panel de entidades (P01, P04), navegación anidada de entidades dentro de entidades (P04) y una sección de ayuda contextual (P03). Algunas de estas se recogen como líneas de trabajo futuro en el capítulo 6.

5.5. Iteraciones de la interfaz derivadas de la evaluación

Cinco direcciones concretas de modificación se desprenden del catálogo anterior. La primera, de mayor prioridad, consiste en **rediseñar la interacción de inclusión y exclusión**. La hipótesis a explorar, sugerida por varios participantes, es que la propia etiqueta del filtro activo se comporte como un toggle: pulsar sobre el «+» verde lo convertiría en «-» rojo y viceversa, de manera que el control segmentado actual quede como un valor por defecto al añadir filtros nuevos pero no como el único punto de cambio. La previsualización por *hover* debería desactivarse o convertirse en un *tooltip* explícito para evitar la confusión observada.

La segunda dirección es **implementar *auto-scroll* durante el arrastre de paneles**: cuando el cursor se acerca al borde superior o inferior del *viewport* mientras se arrastra un panel, la página debe desplazarse en esa dirección. Es un cambio puramente técnico en la lógica de *drag-and-drop* pero recupera la metáfora de manipulación directa que los participantes esperan.

La tercera consiste en **reforzar la diferenciación entre *Restore* y *Go Back***, ya sea mediante etiquetas más explícitas (por ejemplo, «Deshacer último link» y «Restaurar estado guardado»), un *tooltip* la primera vez que el usuario pasa sobre cada botón, o ambos. La diferenciación visual actual ayuda en eficiencia pero no construye el modelo conceptual.

La cuarta es **mejorar la visibilidad de los paneles inferiores**. Caben varias estrategias: compactar la altura inicial del panel de Filtros para que el panel de entidades quede dentro del primer pliegue; introducir un indicador visual de «hay más contenido más abajo»; o reorganizar el orden por defecto de manera que las áreas de acción más frecuente queden primero. La elección entre estas opciones es a su vez candidata a una nueva ronda de evaluación.

La quinta y última es **aclerar el concepto de *Union Set*** mediante una introducción contextual la primera vez que el usuario pulsa «+ Add current» (un *tooltip* explicativo, una pequeña anotación en el panel cuando está vacío, o un recorrido guiado opcional al inicio de la primera sesión). El comportamiento del sistema es correcto; lo que falta es comunicación.

5.6. Casos de uso validados

El cuadro 5.4 resume el estado de validación de cada caso de uso del sistema cruzando las tareas de la evaluación con su tasa de éxito y SEQ medio. Sirve como vista compacta del veredicto del estudio: la mayoría de los casos de uso quedan validados sin reservas; los dos casos asociados a la fricción detectada (filtros de exclusión y operaciones de conjunto / personalización del espacio) se consideran validados con observaciones que deberán abordarse en la siguiente iteración.

Caso de uso	Tareas	Éxito	SEQ	Veredicto
Selección de dataset y colección	T1	5/5	7,0	Validado
Filtrado por metadatos	T2, T3	9/10	6,7	Validado
Filtrado por exclusión	T4	3/5	3,4	Validado con observaciones
Filtros de referencia	T5	5/5	6,6	Validado
Gestión de filtros activos	T6	5/5	6,8	Validado
Navegación transversal por <i>links</i>	T7, T9	10/10	6,3	Validado
Historial visual de la navegación	T8	5/5	6,4	Validado
Inspección de entidad (modal de detalle)	T10	5/5	6,4	Validado
Operaciones de conjunto (<i>Union</i>)	T11	5/5	5,0	Validado con observaciones
Personalización del espacio (reordenar paneles)	T12	4/5	5,2	Validado con observaciones
Cambios destructivos con confirmación	T13, T14	10/10	6,8	Validado

Tabla 5.4: Resumen de casos de uso validados. La columna *Veredicto* distingue entre casos validados sin reservas (todos los participantes los completaron con SEQ medio ≥ 6) y casos validados con observaciones, donde la operativa funciona pero la evaluación detectó fricción que se aborda en la sección 5.5.

5.7. Limitaciones del estudio

El estudio presentado debe interpretarse dentro de varios límites bien delimitados. El más evidente es el **tamaño muestral**: cinco participantes son suficientes para detectar la mayoría de los problemas observables siguiendo la heurística clásica de Nielsen, pero no permiten sostener afirmaciones cuantitativas con valor estadístico inferencial. Las medias de SUS y SEQ que se reportan describen el grupo concreto evaluado y deben leerse como indicadores cualitativos antes que como estimaciones de una población.

El **perfil de los participantes**, aunque diversificado en edad, ocupación y experiencia previa, no es representativo de los usuarios reales del sistema en producción, fundamentalmente porque tales usuarios no existen aún: el sistema se ha evaluado sobre catálogos de prueba y no sobre un despliegue real con población definida. Esto introduce un sesgo inevitable, ya que los participantes interpretan las tareas en abstracto en lugar de hacerlo movidos por una necesidad propia de exploración.

El **dominio de la evaluación** fue el catálogo TMDb, escogido por su familiaridad universal y por reducir el coste cognitivo de comprender los datos. La validación con DBLP, de naturaleza estructural distinta, se quedó en el plano funcional descrito en la sección 5.1 y no se trasladó a la evaluación con usuarios. Es razonable suponer que parte de los hallazgos de usabilidad serían comunes a ambos dominios, pero también que la familiaridad del usuario con películas frente a publicaciones científicas modula las observaciones.

Por último, se trata de un estudio **sin grupo de control**: no se ha comparado el rendimiento de los participantes con el que tendrían sobre una herramienta alternativa (búsqueda directa, filtrado clásico, etcétera), de manera que las afirmaciones sobre la adecuación del sistema deben entenderse en términos absolutos y no comparativos. Una evaluación comparada con una segunda interfaz constituye, junto al despliegue real con usuarios identificados, una de las extensiones naturales del trabajo, recogida también entre las líneas futuras del capítulo 6.

Capítulo 6

Conclusiones y Trabajo Futuro

“Lo que llamamos comienzo es a menudo el final, y llegar al final es comenzar.”
— T. S. Eliot

Este capítulo cierra la memoria recogiendo lo aportado por el trabajo, reconociendo de manera explícita sus límites y enunciando las líneas de continuación orgánica que se abren a partir de él. La sección 6.1 sintetiza los resultados alcanzados respecto a los objetivos planteados en el capítulo 1. La sección 6.2 delimita el alcance del sistema entregado. La sección 6.3 agrupa, en cinco direcciones complementarias, las extensiones que dejamos abiertas como continuación del proyecto.

6.1. Conclusiones

El trabajo ha permitido construir un motor de exploración de colecciones digitales que integra los tres mecanismos planteados desde el inicio: filtrado facetado sobre metadatos, navegación transversal entre colecciones a través de referencias semánticas, y operaciones de conjuntos sobre los resultados parciales. Cada uno de ellos se ha materializado en un componente identificable del sistema (filtros, *links* y conjunto unión) y se ha verificado su comportamiento sobre volcados reales de dos catálogos públicos de naturaleza muy distinta, TMDb (audiovisual) y DBLP (publicaciones científicas), escogidos precisamente por sus diferencias estructurales.

La estrategia metodológica de doble fase ha resultado especialmente acertada. La primera fase, centrada en el prototipo en Java sin interfaz gráfica, permitió validar la viabilidad técnica del modelo de navegación de manera rigurosa, sin contaminar las decisiones de diseño con restricciones derivadas de la presentación. La segunda fase, dedicada a la exposición del motor mediante una *API RESTful* y al desarrollo de la interfaz web, pudo entonces concentrarse en cuestiones de experiencia de usuario sobre una base ya estable. Esta separación clara entre *back-end* y *front-end* también ha facilitado el trabajo en pareja, al definir contratos explícitos entre las dos capas.

La adopción del marco de trabajo **Scrum** adaptado a un equipo reducido ha demostrado su valor al permitir reorientar el proyecto cuando los hallazgos de cada *sprint* obligaban a reconsiderar decisiones previas. En paralelo, la evaluación de

usabilidad realizada con participantes externos siguiendo un protocolo formal ha producido evidencia cualitativa que ha guiado la última iteración de la interfaz, y constituye en sí misma una contribución reusable para futuros desarrollos sobre el mismo motor.

6.2. Limitaciones

El alcance del sistema entregado debe entenderse dentro de varios límites bien delimitados. La validación funcional del motor se ha realizado sobre dos dominios diferenciados (el catálogo audiovisual TMDb y la base bibliográfica DBLP), lo que aporta evidencia razonable de que el modelo no depende de las particularidades de un único corpus, pero no agota el espectro de colecciones a las que el sistema aspira a aplicarse. Su extensión a otros tipos de fondos (repositorios documentales institucionales, catálogos bibliotecarios completos o conjuntos de software libre) sigue siendo, por tanto, una hipótesis razonable que no se ha demostrado empíricamente en este trabajo. Del mismo modo, las pruebas de carga se han limitado a colecciones de unas decenas de miles de entidades, sin abordar escenarios de gran escala que requerirían un trabajo de optimización específico.

El sistema carece, por decisión de diseño y no por descuido, de una capa de usuario persistente: las sesiones son efímeras y no se contempla la identificación, la autenticación ni la sincronización de estado entre dispositivos. La interfaz web, por su parte, se ha optimizado para uso en pantallas de escritorio y, aunque colapsa razonablemente en pantallas estrechas, no se ha trabajado de forma específica el caso móvil. Por último, la evaluación de usabilidad sigue una aproximación cualitativa de muestra reducida (entre cinco y siete participantes), adecuada para detectar la mayoría de los problemas observables pero insuficiente para sostener afirmaciones cuantitativas de carácter estadístico.

6.3. Trabajo futuro

A partir del estado en el que se entrega el sistema, identificamos cinco líneas naturales de continuación. Las dos primeras (asistencia mediante modelos del lenguaje y distribución del motor) tienen un componente de investigación aplicada y abrirían la puerta a un proyecto de mayor envergadura. Las tres restantes (capa de usuario, extensión a nuevos dominios y mejoras de la experiencia de usuario) son de carácter más incremental y podrían abordarse de forma independiente, sin alterar el núcleo del modelo de navegación.

6.3.1. Asistencia mediante modelos del lenguaje

La irrupción de los modelos del lenguaje de gran tamaño (*Large Language Models*) abre una vía interesante para enriquecer el motor sin sustituirlo. En lugar de delegar la navegación en un modelo opaco, se puede emplear el modelo como capa de asistencia: traducción de consultas en lenguaje natural a combinaciones concretas de filtros, sugerencia de filtros pertinentes en función del estado de la exploración,

generación de descripciones sintéticas del conjunto filtrado y explicación de por qué dos entidades aparecen relacionadas a través de un *link* determinado. Este enfoque preserva la agencia del usuario, al mantener visibles las operaciones del motor, y delega únicamente las tareas en las que un modelo del lenguaje aporta valor real.

6.3.2. Distribución del motor sobre múltiples nodos

El motor se ejecuta actualmente como un proceso único frente a una base de datos también única. Para colecciones de mayor escala convendría descomponer el sistema en nodos independientes: particionar el almacenamiento por dominio o por dataset, replicar el cálculo de facetas para balancear carga, e introducir una capa de caché para resultados intermedios frecuentes. Esta línea exigiría revisar la noción de sesión, hoy asociada a un único proceso, y diseñar un protocolo coherente de invalidación que mantenga la sensación de inmediatez sobre la que descansa la experiencia de exploración.

6.3.3. Capa de usuario, persistencia e historial compartido

La introducción de una capa de identidad permitiría conservar el estado entre sesiones, recuperar exploraciones anteriores, marcar conjuntos unión como favoritos y compartir exploraciones mediante *URLs* reproducibles. Esta línea es relativamente independiente del motor y podría desarrollarse en paralelo, aprovechando la separación ya existente entre *back-end* y *front-end*. Su mayor reto no es técnico, sino de diseño de interacción: integrar las nuevas funcionalidades sin sobrecargar la interfaz actual, que ha sido afinada precisamente para ofrecer una primera experiencia ligera y enfocada en la exploración.

6.3.4. Extensión a nuevos dominios y datasets

El sistema se ha pensado desde el principio como agnóstico al dominio. La integración con TMDb y DBLP ha servido para demostrar que el modelo relacional generaliza sin modificaciones sustanciales entre un catálogo audiovisual y una base de publicaciones científicas, pero la elección de futuros dominios sigue siendo el camino más directo para reforzar esa evidencia. Resultarían especialmente interesantes los fondos bibliotecarios completos (con varias clases de objetos coexistiendo bajo un mismo esquema descriptivo), los repositorios documentales institucionales (donde la riqueza de relaciones entre versiones, autores e instituciones pondría a prueba la capa de *links*) y los catálogos de software libre (en los que las relaciones de dependencia y autoría dibujan grafos densos). En cada caso, la incorporación efectiva exige ajustar las heurísticas de identificación de facetas a las particularidades del dominio y, en algunos, ampliar el proceso de carga para acomodar formatos de origen heterogéneos.

6.3.5. Mejoras incrementales en la experiencia de usuario

Sobre la interfaz desarrollada quedan abiertas varias mejoras de carácter incremental que la evaluación de usabilidad ha permitido priorizar: una versión adaptada a dispositivos móviles, internacionalización completa más allá del par castellano-inglés actual, refuerzo de la accesibilidad mediante navegación por teclado y atributos *ARIA*, exportación del historial de navegación y del conjunto unión a formatos reutilizables (*CSV*, *JSON* o referencias bibliográficas), y un trabajo más fino sobre la representación visual del grafo de *links* para hacerla útil con caminos largos.

Introduction

Introduction to the subject area. This chapter contains the translation of Chapter 1.

Conclusions and Future Work

Conclusions and future lines of work. This chapter contains the translation of Chapter 6.

Contribuciones Personales

En caso de trabajos no unipersonales, cada participante indicará en la memoria su contribución al proyecto con una extensión de al menos dos páginas por cada uno de los participantes.

En caso de trabajo unipersonal, elimina esta página en el fichero `TFGTeXiS.tex` (comenta o borra la línea `\include{Capitulos/ContribucionesPersonales}`).

Estudiante 1

Al menos dos páginas con las contribuciones del estudiante 1.

Estudiante 2

Al menos dos páginas con las contribuciones del estudiante 2. En caso de que haya más estudiantes, copia y pega una de estas secciones.

Bibliografía

*Y así, del mucho leer y del poco dormir, se
le secó el cerebro de manera que vino a
perder el juicio.*

(modificar en Cascaras\bibliografia.tex)

Miguel de Cervantes Saavedra

BRITTAİN, J. y DARWIN, I. F. *Tomcat: The Definitive Guide*. O'Reilly Media, 2nd edición, 2007.

BUENDÍA, F., GAYOSO-CABADA, J. y SIERRA, J.-L. Generation of standardized e-learning content from digital medical collections. *Journal of Medical Systems*, vol. 43(7), páginas 188:1–188:8, 2019.

CAMPBELL, B. y GOODMAN, J. M. HAM: A general-purpose hypertext abstract machine. *Communications of the ACM*, vol. 31(7), páginas 856–861, 1988.

COOMBS, J. H., RENEAR, A. H. y DEROSE, S. J. Markup systems and the future of scholarly text processing. *Communications of the ACM*, vol. 30(11), páginas 933–947, 1987.

GAYOSO-CABADA, J., GÓMEZ-ALBARRÁN, M. y SIERRA, J.-L. A smart cache strategy for tag-based browsing of digital collections. En *New Knowledge in Information Systems and Technologies (WorldCIST'19)*, vol. 930 de *Advances in Intelligent Systems and Computing*, páginas 546–555. Springer, 2019.

GAYOSO-CABADA, J., GÓMEZ-ALBARRÁN, M. y SIERRA, J.-L. Reconfigurable metadata schemas with multivalued elements: Towards educational repository management through formal grammars on the Clavy platform. En *Proceedings of the International Symposium on Computers in Education (SIIE 2023)*. IEEE, 2023.

GAYOSO-CABADA, J., RODRÍGUEZ-CEREZO, D. y SIERRA, J.-L. Learning object repositories with dynamically reconfigurable metadata schemata. En *Proceedings of the XVIII International Symposium on Computers in Education (SIIE'16)*. 2016a.

- GAYOSO-CABADA, J., RODRÍGUEZ-CEREZO, D. y SIERRA, J.-L. Multilevel browsing of folksonomy-based digital collections. En *Proceedings of the 17th International Conference on Web Information Systems Engineering (WISE'16)*, vol. 10042 de *LNCIS*, páginas 43–51. Springer, 2016b.
- GAYOSO-CABADA, J., RODRÍGUEZ-CEREZO, D. y SIERRA, J.-L. Browsing digital collections with reconfigurable faceted thesauri. En *Complexity in Information Systems Development. Proceedings of the 25th International Conference on Information Systems Development (ISD'16)*, vol. 22 de *LNISO*, páginas 69–86. Springer, 2017.
- GIFFORD, D. K., JOUVELOT, P., SHELDON, M. A. y O'TOOLE, J. W. Semantic file systems. *SIGOPS Operating Systems Review*, vol. 25(5), páginas 16–25, 1991.
- GODIN, R. y SAUNDERS, G. Lattice model of browsable data space. *Information Sciences*, vol. 40(2), páginas 89–116, 1986.
- GRUZMAN, V. A. y SENICHKIN, V. I. Hypermedia models. *Automation and Remote Control*, 2000. Survey of hypertext and hypermedia reference models, including HAM, Trellis, Dexter and Amsterdam.
- HALASZ, F. G. y SCHWARTZ, M. The dexter hypertext reference model. *Communications of the ACM*, vol. 37(2), páginas 30–39, 1994.
- HANSER, E. Scrum. En *Agile Processes – Agile Teams*, páginas 61–78. Springer Berlin, Heidelberg, 2026.
- HUNTER, J. y CRAWFORD, W. *Java Servlet Programming*. O'Reilly Media, 2nd edición, 2001.
- IEEE LEARNING TECHNOLOGY STANDARDS COMMITTEE. Ieee standard for learning object metadata. Informe Técnico 1484.12.1-2002, IEEE, 2002.
- IMS GLOBAL LEARNING CONSORTIUM. Ims content packaging information model, version 1.1.4. Informe técnico, IMS Global, 2004.
- KATZ, U., LEVY, M. y GOLDBERG, Y. Knowledge navigator: LLM-guided browsing framework for exploratory search in scientific literature. En *Findings of the Association for Computational Linguistics: EMNLP 2024*. 2024.
- VEPSÄLÄINEN, J. Revisiting hypermedia, the forgotten web application development paradigm. *IEEE Access*, 2024.

Guion de Evaluación de Usabilidad

Este apéndice reproduce, en su forma definitiva, el guion utilizado en la evaluación de usabilidad del sistema descrita en el capítulo 5. Se conserva la estructura original con la que se llevaron a cabo las cinco sesiones, de manera que pueda reutilizarse o replicarse sin modificaciones por terceros que deseen comparar los resultados.

A.1. Información general

Versión	1.0
Duración por sesión	35–45 minutos
Participantes objetivo	5–7
Método	<i>Think-aloud</i> + observación directa + cuestionarios pre/post

A.2. Información para el facilitador

A.2.1. Objetivos

- Facilidad de aprendizaje en la primera toma de contacto.
- Eficiencia al realizar tareas comunes de búsqueda y navegación.
- Comprensión de los conceptos clave (filtros *include/exclude*, *links*, *Union Set*, historial, *restore* vs *goback*).
- Recuperación ante errores y manejo de acciones destructivas (*Exit*, *Change collection*).
- Satisfacción subjetiva (SUS).

A.2.2. Perfil del participante

Cualquier persona con experiencia básica en navegadores web. No es necesario conocimiento previo del dominio del *dataset* cargado.

A.2.3. Material necesario

- Ordenador con la aplicación arrancada en una pestaña.
- *Dataset* cargado con varias colecciones, metadatos, referencias y al menos un *link* entre colecciones.
- Cronómetro.
- Cuestionario SUS impreso o en formato digital.
- Hoja de consentimiento informado.

A.2.4. Checklist previo a la sesión

- Verificar que la aplicación arranca sin errores en consola.
- Asegurar que existen al menos dos *datasets* distintos.
- Asegurar que el *backend* responde (probar `/datasets` desde la URL).
- Tener a mano la URL exacta de la aplicación.

A.2.5. Protocolo general

- No interrumpir al participante salvo bloqueo total superior a 90 segundos.
- No dar pistas. Si pregunta cómo hacer algo, devolver con «¿Qué harías tú?» o «¿Por dónde empezarías?»
- Tomar notas en silencio. Marcar tiempo de inicio y fin de cada tarea.
- Mantener un tono neutro: ni felicitar éxitos ni lamentar fracasos.
- Si el participante quiere abandonar, agradecer y cerrar sin presionar.
- Recordarle (una o dos veces si hace falta) que piense en voz alta.

A.3. Introducción al participante (texto literal)

«Hola, muchas gracias por participar. Vamos a estar aquí unos 40 minutos.

El objetivo de esta sesión es evaluar la aplicación, NO evaluarte a ti. No hay respuestas correctas o incorrectas. Si algo no funciona o te resulta confuso, es información muy valiosa para nosotros: significa que tenemos algo que mejorar.

Te voy a pedir que realices una serie de tareas usando la aplicación. Mientras las haces, me gustaría que pensaras en voz alta: di lo que estás mirando, lo que esperas que pase, lo que te confunde, lo que te gusta o no

te gusta. No tengas miedo de criticar; al contrario, cuanto más honesto seas, más útil será tu *feedback*.

Yo voy a estar tomando notas y manteniéndome en silencio la mayor parte del tiempo. Si te bloqueas, intenta resolverlo a tu manera. Solo intervendré si llevas mucho rato sin saber por dónde seguir. Puedes parar la sesión en cualquier momento sin necesidad de explicar nada.

¿Tienes alguna duda antes de empezar?»

A.4. Consentimiento informado

«Antes de empezar te pido que leas y firmes este consentimiento. Resumido:

- Los datos recogidos en esta sesión serán usados únicamente para mejorar la aplicación.
- Tu identidad no aparecerá en ningún informe.
- Podemos grabar audio y vídeo si das tu permiso explícito; la grabación se borrará una vez analizada.
- Puedes retirarte en cualquier momento sin consecuencias.

¿Estás de acuerdo?»

Casillas a marcar: consentimiento verbal, consentimiento por escrito, permiso de grabación (S / N).

A.5. Cuestionario previo (5 minutos)

1. ¿Cuál es tu rango de edad? 18–24 / 25–34 / 35–44 / 45–54 / 55+
2. ¿Cuál es tu ocupación o área de estudios?
3. ¿Con qué frecuencia usas aplicaciones web? Varias horas al día / A diario, ocasionalmente / Varias veces por semana / Esporádicamente.
4. ¿Has usado antes herramientas de búsqueda con filtros (Amazon, Idealista, bibliotecas digitales, archivos online)? Sí, con frecuencia / Sí, alguna vez / No.
5. ¿Te dedicas o has estudiado algo relacionado con catalogación, bibliotecas, archivos, museos, bases de datos o similares? Sí, profesionalmente / Sí, en estudios / No.
6. En una escala del 1 al 5, ¿cómo de cómodo te sientes aprendiendo una herramienta nueva por tu cuenta? 1 (muy incómodo) – 5 (muy cómodo).

A.6. Instrucciones sobre el método (2 minutos)

«Antes de empezar las tareas:

- La aplicación ya está abierta en una pestaña del navegador. Empezar siempre desde la pantalla de selección de *dataset*.
- Te leeré cada tarea en voz alta y te daré un escenario realista. No te diré los pasos exactos: tú decides cómo resolverla.
- Recuerda: piensa en voz alta. Cuanto más digas, mejor.
- Cada tarea tiene un tiempo límite aproximado. Si te bloqueas no pasa nada: pasamos a la siguiente.

¿Listo para empezar?»

A.7. Tareas

Para cada tarea el facilitador anota tiempo de inicio y fin, completitud (*Éxito / Parcial / Fallo*), errores observados, caminos tomados y comentarios espontáneos. Tras cada tarea se pregunta: «En una escala del 1 (muy difícil) al 7 (muy fácil), ¿cómo de fácil te ha resultado?» (SEQ) y se invita a comentar.

A.7.1. Tarea 1 – Empezar a navegar (límite: 2 min)

Escenario: «Acabas de abrir la aplicación. Quieres empezar a explorar uno de los *datasets* disponibles. Elige el *dataset* que prefieras y, dentro de él, una colección para empezar.»

Criterio de éxito: llega a la vista de navegación con los paneles Filtros, *Links*, *Filtered Collection* y *Union Set* visibles.

Observar: ¿Entiende el flujo *Dataset* → Colección? ¿Lee los nombres antes de elegir o pulsa al azar? ¿Reconoce el *breadcrumb* superior tras entrar?

A.7.2. Tarea 2 – Filtrar por metadatos (límite: 3 min)

Escenario: «Imagina que sólo te interesan las entidades que cumplan un criterio concreto. Aplica un filtro de metadatos para reducir el conjunto de resultados.»

Criterio de éxito: se aplica al menos un filtro; el conteo de *Filtered Collection* se reduce; aparece la etiqueta verde con «+» en la sección de Metadata.

Observar: ¿Encuentra los desplegados sin ayuda? ¿Usa la búsqueda dentro del *combobox* o se desplaza? ¿Comprende que la etiqueta verde con «+» representa un filtro activo?

A.7.3. Tarea 3 – Combinar varios filtros (límite: 3 min)

Escenario: «Añade dos filtros más, combinando atributos diferentes. Comprueba cómo el conteo se va reduciendo.»

Criterio de éxito: se aplican al menos tres filtros activos en total.

Observar: ¿Nota que los valores ya elegidos desaparecen del desplegable? ¿Le sorprende algún comportamiento?

A.7.4. Tarea 4 – Filtros de exclusión (límite: 3 min)

Escenario: «Ahora quieres lo contrario: hay un valor concreto que NO quieres ver en los resultados. Excluye una opción del conjunto.»

Criterio de éxito: cambia el modo a *Exclude* y aplica al menos un filtro de exclusión (etiqueta roja con símbolo menos).

Observar: ¿Encuentra el toggle *Include/Exclude*? ¿Entiende la previsualización al pasar el ratón? ¿Sabe diferenciar visualmente entre filtro de inclusión y exclusión?

A.7.5. Tarea 5 – Filtros de referencia (límite: 3 min)

Escenario: «Además de los metadatos, hay filtros basados en referencias entre entidades de otras colecciones. Aplica un filtro de referencia para reducir el conjunto.» (Si no hay referencias en la colección actual, el facilitador omite la tarea y lo anota.)

Criterio de éxito: se aplica al menos un filtro de referencia.

Observar: ¿Distingue entre la sección Metadata y *References*? ¿Le resulta clara la sintaxis «Colección → motivo»?

A.7.6. Tarea 6 – Quitar filtros (límite: 2 min)

Escenario: «Quita primero solo uno (el que prefieras) y comprueba que el conjunto se actualiza. Después quita todos los filtros activos a la vez.»

Criterio de éxito: elimina un filtro individual con la «x» de la etiqueta y después usa *Clear all* para vaciar el resto.

Observar: ¿Ve la «x» de cada etiqueta sin ayuda? ¿Encuentra el botón *Clear all* sin pista? ¿Le resulta claro que el botón se desactiva cuando no hay filtros?

A.7.7. Tarea 7 – Seguir un *link* a otra colección (límite: 2 min)

Escenario: «En el panel de *Links* hay accesos directos a otras colecciones relacionadas. Sigue uno de los *links* para navegar a otra colección.»

Criterio de éxito: pulsa un *link*, ve que el *breadcrumb* cambia y aparece el contenido de la nueva colección.

Observar: ¿Le queda claro que ha cambiado de colección? ¿Comprende el papel del *breadcrumb* superior?

A.7.8. Tarea 8 – Ver el historial de navegación (límite: 2 min)

Escenario: «Has saltado entre colecciones. Te gustaría ver visualmente el camino que has seguido. Encuentra la forma de mostrarlo.»

Criterio de éxito: pulsa «History » en el panel de *Links* y ve el grafo horizontal de cajas conectadas por flechas. Reconoce el nodo actual.

Observar: ¿Encuentra el botón *History* o lo busca por otros sitios? ¿Reconoce que los nodos representan colecciones y las flechas *links*? ¿Vuelve a cerrar el historial?

A.7.9. Tarea 9 – Volver atrás un paso (límite: 1 min)

Escenario: «Quieres deshacer el último salto y volver a la colección donde estabas antes.»

Criterio de éxito: pulsa *Go Back* en la barra superior y vuelve a la colección previa.

Observar: ¿Distingue *Go Back* del botón «atrás» del propio navegador? ¿Diferencia *Go Back* de *Restore*?

A.7.10. Tarea 10 – Ver el detalle de una entidad (límite: 2 min)

Escenario: «Elige cualquier entidad de la colección y abre su ficha completa. Echa un vistazo a la información que contiene.»

Criterio de éxito: abre el *modal* de detalle y ve metadatos, referencias y recursos si los hay.

Observar: ¿Le sorprende el *modal* o es lo que esperaba? ¿Encuentra cómo cerrarlo («x», *click* fuera, ESC)? ¿Le resulta clara la información mostrada?

A.7.11. Tarea 11 – Guardar en *Union* y cambiar de colección (límite: 4 min)

Escenario: «Tienes ahora un conjunto filtrado interesante que querrías conservar mientras exploras OTRA colección en paralelo. Guarda los resultados actuales en el conjunto *Union* y elige otra colección para seguir explorando.»

Criterio de éxito: pulsa «+ Add current» en el panel *Union Set*, elige otra colección en el selector que aparece y vuelve a la vista de navegación. Comprueba que el contador de *Union* es mayor que cero y que las entidades anteriores aparecen ahí.

Observar: ¿Entiende el concepto de *Union* sin pistas? ¿Ve el *toast* verde de confirmación? ¿Reconoce que la colección principal ha cambiado pero *Union* conserva lo anterior? ¿Localiza fácilmente el botón «+ Add current»?

A.7.12. Tarea 12 – Reordenar paneles (límite: 2 min)

Escenario: «Te gustaría tener la lista de *Links* arriba del todo, o reordenar cualquier panel a tu gusto. Intenta mover los paneles.»

Criterio de éxito: arrastra al menos un panel a una nueva posición y comprueba que el cambio se mantiene tras refrescar la página (F5).

Observar: ¿Descubre que las cabeceras son arrastrables? ¿Cuánto tarda en darse cuenta? ¿Hay gestos fallidos (*click* vs *drag*)? ¿Le sorprende que se mantenga al refrescar?

A.7.13. Tarea 13 – Cambiar de colección (sin *Union*) (límite: 2 min)

Escenario: «Quieres empezar de cero en una colección distinta, sin guardar nada ni mantener los filtros actuales. Cambia de colección.»

Criterio de éxito: pulsa *Change collection*, lee el diálogo de confirmación y acepta. Selecciona una nueva colección. Filtros e historial quedan vacíos.

Observar: ¿Lee el mensaje de confirmación antes de aceptar? ¿Diferencia *Change collection* del flujo de *Add to Union* del paso anterior? ¿Le parece adecuada la diferenciación visual (color ámbar) frente a *Goback* y *Restore* (azul y gris)?

A.7.14. Tarea 14 – Salir del modo navegación (límite: 1 min)

Escenario: «Has terminado. Sal del modo navegación y vuelve a la pantalla inicial de selección de *datasets*.»

Criterio de éxito: pulsa el botón «x» de la barra superior, confirma el diálogo, vuelve a la pantalla de selección de *datasets*.

Observar: ¿Encuentra el botón «x» fácilmente? ¿Le sorprende el diálogo de confirmación? ¿Diferencia «salir» de «cambiar de colección»?

A.8. Cuestionario posterior (SUS, 5 minutos)

Indica tu grado de acuerdo con cada afirmación en una escala de 1 a 5, donde 1 = totalmente en desacuerdo y 5 = totalmente de acuerdo.

1. Creo que me gustaría usar este sistema con frecuencia.
2. He encontrado el sistema innecesariamente complejo.
3. He pensado que el sistema era fácil de usar.
4. Creo que necesitaría ayuda de un técnico para poder usar este sistema.
5. He encontrado que las distintas funciones del sistema estaban bien integradas.
6. He pensado que había demasiada inconsistencia en el sistema.

7. Imagino que la mayoría de la gente aprendería a usar este sistema muy rápidamente.
8. He encontrado el sistema muy complicado de usar.
9. Me he sentido muy seguro/a usando el sistema.
10. He necesitado aprender muchas cosas antes de poder empezar a usar este sistema.

Cálculo de la puntuación SUS (uso interno):

- Ítems impares (1, 3, 5, 7, 9): restar 1 al valor.
- Ítems pares (2, 4, 6, 8, 10): restar el valor a 5.
- Sumar los diez resultados y multiplicar por 2,5; el resultado queda en el rango 0–100. Umbral aceptable: ≥ 68 . Excelente: ≥ 80 .

A.9. Preguntas abiertas de cierre (10 minutos)

1. ¿Qué es lo que más te ha gustado del sistema?
2. ¿Qué es lo que más te ha confundido o frustrado?
3. Si pudieras cambiar UNA cosa, ¿cuál sería?
4. ¿Hubo algún momento en el que no sabías qué hacer?
5. ¿Cómo describirías el sistema a un compañero/a?
6. ¿Hay alguna funcionalidad que esperabas encontrar y no está?
7. ¿Te ha resultado claro el papel del *Union Set*? Explica con tus palabras para qué sirve.
8. ¿Te resultó claro lo que hacía cada uno de los botones de la barra superior (*X*, *Change collection*, *Go Back*, *Restore*)?

A.10. Cierre y agradecimiento

«Hemos terminado. Muchas gracias por dedicar este tiempo. Tus comentarios son muy útiles para mejorar la aplicación. ¿Tienes alguna pregunta para mí antes de irnos?»

A.11. Plantilla de observación del facilitador

A.11.1. Datos de la sesión

- ID participante.
- Fecha y hora de inicio y fin.
- Facilitador y, en su caso, observador.
- Indicador de grabación (S/N).

A.11.2. Registro por tarea

Para cada una de las catorce tareas se cumplimenta una fila con: tiempo de inicio, tiempo de fin, completitud (*Éxito / Parcial / Fallo*), valor SEQ y notas sobre errores observados o caminos alternativos.

A.11.3. Incidencias destacables y citas literales

Dos tablas adicionales recogen, respectivamente, las incidencias críticas o momentos de bloqueo (con marca temporal, tarea afectada y descripción) y las citas literales del participante con marca temporal. Esta última información alimenta el análisis cualitativo posterior.

A.11.4. Conclusiones del facilitador

Tras la sesión, el facilitador rellena un breve bloque libre con los patrones observados, las hipótesis surgidas y los cambios sugeridos sobre la aplicación.

A.12. Notas metodológicas

A.12.1. Número de participantes

Nielsen (2000) sostiene que con cinco participantes se descubre aproximadamente el 85 % de los problemas de usabilidad. Para esta evaluación se proponen entre cinco y siete para absorber la variabilidad entre perfiles.

A.12.2. Perfil equilibrado recomendado

- Una persona con experiencia en catalogación, archivos o bibliotecas.
- Una persona sin experiencia previa en búsqueda facetada.
- Una persona del entorno académico (potencial usuario real).
- Dos o tres perfiles generalistas (usuarios web habituales).

A.12.3. Análisis posterior

1. Calcular SUS por participante y media (umbral aceptable: ≥ 68).
2. Para cada tarea, calcular tasa de éxito, tiempo medio (mediana más robusta para muestras pequeñas) y SEQ medio.
3. Catalogar incidencias por gravedad según la escala de Nielsen: 0 = no es problema, 1 = cosmético, 2 = menor, 3 = mayor, 4 = catastrófico.
4. Priorizar correcciones por gravedad $\times$ frecuencia $\times$ impacto en tareas críticas.

A.12.4. Criterios de éxito por tarea

- **Éxito:** el participante completa el criterio de éxito sin ayuda y dentro del límite de tiempo.
- **Parcial:** completa la tarea con ayuda o supera el tiempo límite.
- **Fallo:** abandona o no consigue completarla.

A.12.5. Variables a controlar entre sesiones

- Mismo *dataset* y misma colección inicial.
- Misma opciones de filtro disponibles.
- Mismo orden de paneles al arrancar (preferencia local limpia).
- Misma versión del software (*tag* o *commit* fijo).
- Mismo dispositivo si es posible (la resolución afecta al *layout*).

Integración e ingesta de fuentes de datos

La aplicación está diseñada para operar sobre datos provenientes de fuentes externas heterogéneas, lo que obliga a disponer de una capa de integración que medie entre cada origen y el modelo relacional descrito en el cuerpo de la memoria. Este apéndice documenta el conjunto de scripts que conforman dicha capa, distinguiendo entre la descarga remota, el procesado y la carga final en PostgreSQL. Se detallan los dos orígenes incorporados a día de hoy, TMDb y DBLP, y se describe el script de poblado, común a ambos, que materializa los objetos en las cinco tablas del esquema.

B.1. Pipeline general

La integración de cada fuente sigue un esquema de tres etapas diferenciadas. La primera etapa, de *extracción*, obtiene los datos brutos desde el sistema de origen, ya sea mediante una API REST autenticada o a través de volcados públicos comprimidos. La segunda etapa, de *procesado*, transforma esos datos en una representación intermedia uniforme, en formato JSON Lines (un objeto JSON por línea), que codifica de forma homogénea el modelo común de colecciones, entidades, metadatos, referencias y recursos. La tercera etapa, de *ingesta*, vuelca ese JSONL en la base de datos PostgreSQL siguiendo el esquema descrito en el capítulo de desarrollo.

Esta separación tiene dos consecuencias prácticas. Por un lado, permite incorporar nuevas fuentes implementando únicamente las dos primeras etapas, ya que la tercera es agnóstica al origen siempre que el JSONL generado se ajuste al formato esperado. Por otro, hace que los archivos intermedios actúen como caché reutilizable, evitando rehacer descargas o procesados costosos cuando solo se modifica una etapa posterior.

Modelo de representación intermedio

La pieza que articula esta arquitectura es el formato del JSONL intermedio. El archivo contiene un objeto JSON por línea (lo que permite procesarlo en streaming sin cargarlo entero en memoria) y se estructura en dos partes. La primera línea es un objeto de cabecera con la lista de colecciones presentes en el dataset (nombre e identificador de cada una). Las líneas siguientes son entidades, cada una con

cinco campos: `id` y `collection_id` que identifican la entidad dentro de su colección, `metadata` con los atributos filtrables agrupados por clave, `references` con los vínculos tipados hacia otras entidades del mismo archivo y `contents` con el texto descriptivo y los recursos externos. El Código B.1 reproduce un fragmento ilustrativo del formato.

Código B.1: Esquema del JSONL intermedio

```

1 {"collections": [{"name": "Movies", "id": 1}, {"name": "People",
2   "id": 3}]}
3 {
4   "id": 550,
5   "collection_id": 1,
6   "metadata": {
7     "Genres": ["Drama"],
8     "Release date (Year)": ["1990-1999"],
9     "Runtime": ["120-149"]
10  },
11  "references": [
12    {"reason": "Director", "reference_id": 7467, "
13      reference_collection_id": 3}
14  ],
15  "contents": {
16    "Name": "Fight Club",
17    "Overview": "...",
18    "_resources": [
19      {"type": "image", "label": "Poster", "url": "https://
20        image.tmdb.org/..."}
  ]
}

```

Este formato cumple varios objetivos a la vez. Es directamente proyectable al esquema relacional: cada clave del bloque `metadata` produce una o más filas en la tabla `metadata`, cada elemento de `references` se convierte en una fila de `reference`, y el bloque `contents` se almacena tal cual en la columna `JSONB` de `entities`. Es también extensible, ya que la ausencia de un esquema fijo en los bloques `metadata` y `contents` permite que cada colección exponga los campos propios de su dominio sin necesidad de coordinarse con las demás. Y, por último, es independiente de la fuente: tanto el procesamiento de TMDB como el de DBLP producen archivos con esta misma estructura, lo que hace que la etapa de ingesta sea un único script común a todos los orígenes.

B.2. TMDB

The Movie Database (TMDB) es una base de datos colaborativa de información cinematográfica y televisiva, mantenida por una comunidad amplia de contribuyentes y consumida por servicios de terceros a través de una API REST pública. Su

catálogo incluye decenas de miles de películas, series, personas (actores, directores, equipo técnico), productoras y cadenas de televisión, todos enlazados entre sí por relaciones tipadas. Esa combinación de tamaño, riqueza de metadatos y densidad de relaciones la convertía en el banco de pruebas ideal para un sistema cuyo objetivo es precisamente explorar grafos de información mediante metadatos y referencias.

La extracción se apoya en un par de scripts en Python independientes, uno por etapa. El primero descarga los identificadores diarios publicados por TMDB y, a continuación, consulta el endpoint de detalle correspondiente para cada uno, escribiendo el JSON devuelto en un archivo JSONL por colección. El acceso a la API requiere una clave personal, gestionada como variable de entorno `TMDB_API_KEY` definida en `.env.thirdparty` para mantenerla fuera del control de versiones. La descarga utiliza un pool de corrutinas asíncronas con un limitador que respeta el ritmo admitido por la API y un mecanismo de reanudación que evita volver a pedir identificadores ya presentes en el archivo de salida, lo que hace la descarga idempotente ante interrupciones.

El interés de trabajar con un subconjunto

Operar con el catálogo completo durante el desarrollo no era práctico. Las decenas de miles de entidades por colección, multiplicadas por las referencias cruzadas entre ellas, habrían provocado tiempos de poblado de la base de datos y de respuesta del frontend prohibitivos para una iteración rápida. Más relevante todavía, una parte significativa del catálogo corresponde a contenidos poco conocidos o con muy pocos votos, cuya presencia en la exploración aporta ruido sin mejorar el valor demostrativo del sistema.

La solución adoptada fue puntuar cada entidad mediante un promedio bayesiano sobre el número de votos y la nota media, y retener únicamente las primeras `TOP_K` entidades de cada colección. Con `TOP_K = 500`, el dataset resultante (denominado `dataset_top500.jsonl`) cabe holgadamente en memoria, se ingesta en pocos minutos y mantiene precisamente los títulos sobre los que el usuario tiene más probabilidades de querer navegar. El mismo script admite ampliar el subconjunto a varios miles de entidades sin tocar código, ajustando únicamente el parámetro de corte, lo que permite escalar la prueba con escaso esfuerzo.

Configuración por colección

El procesado se parametriza mediante `scripts/TMDB/format.json`, un archivo declarativo que describe las cinco colecciones consideradas (películas, series, personas, compañías y cadenas) y, para cada una, los campos que deben extraerse del JSON crudo y cómo se clasifican dentro del modelo. La configuración define qué campos son *metadatos* (filtrables), *contenidos* (texto descriptivo) o *recursos* (URLs externas, como pósteres o enlaces a IMDb), así como el tipo de cada metadato: cadena, numérico, fecha, ubicación o codificado. La discretización en rangos para los campos numéricos y de fechas, motivada en el capítulo de desarrollo, se controla también desde este archivo mediante un tamaño de bucket por campo.

Centralizar la configuración en este archivo permite extender el dataset con nue-

vos campos sin tocar el código Python, y facilita además que el mismo motor de procesado se aplique a fuentes distintas con un esquema diferente pero un objetivo análogo, idea que se materializa en la integración con DBLP.

B.3. DBLP

La segunda fuente integrada es DBLP, un servicio bibliográfico mantenido por la Universidad de Tréveris que cataloga la producción científica en informática: artículos en revistas y conferencias, monografías, autores y lugares de publicación. A diferencia de TMDb, DBLP no expone una API de consulta detallada; en su lugar, publica un volcado RDF completo en formato N-Triples (`dblp.nt.gz`) que, en su variante íntegra, ronda decenas de gigabytes una vez descomprimido. La fuente es interesante para este proyecto por dos motivos: aporta un dominio ortogonal al cinematográfico, lo que valida la generalidad del modelo, y permite construir un dataset alineado con la comunidad universitaria filtrando por afiliación.

El procesado se ha implementado en Go por razones de rendimiento: la lectura completa del volcado debe poder hacerse en tiempos razonables y con un consumo de memoria acotado. La estrategia se basa en una primera pasada lineal sobre todas las tripletas que asigna identificadores enteros a las entidades reconocidas y, en paralelo, reparte las tripletas en sesenta y cuatro archivos de partición usando una función hash sobre el sujeto. Esto garantiza que toda la información de una misma entidad acabe en la misma partición y permite procesar el grafo en bloques que sí caben en memoria, sin necesidad de ordenar el archivo completo. Una segunda pasada recorre las particiones, construye los objetos JSONL aplicando la misma estructura declarativa de `format.json` que TMDb y elimina cada partición inmediatamente después de procesarla para acotar el uso de disco.

Un detalle relevante es el filtrado opcional por afiliación a la UCM: cuando está activo, el script conserva únicamente personas cuya información contiene alguna referencia a `.ucm.es` y, en una fase de limpieza posterior, descarta las obras que ya no quedan ligadas a ninguna persona conservada. Este filtro reduce drásticamente el tamaño del dataset y produce un grafo autoconsistente alineado con el caso de uso académico previsto. La fase de limpieza realiza además una poda de referencias colgantes (apuntando a entidades que dejaron de formar parte del subconjunto), de modo que el JSONL final puede ingerirse directamente sin pasos de validación adicionales.

B.4. Ingesta en PostgreSQL

La etapa final del pipeline es común a ambas fuentes y se implementa en `scripts/populate_db_jsonl.py`. El script consume un archivo JSONL cuyo primer registro describe las colecciones y los registros siguientes son objetos individuales, y los inserta en las cinco tablas del esquema (`datasets`, `collections`, `entities`, `metadata` y `reference`).

La carga se realiza en dos pasadas. La primera pasada inserta las entidades y

construye en memoria un diccionario que asocia los identificadores originales del JSON con los identificadores internos asignados por PostgreSQL. Esta tabla de equivalencias es la que permite, en la segunda pasada, traducir las referencias del archivo en referencias internas entre filas de la tabla `entities`, evitando así las referencias colgantes y garantizando la integridad del grafo. Durante la carga, las inserciones se agrupan en lotes para reducir el número de viajes a la base de datos, y los índices secundarios se desactivan temporalmente y se reconstruyen al finalizar, lo que acelera de forma notable el poblado inicial.

La conexión a la base de datos se configura íntegramente mediante las variables de entorno `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASS` y `DB_PORT`, definidas en `.env.db`. El script invoca `psycopg2` sin credenciales embebidas, lo que mantiene el archivo libre de información sensible y facilita reutilizarlo entre entornos de desarrollo y producción. El nombre del dataset y la ruta al JSONL pueden pasarse como argumentos de la línea de comandos, lo que permite cargar varios datasets independientes sobre la misma base de datos.

B.5. Ejecución completa desde cero

Para reproducir el pipeline completo desde un repositorio recién clonado los pasos son los descritos en el Código B.2. Se asume que la base de datos ya está arrancada (véase el Apéndice C) y que las variables de entorno están definidas en `.env.db` y `.env.thirdparty`.

Código B.2: Ejecución del pipeline completo

```
1 # 1. Instalar dependencias Python
2 cd scripts
3 pip install -r requirements.txt
4
5 # 2. TMDb: descargar y procesar
6 python TMDb/pull.py
7 python TMDb/process.py
8
9 # 3. (Alternativa) DBLP: procesar el volcado
10 cd DBLP
11 go run main.go dblp.nt.gz format.json
12 cd ..
13
14 # 4. Ingesta en PostgreSQL
15 python populate_db_jsonl.py "./TMDb/dataset_top500.jsonl" "
    TMDb Top 500"
```


Despliegue del sistema

El sistema está pensado para operar como un conjunto de servicios independientes: una base de datos PostgreSQL, un backend Java sobre Apache Tomcat y un frontend estático servido a través de Nginx. Cuando se quiere compartir una instancia con terceros (por ejemplo, para la defensa del proyecto o para demostraciones puntuales) se añade un cuarto componente, un túnel `cloudflared`, que expone la aplicación al exterior sin necesidad de abrir puertos en el anfitrión. Este apéndice resume las decisiones de despliegue y los modos de arranque previstos. Los pasos exactos, junto con sus *troubleshooting* habituales, están documentados en el `README.md` de la raíz del repositorio¹, al que se remite para la secuencia operativa concreta.

C.1. Modo de desarrollo

Durante el desarrollo se priorizan dos cosas: el aislamiento entre componentes y la velocidad de iteración. Por eso solo PostgreSQL se ejecuta como contenedor, mientras que backend y frontend se levantan directamente en la máquina del desarrollador. El contenedor de PostgreSQL se arranca con `docker-compose.dev.yml`, que monta el directorio `database/` sobre el punto de inicialización de la imagen oficial, lo que provoca que el esquema (descrito más adelante) se cree automáticamente en el primer arranque sin pasos manuales.

El backend se compila con Maven y se despliega en una instalación local de Apache Tomcat 10.1 a través de Eclipse, lo que aporta recompilación incremental y depuración paso a paso. Las credenciales de la base de datos se inyectan al servidor como variables de entorno, leyendo los nombres definidos en `.env.db` (`DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASS` y `DB_PORT`), sin que valor alguno quede embebido en el código ni en el WAR. El frontend, por su parte, se sirve con el servidor de desarrollo de Vite, que ofrece recarga en caliente sobre los cambios y permite apuntar a un backend remoto modificando únicamente `VITE_API_BASE_URL` en `.env.development.local`.

Esta separación introduce una particularidad: durante el desarrollo, frontend y backend se sirven en orígenes distintos (puertos diferentes del mismo `localhost`), lo

¹<https://github.com/hibjan/TFG/blob/main/README.md>

que obliga a configurar explícitamente CORS y a marcar las cookies de sesión como `SameSite=None`. La razón está discutida en el cuerpo del capítulo de desarrollo; el descriptor `web.xml` y el `context.xml` del backend incorporan esta configuración para que el navegador acepte la cookie de sesión en peticiones *cross-site*. En el modo compartido, como se verá a continuación, el problema desaparece porque ambos servicios quedan tras un mismo origen.

C.2. Instancia compartida con túnel Cloudflare

Para compartir una instancia funcional con un público externo sin exigirle instalar nada, el stack completo se levanta con un único comando de `docker compose` sobre `docker-compose.prod.yml`, que construye y orquesta los cuatro servicios y los conecta en una red interna privada. La elección de empaquetar también el backend y el frontend, y no solo la base de datos, responde a dos motivos. Por un lado, evita depender del entorno del anfitrión (versiones de Java, de Node, de Tomcat) y hace el despliegue reproducible. Por otro, permite unificar el frontend estático y la API tras un mismo origen mediante un reverse proxy de Nginx, lo que elimina por completo las complicaciones de CORS y cookies que sí están presentes durante el desarrollo.

El servicio de frontend utiliza una imagen multi-etapa que primero ejecuta `npm run build` y a continuación copia los artefactos a una imagen `nginx:alpine` con una configuración personalizada. Esa configuración sirve los archivos estáticos en la raíz y hace de proxy hacia el contenedor del backend para las rutas bajo `/backend/`, de modo que el navegador percibe siempre un único dominio. El servicio de backend sigue un esquema análogo: una primera etapa compila el WAR con Maven y una segunda etapa lo despliega sobre `tomcat:10.1-jdk17`, aplicando además el ajuste de `CookieProcessor` necesario para sesiones *cross-site*.

El componente diferencial es `cloudflared`, que abre un túnel saliente hacia la red de Cloudflare y obtiene a cambio una URL pública en el subdominio `*.try cloudflare.com`. Esto permite exponer la aplicación al exterior sin abrir puertos en el cortafuegos del anfitrión, sin configurar DNS y sin gestionar certificados TLS, ya que el cifrado y la terminación los proporciona Cloudflare en su extremo del túnel. La URL concreta se extrae de los logs del contenedor tras el arranque y puede compartirse directamente con quien deba acceder a la instancia. La topología resultante, presentada de forma coherente con la Figura 3.2 del capítulo de arquitectura, se recoge en la Figura C.1.

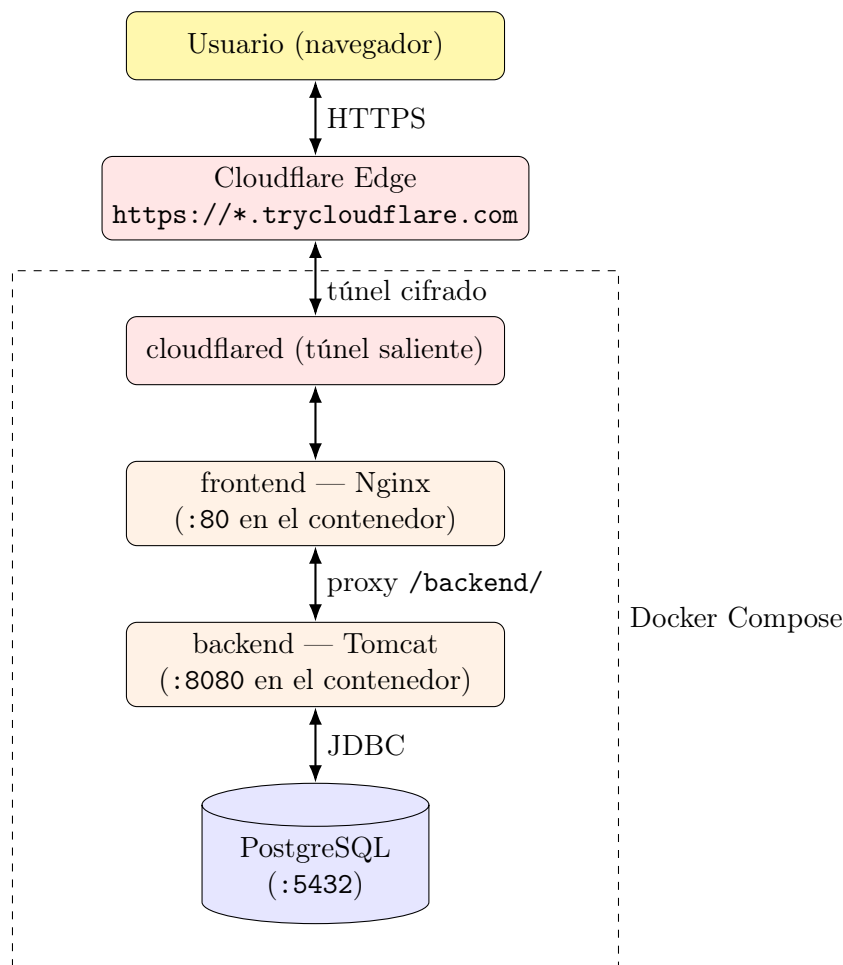


Figura C.1: Topología de la instancia compartida con túnel Cloudflare.

C.3. Base de datos e ingesta inicial

El esquema relacional, descrito en el cuerpo de la memoria, se materializa en `database/01_schema.sql`, que crea las cinco tablas (`datasets`, `collections`, `entities`, `metadata` y `reference`) junto con sus claves foráneas e índices. Este archivo se monta como volumen en el directorio de inicialización de la imagen oficial de PostgreSQL, lo que provoca que se ejecute automáticamente la primera vez que el contenedor arranca con un volumen vacío. La convención de prefijos numéricos en los nombres de los scripts SQL permite añadir migraciones futuras sin alterar el mecanismo: PostgreSQL los ejecutará en orden lexicográfico durante el mismo arranque inicial.

Una vez creado el esquema, la carga de un dataset concreto se realiza invocando `populate_db_jsonl.py` desde la máquina anfitriona, según se detalla en el Apéndice B. El script publica los datos a través del puerto expuesto por el contenedor de PostgreSQL, leyendo las credenciales de `.env.db`. La separación entre creación del esquema (automática en el primer arranque) y carga de datasets (manual, posterior) hace el sistema flexible: la base de datos puede convivir con varios datasets independientes, cargados en momentos distintos, y un fallo durante la ingesta no compromete la estructura ya validada.

Si en algún momento se desea regenerar el estado de la base de datos por comple-

to, basta con detener el contenedor descartando el volumen asociado, lo que provoca que el siguiente arranque vuelva a ejecutar `01_schema.sql` sobre un volumen vacío. Esta operación elimina todos los datasets cargados previamente y debe usarse, por tanto, con cuidado; el `README.md` la documenta como recurso de *troubleshooting* para los casos en los que el estado de la base de datos haya quedado inconsistente.

Este texto se puede encontrar en el fichero Cascaras/fin.tex. Si deseas eliminarlo, basta con comentar la línea correspondiente al final del fichero TFGTeXiS.tex.

*–¿Qué te parece desto, Sancho? – Dijo Don Quijote –
Bien podrán los encantadores quitarme la ventura,
pero el esfuerzo y el ánimo, será imposible.*

*Segunda parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

*–Buena está – dijo Sancho –; fírmela vuestra merced.
–No es menester firmarla – dijo Don Quijote–,
sino solamente poner mi rúbrica.*

*Primera parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

